
네트워크 Basic

인터넷네트워크와 TCP/IP의 이해

1장 컴퓨터 네트워크와 인터넷

1.1 인터넷이란 무엇인가

1.2 네트워크의 가장자리(edge)

1.3 네트워크의 코어 (core)

1.4 네트워크 접속과 매체

1.5 ISP와 인터넷 백본

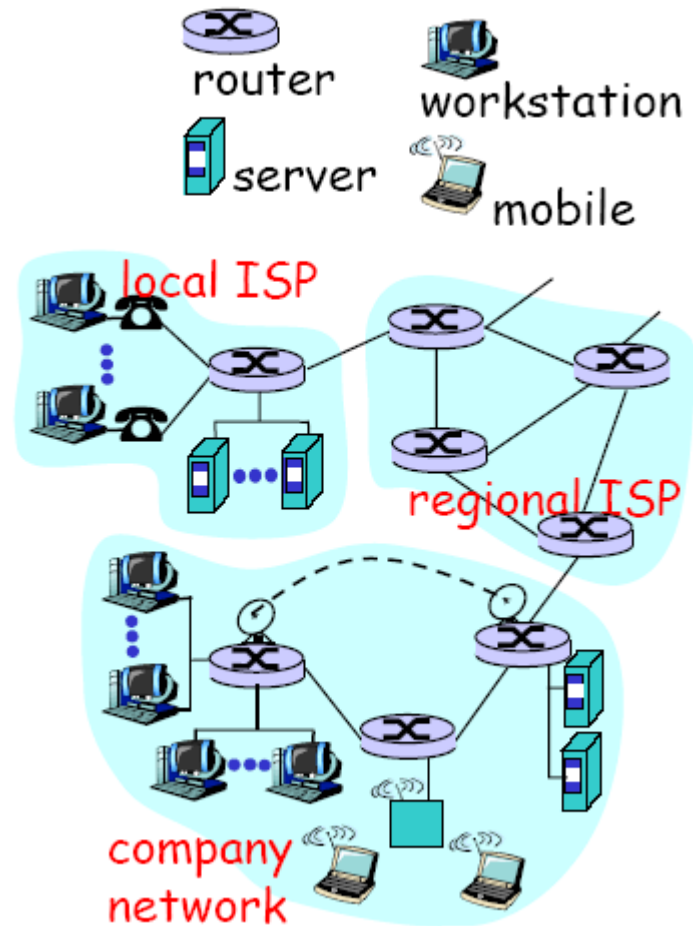
1.6 패킷교환 네트워크에서 지연과 손실

1.7 프로토콜계층과 서비스 모델

1.8 역사

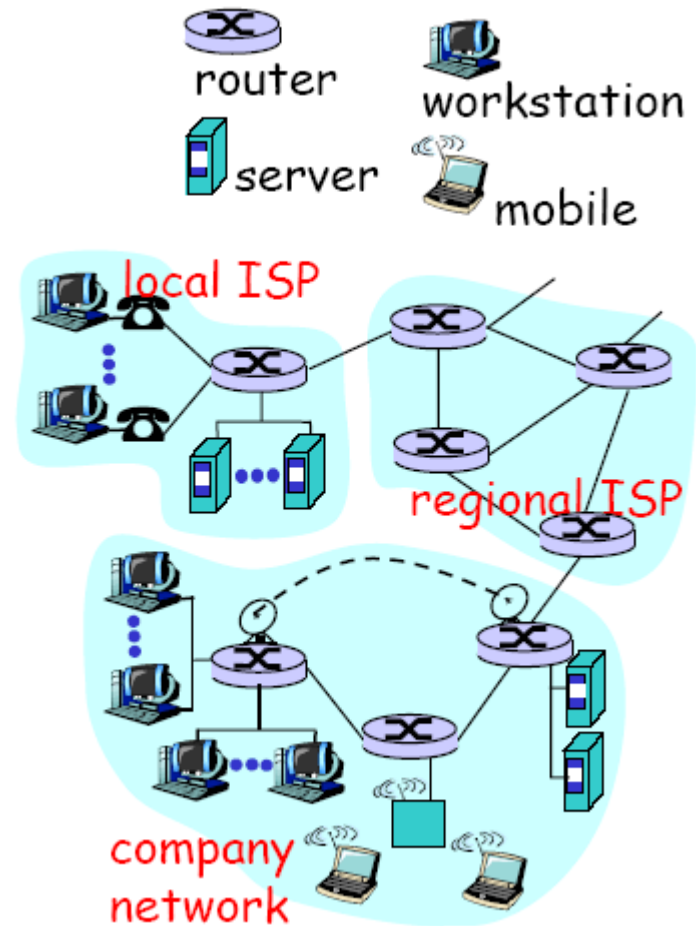
인터넷이란 무엇인가? : 구성

- 전세계 수백만 대의 컴퓨팅 장비를 연결한 네트워크이다
 - 인터넷에 연결된 장치
 - host = end system**
- 호스트들은 **네트워크 응용 프로그램**을 통해 인터넷에 참여한다.
- 각 호스트는 **통신링크**를 통해 연결된다.
 - 동축, 광, 무선, 위성 등의 물리적인 매체를 이용
 - 다양한 전송률을 가진다.
 - (bandwidth)**
- 패킷스위치 (**라우터**, 링크계층스위치) 이용



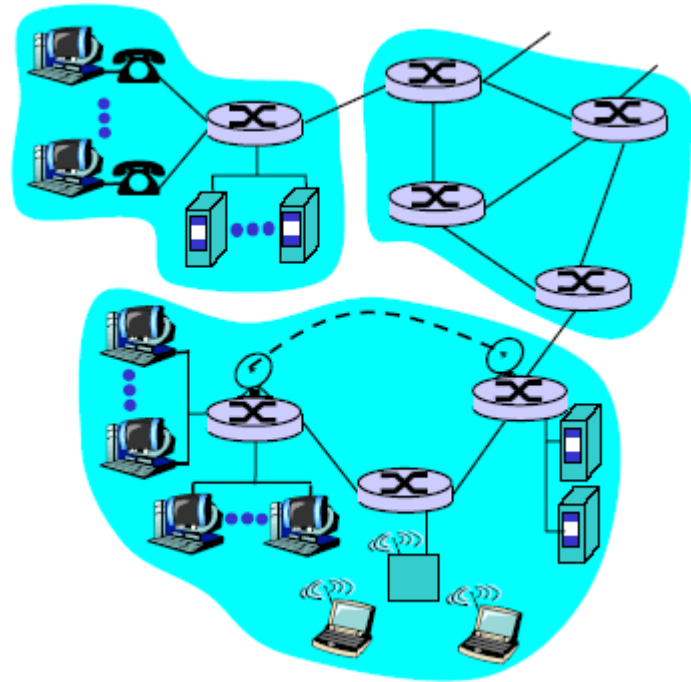
인터넷이란 무엇인가? : 구성

- 프로토콜 : 송수신 데이터를 제어한다.
 - 예 : TCP, IP, HTTP, FTP, PPP
- 인터넷 : “network of network”
 - 느슨한 계층적 구조
 - Public Internet ver private Intranet
- 인터넷 표준
 - RFC : Request for comments
 - IETF : Internet Engineering Task Force



인터넷이란 무엇인가? : 서비스

- 분산 응용 프로그램을 사용할 수 있는 통신 기반 구조
 - Web, email, game, e-commerce, file sharing
- 두가지 통신 서비스 유형
 - 비연결형 서비스
 - 연결지향형 서비스



What's a protocol ?

사람간의 의사교환

(human protocol)

- “질문있냐?”
- “질문 있음당”
-

사람간의 Protocol

상호간에 보내는 특정한 메시지가 있고 수신된 응답메시지 혹은 다른 상황에 대응해서 취하는 특정한 행동이 있다.

Network protocol

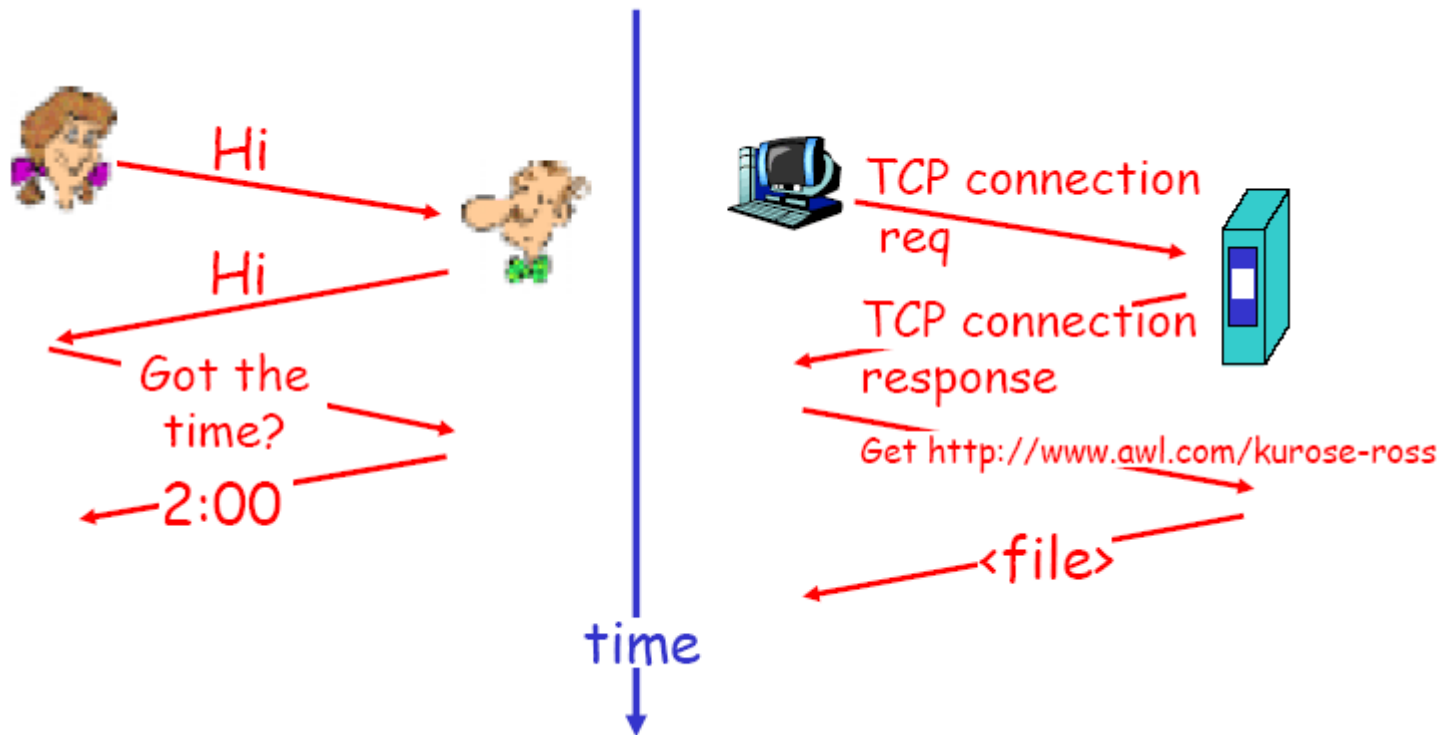
- 사람 간의 의사전달과 매우 비슷하다. (좀더 단순하다.)
- 모든 데이터 송수신은 인터넷에 지정된 전송 규격에 따라야 한다.

Protocol의 정의

프로토콜은 둘 이상의 통신 개체 간에 교환되는 메시지 포맷과 순서 뿐 아니라 메시지의 송수신과 다른 이벤트에서 취하는 행동들을 정의한다.

What's a protocol ?

- 사람이나 컴퓨터간의 통신 protocol



1.1 인터넷이란 무엇인가

1.2 네트워크의 가장자리(edge)

1.3 네트워크의 코어 (core)

1.4 네트워크 접속과 매체

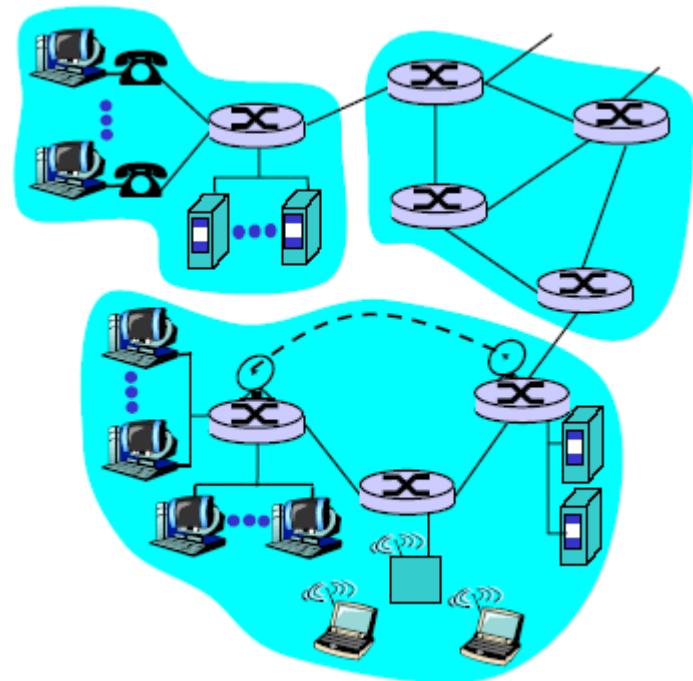
1.5 ISP와 인터넷 백본

1.6 패킷교환 네트워크에서 지연과 손실

1.7 프로토콜계층과 서비스 모델

1.8 역사

- Network edge
 - 응용프로그램, 호스트
- Network core
 - Router
 - network of network
- Access networks, Physical media: communication links



➤ 종단시스템

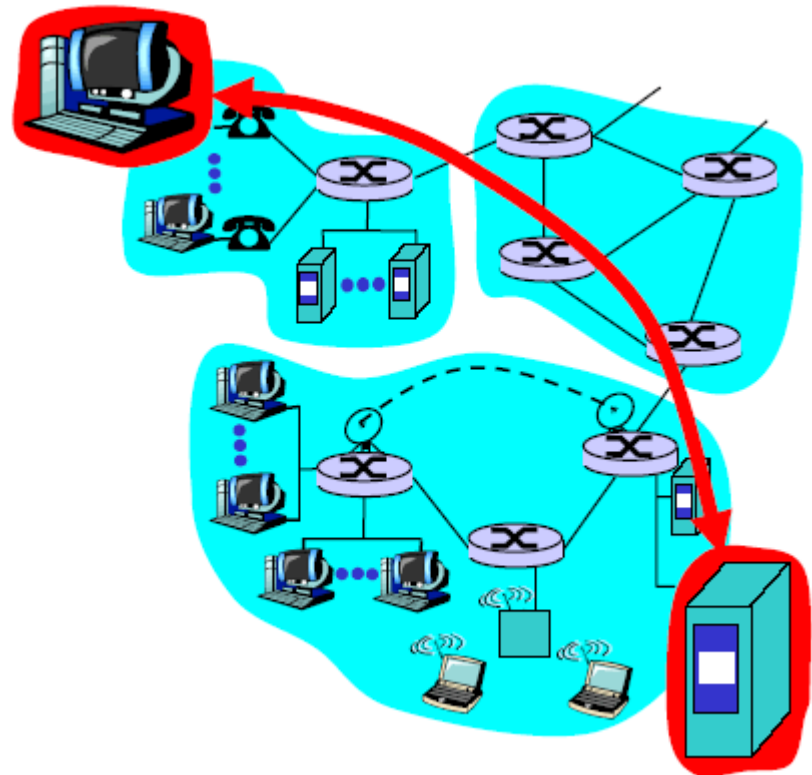
- end system = hosts
- 응용 프로그램 실행
- 예. Web, email
- at “edge of network”

➤ Client/Server model

- Client : 질의 (request)
- Server : 응답 (response)
- 예. Web browser/server
email
client/server

➤ Peer-peer model

- Client와 server 기능을 동시에 구현
- 예. 소리바다, 당나귀...



1.1 인터넷이란 무엇인가

1.2 네트워크의 가장자리(edge)

1.3 네트워크의 코어 (core)

1.4 네트워크 접속과 매체

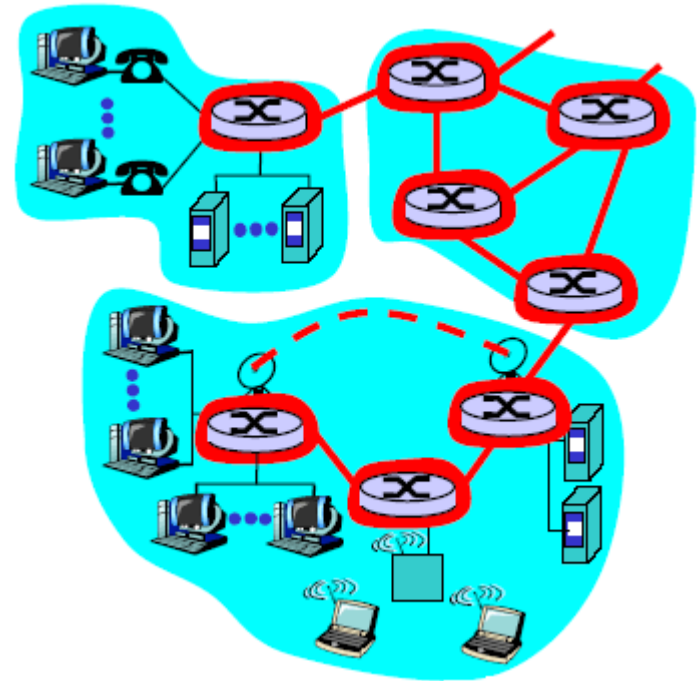
1.5 ISP와 인터넷 백본

1.6 패킷교환 네트워크에서 지연과 손실

1.7 프로토콜계층과 서비스 모델

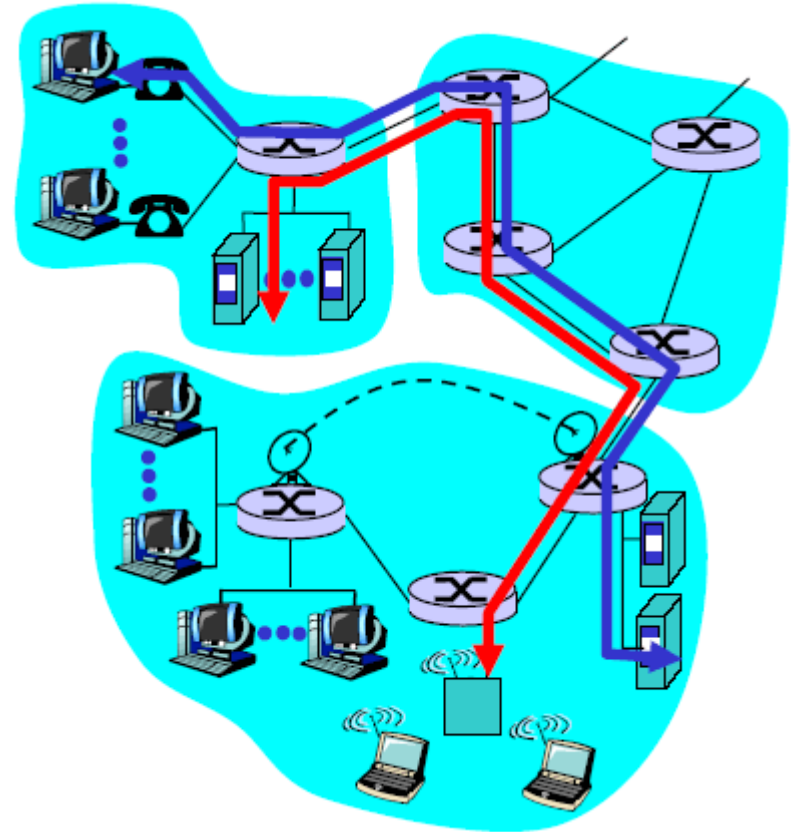
1.8 역사

- 인터넷의 종단 시스템들을 연결하는 라우터의 연결 조직
- 근본적인 문제 :
어떻게 네트워크를 통해서 데이터를 전송하는가
 - **Circuit switching**(회선 교환)
통신 세션 동안 전용 회선(예약된 자원)을 이용하는 방법
 - **Packet switching** (패킷 교환)
데이터를 일정 크기의 단위로 나누어 불연속적으로 네트워크에 전송하는 방법



종단간 통신을 위해 자원을 예약하는 통신 방식

- 송수신자간의 경로에 있는 스위치들이 연결상태를 유지하는 연결
이 연결을 회선(circuit)이라 한다.
- 회선은 네트워크에 일정한 전송 대역폭 (bandwidth)을 예약한다.
- 예약된 자원은 전용되며 공유되지 않는다.
- 연결된 회선은 일정한 성능을 보장한다.
- 통신을 위해서는 사전 회선예약이 필요하다.



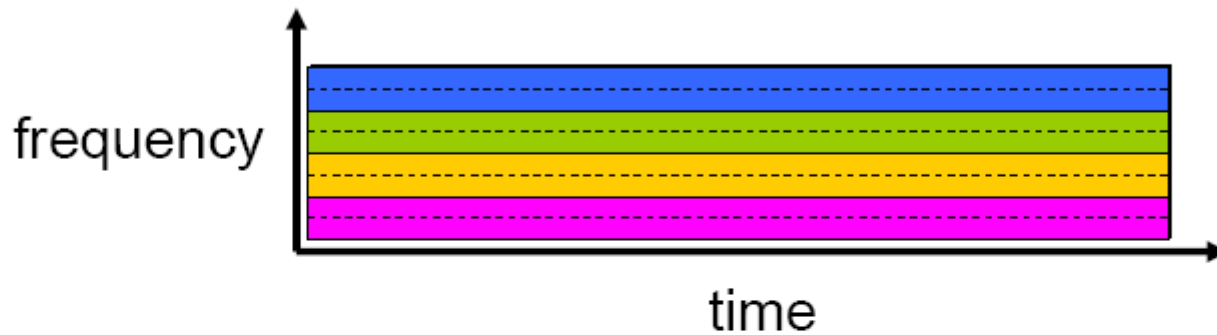
네트워크 자원을 분할해서 사용한다.

- 분할된 자원은 세션에 할당 되어 회선이 구성된다.
- 세션 내에 전송데이터가 없다면 회선은 낭비된다. (할당된 자원은 공유 되지 않는다.)
- 링크내에 회선은 FDM(frequency division)이나 TDM(time division)으로 구현된다.

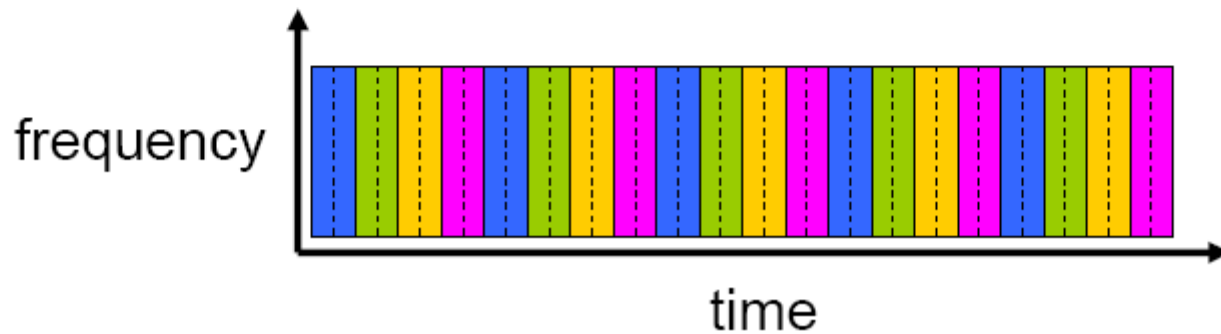
FDM

Example:

4 users



TDM



종단간에 전송되는 메시지를 packet
이라는 작은 단위로 분할해서 전송한다.

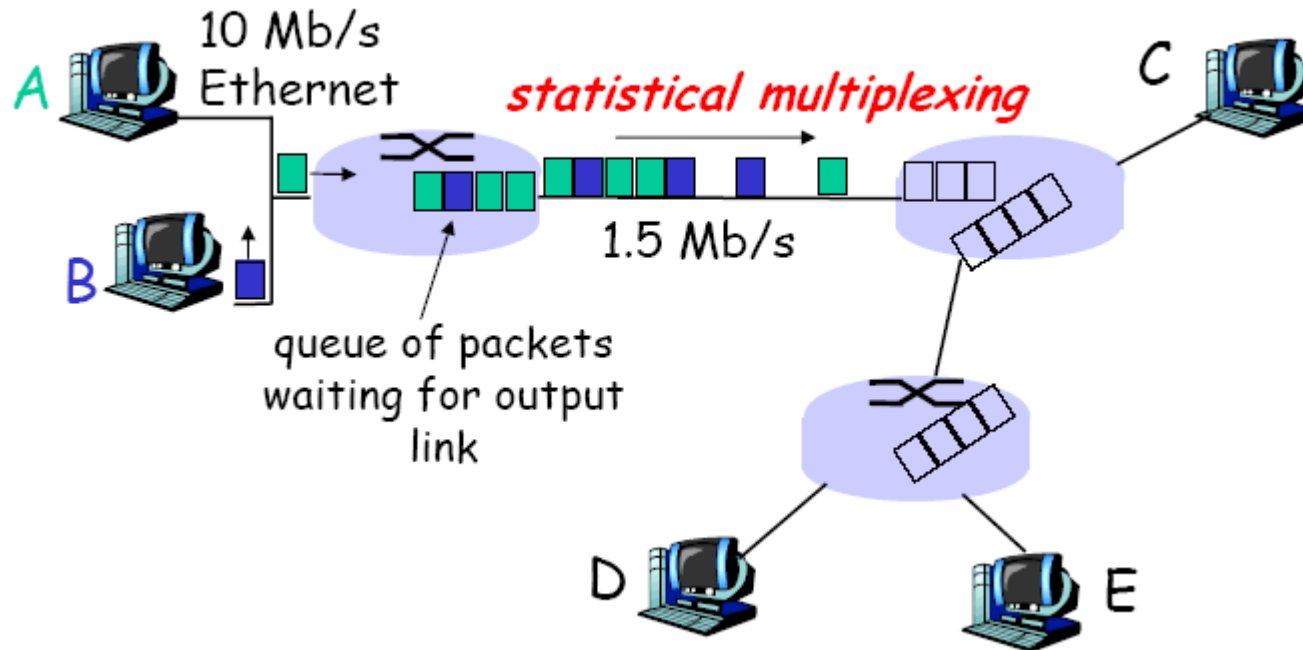
- 사용자들의 패킷은 네트워크자원을 공유한다.
- 전송되는 패킷은 대역폭을 모두 사용한다.
- 자원은 필요에 따라 사용된다.

전송을 위해 대역폭을 나누거나 자원을
예약하지 않는다.

자원의 경합이나 지연

- 네트워크가 전송 가능한 용량을 초과해서 자원(전송)을 요구하는 경우
- 출력 버퍼나 큐에서 나타나는 지연 및 혼잡
- Store and forward :
- 패킷이 각 패킷 스위치에서 처리를 위해 지연되는 시간
 - 스위치가 출력 링크로 패킷을 전송하기 위해서는 전체 패킷을 일단 받아야 한다.

패킷 교환 : 통계적 다중화



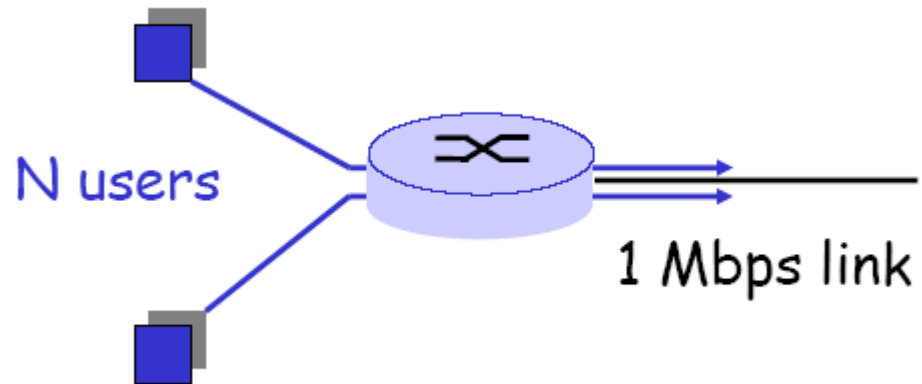
A와 B 패킷의 순서가 일정하지 않다. 즉 보내지는 순서에 일정한 패턴이 없다.
⇒ **statistical multiplexing** (통계적 다중화)

반복되는 TDM프레임 안의 일정 slot를 각 호스트가 할당 받는 TDM과 비교된다.

패킷 교환과 회선 교환의 비교

패킷교환 방식이 대역폭 공유에 더 효율적이다.

- 1M의 링크를 공유하는 경우
- 조건
 - 100kb/s when “active”
 - Active 10% time
- 회선 교환 방식
 - 단지 10명만이 회선을 사용한다.
- 패킷교환 방식
 - 35명의 사용자가 있는 경우
11명 이상이 동시에 active 할
확률은 0.0004%이다.



패킷교환이 진정한 해결책인가???

- 불규칙하게 발생하는 대량의 데이터 처리에 적합
 - 자원 공유가 효율적이다.
 - 단순하고 데이터전송을 위한 setup이 불필요 하다.
- 과도한 혼잡이 발생할 경우 : 패킷이 지연되거나 손실된다.
 - 혼잡 제어가 가능한 신뢰할 수 있는 protocol이 필요하다.
- Q: How to provide circuit-like behavior?
 - 대역폭의 보장이 필요한 오디오/비디오 관련 응용 프로그램
 - Still an unsolved problem (chapter 6)

패킷 교환 : store-and-forward



- L비트의 패킷을 R bps 속도를 가진 link에 보내기 위해서는 L/R 시간이 걸린다.

Example

$L = 7.5 \text{ Mbits}$

$R = 1.5 \text{ Mbps}$

Delay = 15sec

- 위의 그림과 같이 패킷은 반드시 2개의 라우터를 통해서 전송된다.
: store and forward

- Delay = $3L/R$

Goal : 목적지까지 라우터를 통해 패킷을 이동 시키는것

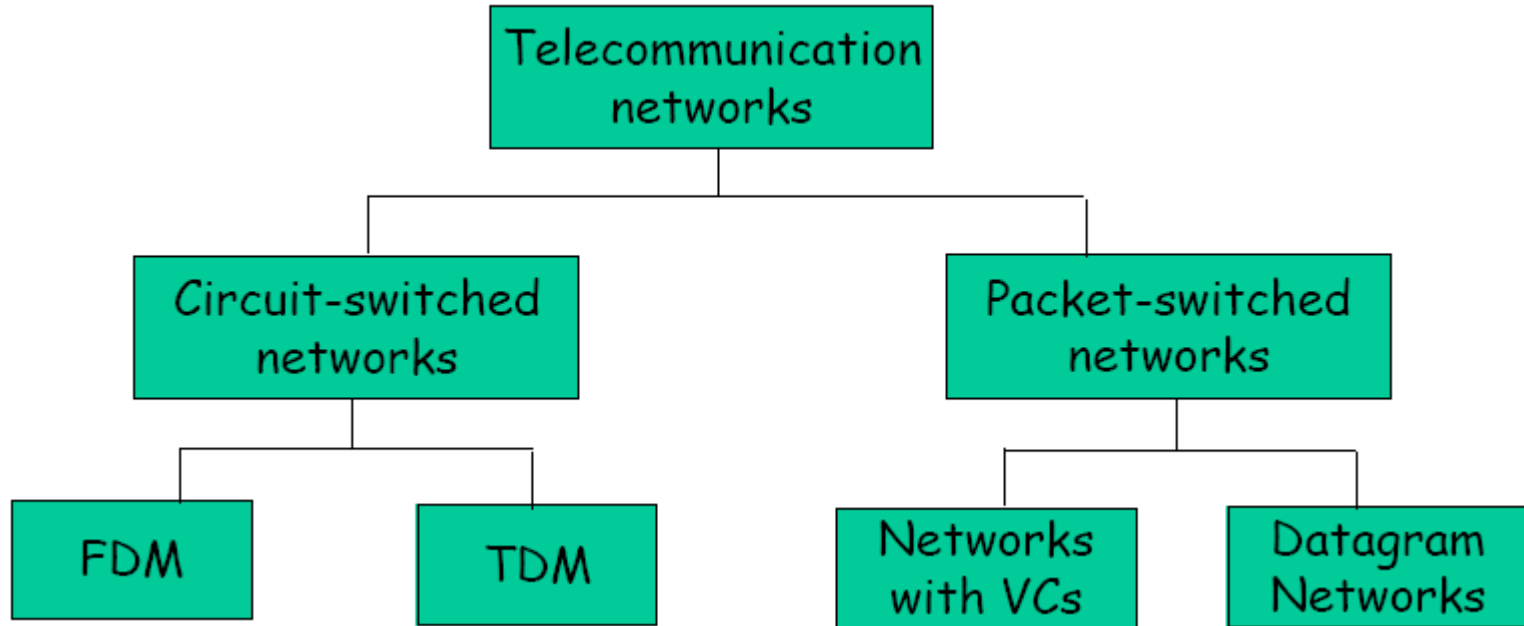
- 경로를 선정하는 라우팅 알고리즘에 대해서는 4장에서 학습한다.

➤ Datagram network (데이터그램 네트워크)

- 다음 hop까지의 경로 결정에 목적지의 주소를 이용한다.
- 세션 동안 경로가 변경될 수 있다.
- 길을 물어서 목적지까지 운전하는 방법과 유사하다.

➤ Virtual circuit network (가상 회선 네트워크)

- 어떤 패킷은 자신의 헤더에 VC id를 갖는데 이것은 다음 hop으로의 경로를 결정하는데 이용된다.
- VC id는 출발지에서 도착지까지의 경로가 확립될 때 할당 된다.
- 각 스위치(라우터)는 VC네트워크 교환에 이용되는 정보를 유지한다.



데이터그램 네트워크는 연결지향 또는 비연결형이 아니다.

- 패킷의 목적지 주소에 의해서 스위치가 패킷을 전달하지만 연결상태를 유지하지 않는다.

Internet은 연결지향에 TCP를 비연결형에 UDP를 제공한다.

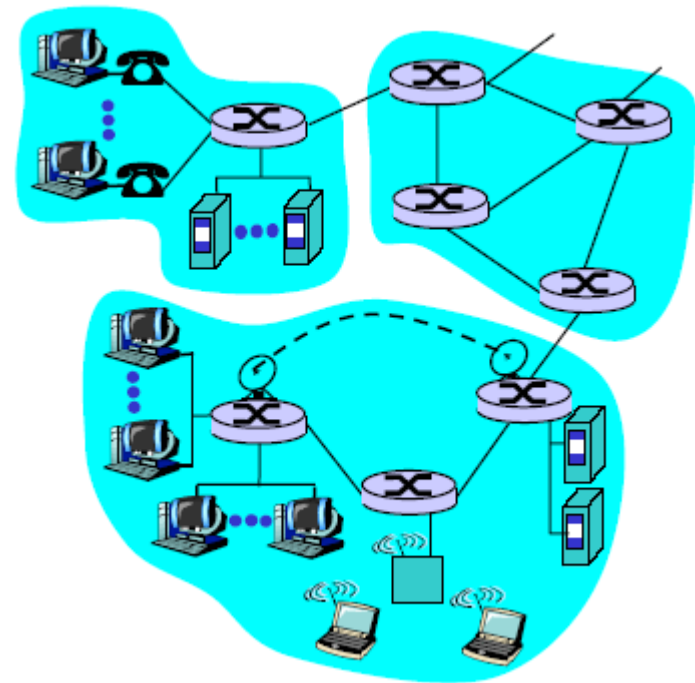
- 1.1 인터넷이란 무엇인가
- 1.2 네트워크의 가장자리(edge)
- 1.3 네트워크의 코어 (core)
- 1.4 네트워크 접속과 매체
- 1.5 ISP와 인터넷 백본
- 1.6 패킷교환 네트워크에서 지연과 손실
- 1.7 프로토콜계층과 서비스 모델
- 1.8 역사

Q : 종단 시스템이 종단 라우터에 어떻게 접속하는가?

- 가정 접속 (residential access)
- 기관 접속 (institutional access) :
- 이동 접속 (mobile access)
- 무선 접속 (wireless access)

중요한 관점

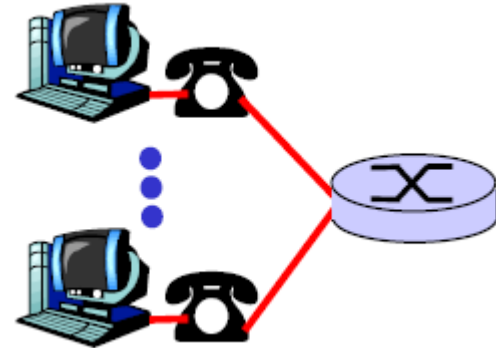
- 접속에서의 대역폭 (bits per second)
- 공유, 전용 여부



가정 접속 : point to point

➤ Dialup modem

- 56Kbps를 제공하는 직접 접속
(ISP 라우터의 모뎀과 연결)
- 모뎀을 사용하는 동안 음성전화는 불가능
(느리며 항상 접속을 구현 하기 힘들다)



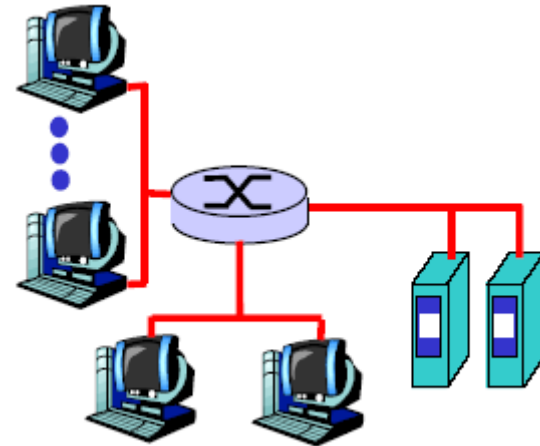
➤ ADSL : asymmetric digital subscriber line

- 1M 이상의 upload 속도
- 8M 이상의 download 속도
- FDM :
 - 50 – 1MHz : downstream 채널
 - 4 – 50KHz : upstream 채널
 - 0 – 4KHz : 일반적인 전화 채널

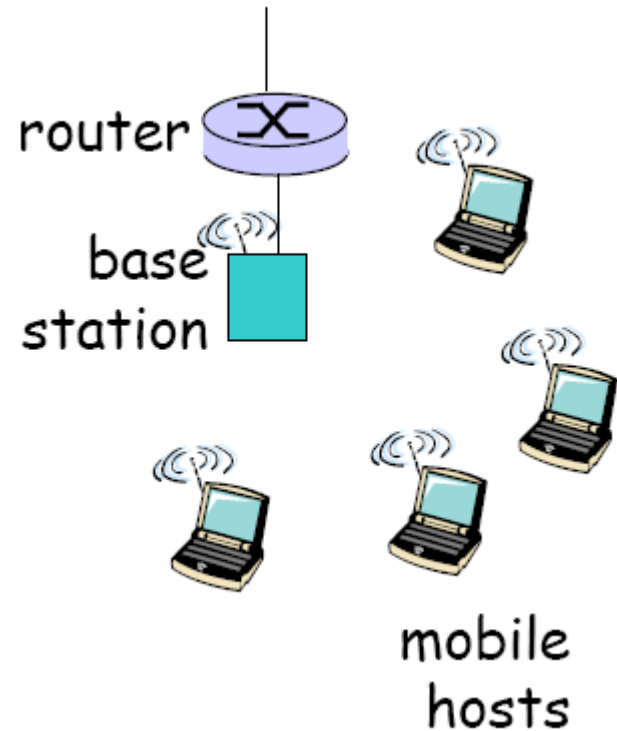
- HFC : 동축케이블을 이용 (Hybrid fiber coax)
 - 30M 이상의 downstream, 2M 이상의 upstream을 제공 가능
- 네트워크는 광케이블이나 동축케이블을 이용 가정에서 ISP까지 연결된다.
 - 헤드엔드나 광노드를 공유하는 방법으로 접속한다.
(router를 공유한다.)
- 공유방송매체를 이용한다.
(TV cable을 이용)

기관 접속 : local area network

- 기업이나 대학에서는 LAN을 이용 종단 시스템이 edge 라우터에 연결한다.
- Ethernet
 - 이더넷은 여러 종단 시스템에서 가장자리 라우터까지 연결을 담당한다. (공유, 전용)
 - 10M, 100M, Gigabit Ethernet

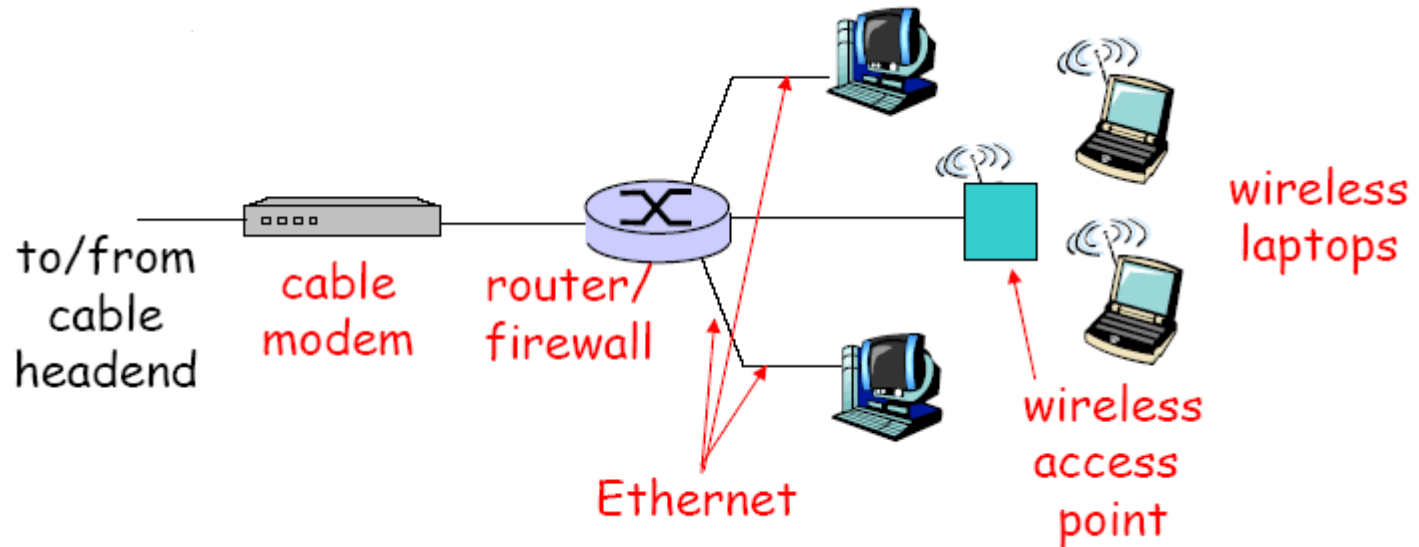


- 종단 시스템에서 라우터까지 공유 무선 접속 네트워크
 - 무선 기지국 : 액세스포인트 (access point)
- 무선랜
 - 802.11b (WiFi) : 11Mbps (802.11g : 54Mbps)
- Wider-area wireless access (광대역 무선 접속)
 - 전화 회사에서 제공
 - 3G – 384Kbps
 - WAP/GPRS (유럽)



Home network 의 구성

- ADSL 또는 cable modem
- Router / firewall / NAT
- Ethernet
- Wireless access point



- **Bit** : 전송시에 일련의 송수신기 쌍을 거쳐서 전달된다.
- **물리적 매체** : 송수신기 사이에 연결 경로
 - 유도 매체(guided media)
 - 비유도 매체(unguided media)
- **유도 매체**
 - 광케이블, 동축케이블, UTP와 같이 견고한 매체를 따라서 파형을 유도하는것
- **비유도 매체**
 - 무선 LAN, 디지털 위성채널등과 같이 대기와 야외공간으로 파형을 전파하는것

꼬임쌍 동선(Twisted pair : TP)

- 절연된 두 가닥의 꼬인 동선
 - Category 3 : 일반적으로 전화망에서 이용, 10Mbps
 - Category 5 ~ : 100Mbps. 1Gbps



물리 매체 : 동축케이블, 광케이블

동축케이블 (Coaxial cable)

동심원형태의 두 개의 구리 선으로 되어있다.

- 양방향
- Baseband
 - 단일 채널
 - 10base2 ethernet
- Broadband
 - 여러 채널
 - HFC



광케이블 (Fiber optic cable)

비트를 나타내는 빛의 파동을 전달하는 가
늘고 유연한 유리섬유

- 매우 빠른 전송 속도
 - 두 지점간 매우 빠른 전송이 가능
(5Gbps)
- 전자기성 간섭의 영향이 받지 않고
100Km까지 신호 감쇄 현상이 적으며
도청이 어렵다.
에러율이 적다.



- 신호를 전자기 스펙트럼을 이용 전달
- wire를 이용하지 않는다.
- 양방향
- 전파 환경과 신호가 전달 거리에 영향을 받는다.
 - Reflection : 방해물질을 돌아가거나 통과함에 따라 신호가 약해짐
 - Obstruction by objects : 간섭물체에 의해 신호 방해
 - Interference : 다른 전자기 신호로 인한 전파 간섭

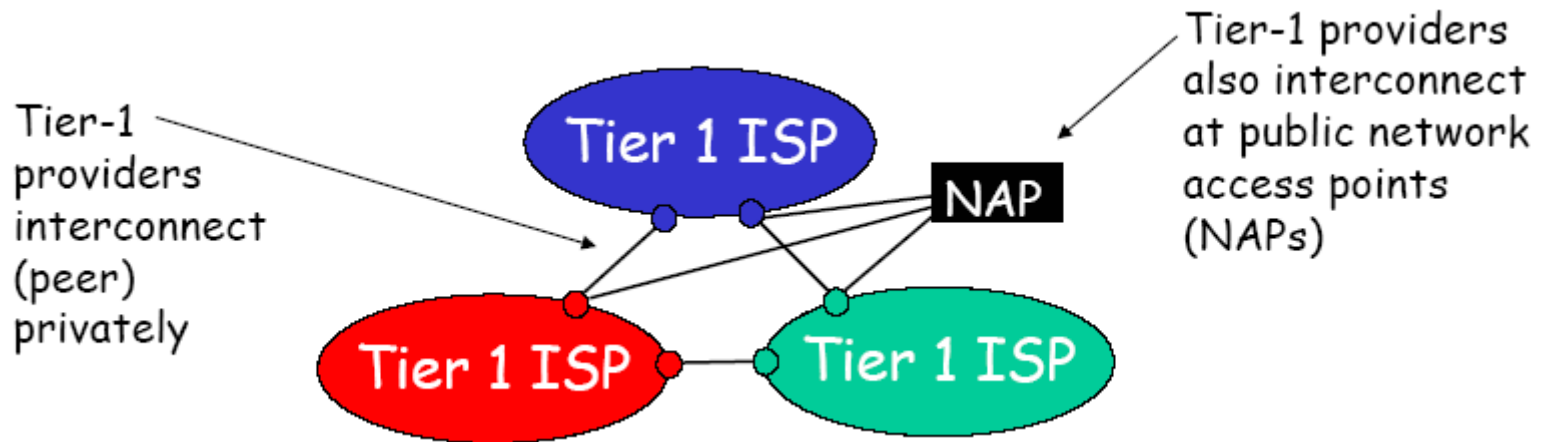
Radio link types

- 지상파
 - 예: 45Mbps 이상의 채널
- LAN (WiFi...)
 - 2Mbps, 11Mbps, 54Mbps...
- Wide-area (cellular...)
 - 예: 3G (수백 Kbps)...
- 위성
 - 50Mbps 이상의 채널 (또는 이보다 적은 멀티 채널)
 - 250ms의 신호전파 지연
 - 정지위성이나 저궤도 위성

- 1.1 인터넷이란 무엇인가
- 1.2 네트워크의 가장자리(edge)
- 1.3 네트워크의 코어 (core)
- 1.4 네트워크 접속과 매체
- 1.5 ISP와 인터넷 백본
- 1.6 패킷교환 네트워크에서 지연과 손실
- 1.7 프로토콜계층과 서비스 모델
- 1.8 역사

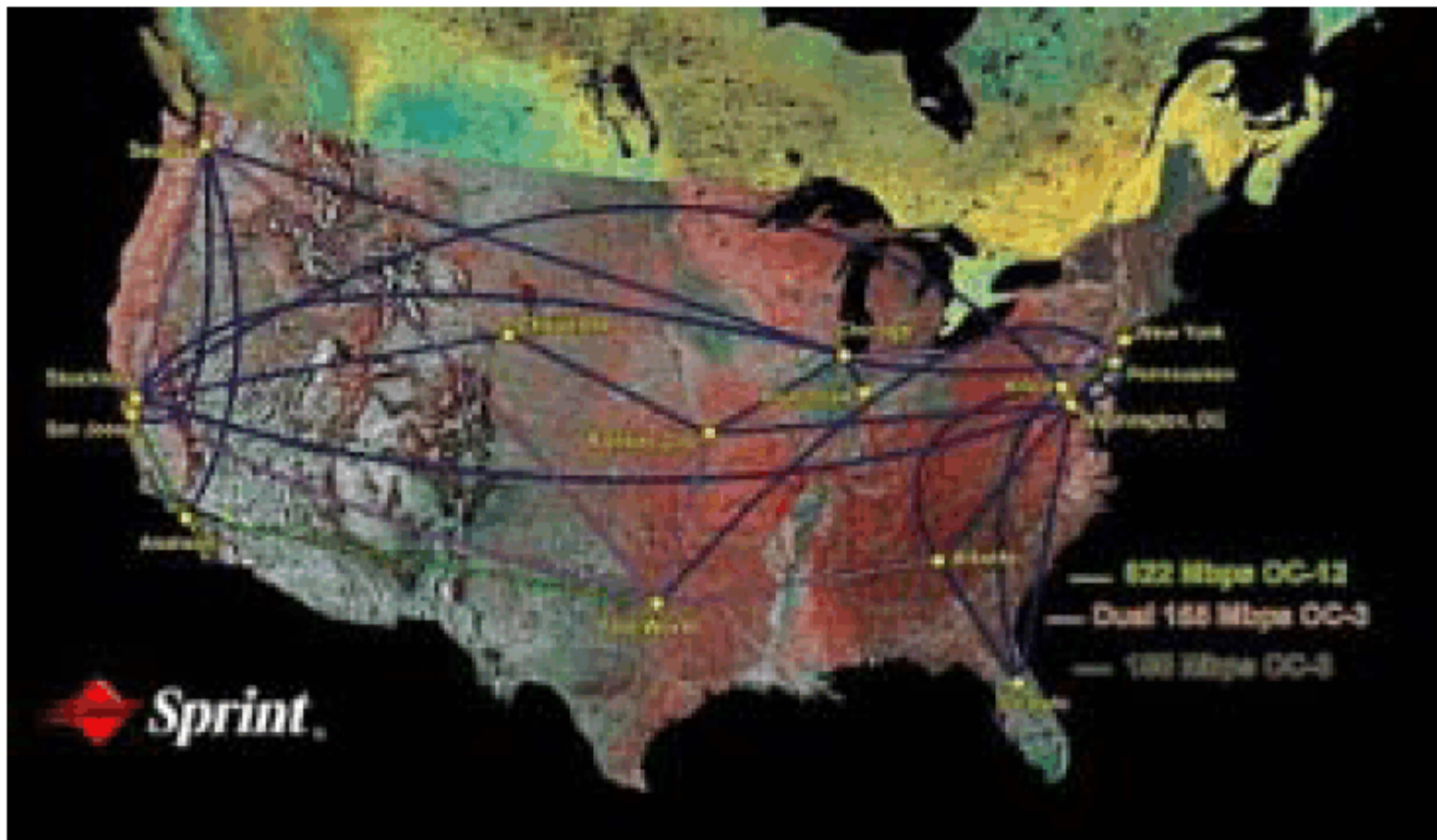
인터넷의 구조 : net of net

- Roughly hierarchical
- 인터넷의 중앙에 **tier-1 ISP**가 서비스를 담당한다. 이들은 다른 tier-1 ISP들과 직접 고속으로 연결되며 많은 수의 tier-2 ISP들과 연결된다. 또한 서비스 범위가 국제적이다.
 - 이들은 서로 동일하게 취급된다.



Tier-1 ISP : Sprint

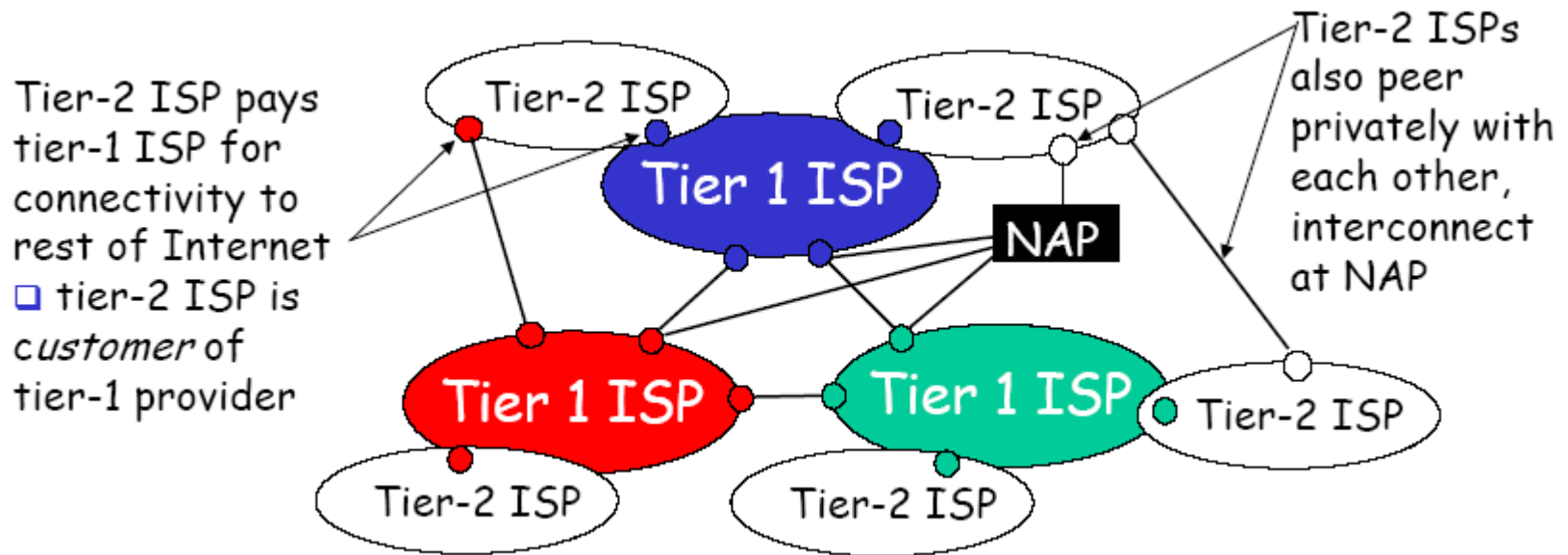
- Sprint US backbone network



인터넷의 구조 : net of net

➤ Tier-2 ISP

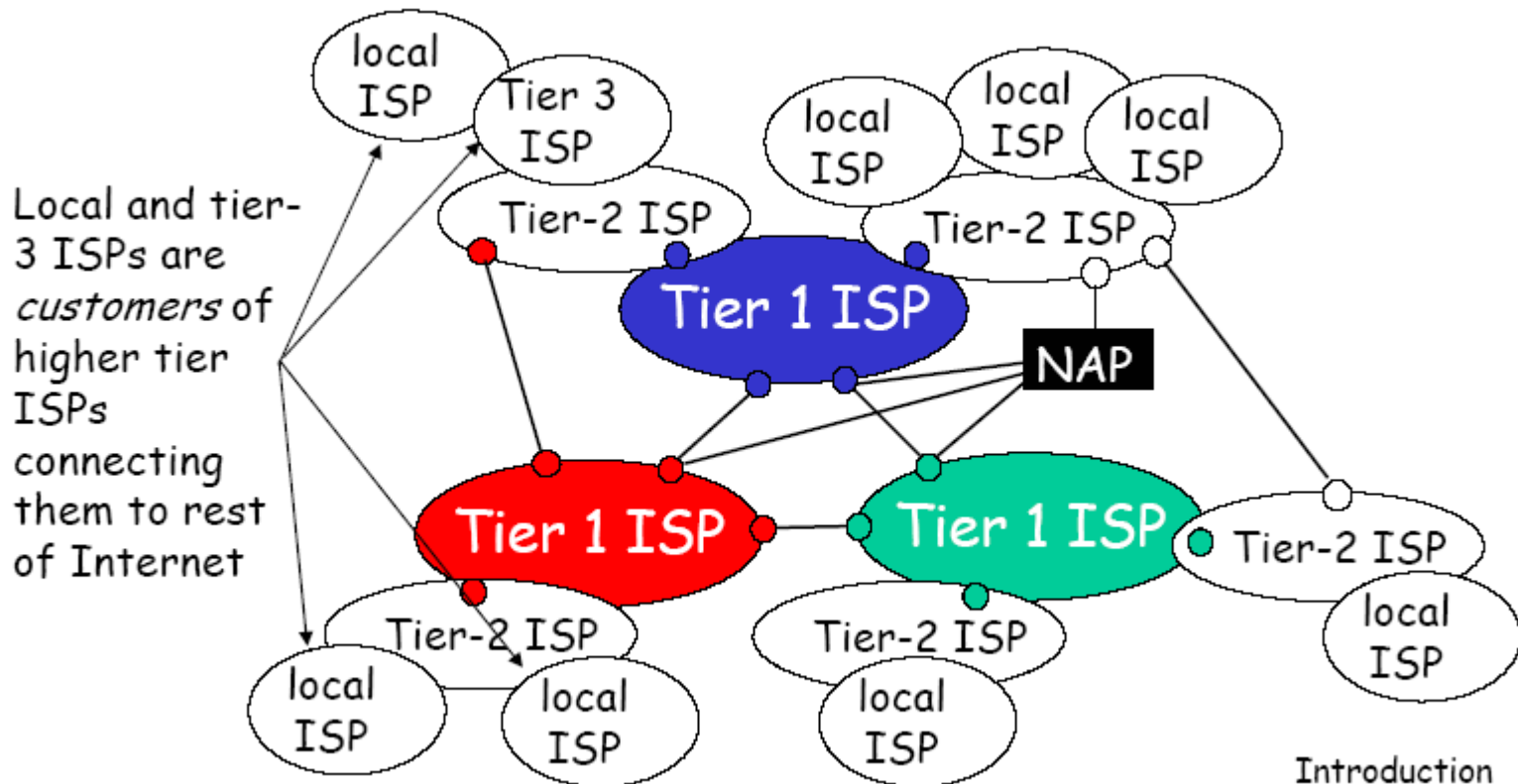
- 하나 이상의 tier-1 ISP에 연결되고 tier-2 ISP와도 NAP에서 상호 연결된다.
- Tier-2 는 Tier-1 의 고객이 되며 Tier-2간에는 서로 peer이다.



인터넷의 구조 : net of net

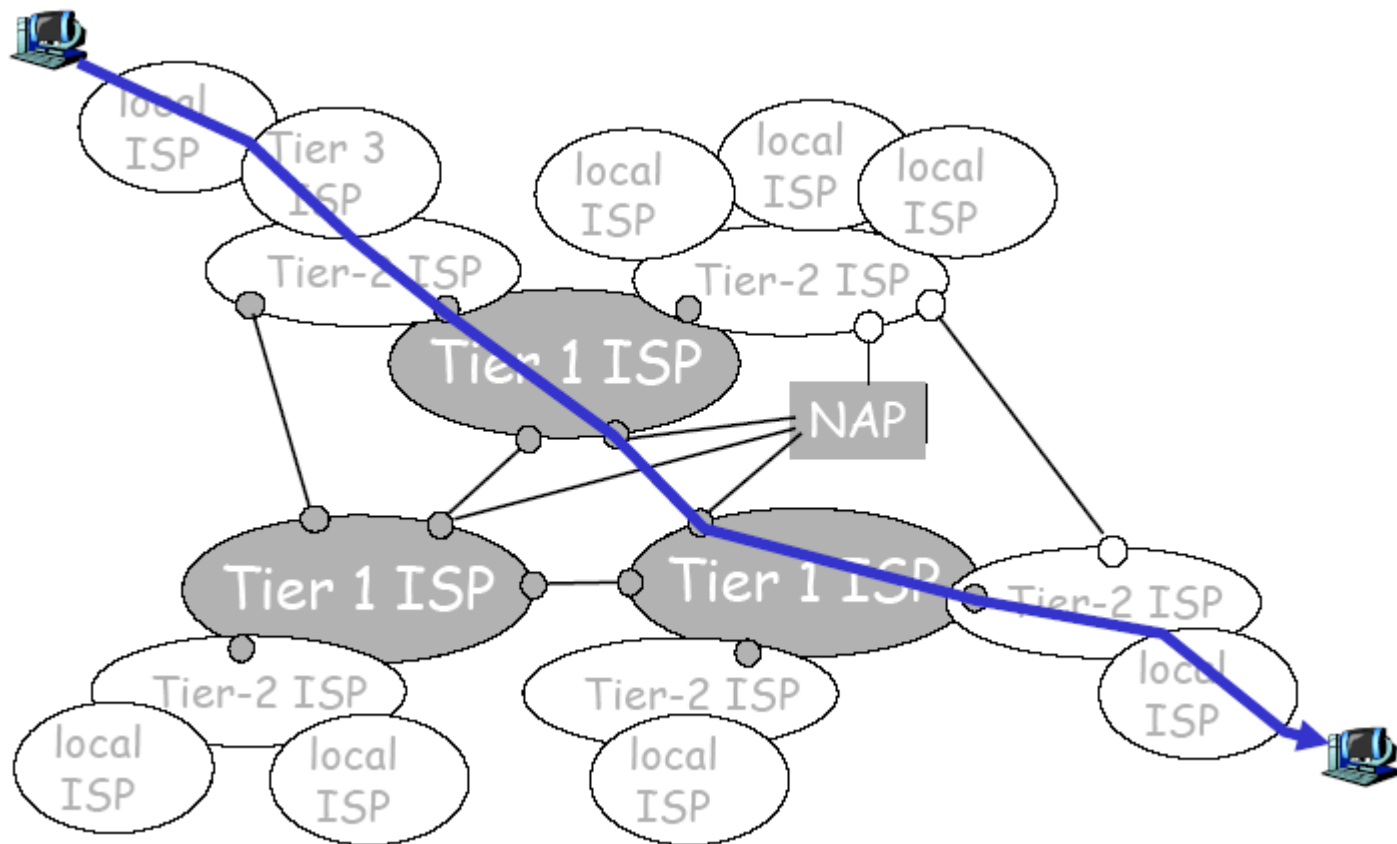
➤ Tier-3 ISP와 local ISP

- Last hop (access) network (closest to end systems)



인터넷의 구조 : net of net

➤ 패킷은 수많은 네트워크를 통과해서 전달된다.

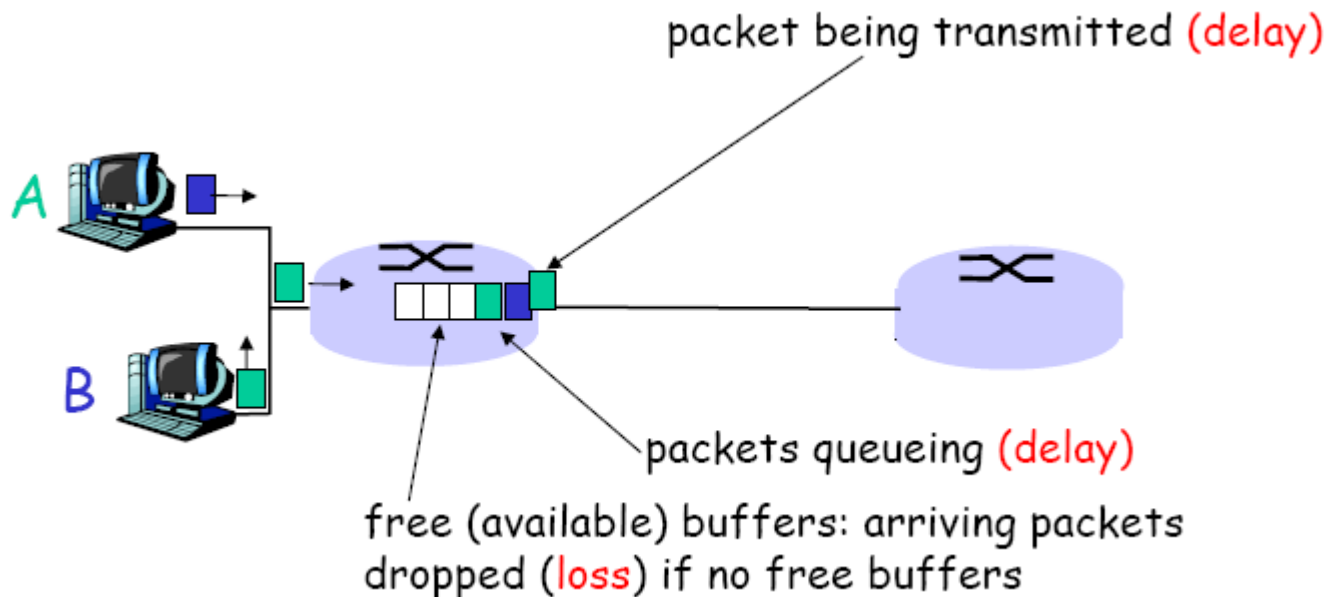


- 1.1 인터넷이란 무엇인가
- 1.2 네트워크의 가장자리(edge)
- 1.3 네트워크의 코어 (core)
- 1.4 네트워크 접속과 매체
- 1.5 ISP와 인터넷 백본
- 1.6 패킷교환 네트워크에서 지연과 손실
- 1.7 프로토콜계층과 서비스 모델
- 1.8 역사

패킷의 지연과 손실

패킷은 라우터의 버퍼에 시간순으로 저장된 이후 전송된다.

- 패킷이 라우터에 들어오는 량이 라우터가 전송 가능한 량을 초과한 경우
- 패킷이 라우터의 큐에서 처리순서를 기다리는 경우



패킷이 지연되는 4가지 요소

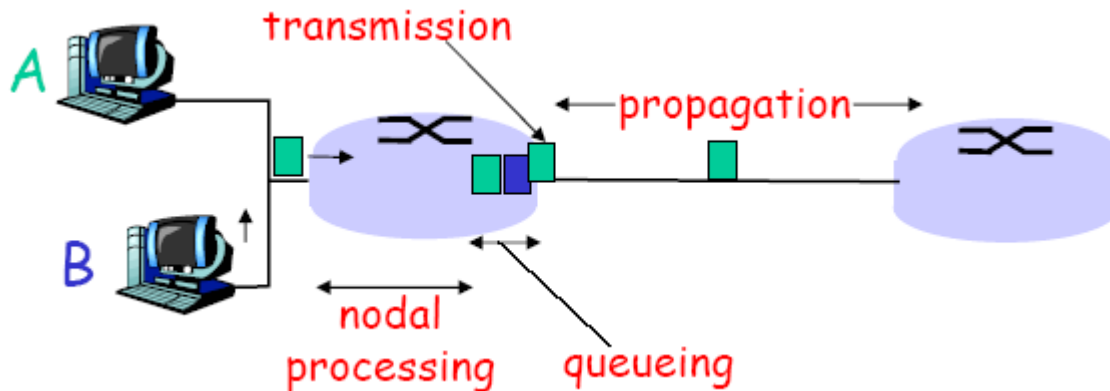
1. 처리 지연

- 패킷의 헤더를 조사 에러를 점검
- 출력 방향을 결정

2. 큐잉 지연

- 큐에 저장되어 전송을 기다리는 시간
- 라우터의 혼잡처리에 의존한다.

Link : node간의 connection
Path : a sequence of link



패킷이 지연되는 4가지 요소

3. 전송 지연

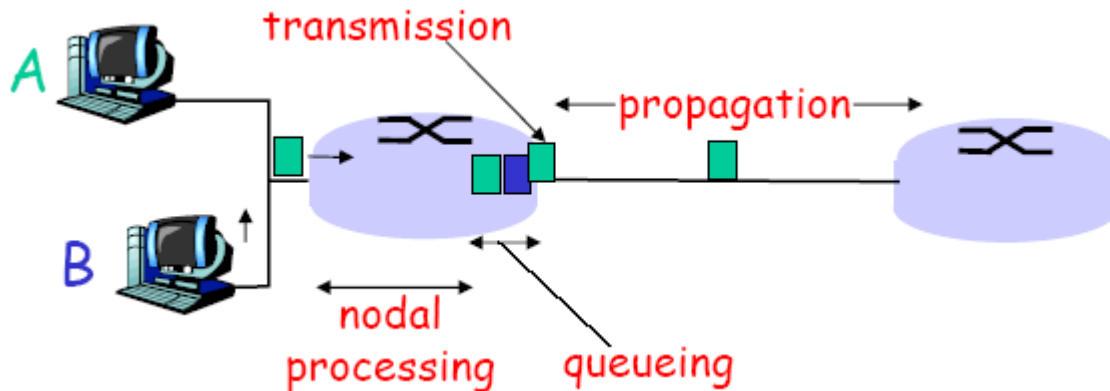
- R = 링크의 전송률
- L = 패킷의 길이
- 전송지연 = L/R

4. 전파 지연

(Propagation delay)

- d = 링크의 길이
- s = 전달 속도(2×10^8 m/sec)
- 전파 지연 = d/s

R 과 s 는 크기가
매우 다르다



$$d_{\text{noda1}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

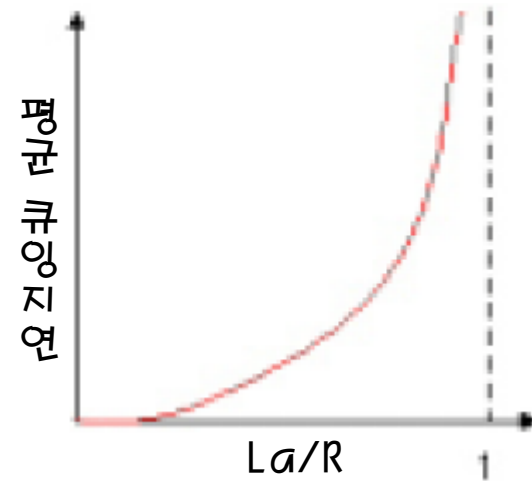
- d_{proc} : 처리 지연
 - 수 마이크로초 정도로 보통 무시된다.
 - 라우터의 최대 처리률에는 상당한 영향을 준다.
- d_{queue} : 큐잉 지연
 - 혼잡도에 따라 다르다
- d_{trans} : 전송 지연
 - L/R : 일반적으로 무시 가능하다. (10Mbps 이상에서)
 - 매우 저속의 링크에서는 수 백밀리초에 이를 수 있다.
- d_{prop} : 전파 지연
 - 수 마이크로 초로 무시 가능하다.
 - 정지위성의 경우 수백 밀리초에 이를 수 있다.

큐잉 지연 (queueing delay)

- R = Link의 bandwidth (bps)
 - 전송률
- L = 패킷의 길이 (bits)
- a = 패킷이 큐에 도착하는 평균 속도 (패킷/초 단위)

트래픽 강도 = La/R

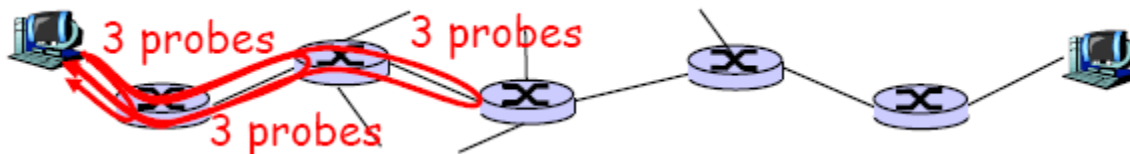
- $La/R \sim 0$: 큐잉 지연이 매우 작다
- $La/R \rightarrow 1$: 지연이 커진다.
- $La/R > 1$: 큐잉지연은 무한대로 증가한다. (큐의 길이가 무한히 늘어난다)
 - 큐에 도착하는 비트가 큐에서 전송되는 비율을 초과하는 경우



“트래픽 강도가 1보다 크지 않게 시스템을 설계한다.”

인터넷에서 지연과 경로 확인

- 어떻게 인터넷에서 지연과 손실을 볼 수 있는가?
- Traceroute : 출발지에서 목적지 호스트까지 경로상에 있는 모든 라우터까지의 전달과정과 지연을 측정한다.
 - 경로에 N-1개의 라우터가 존재한다면 출발지는 1부터 N까지 표시된 패킷을 전송한다.
 - n번째 라우터는 n번째 패킷을 받으면 패킷을 목적지에 전송하지 않고 출발지로 메시지를 전송한다.
 - 출발지는 패킷을 보낸 이후 응답을 받을 때까지의 시간과 메시지를 보내온 라우터의 이름과 주소를 기록한다.
 - 목적지가 N번째 패킷을 받으면 목적지는 그 패킷을 폐기하고 출발지로 응답 패킷을 전송한다.
 - 위의 과정은 각 단계별로 세번 반복한다.



traceroute: gaia.cs.umass.edu to www.eurecom.fr

Three delay measurements from
gaia.cs.umass.edu to cs-gw.cs.umass.edu



```
1 cs-gw (128.119.240.254) 1 ms 1 ms 2 ms
2 border1-rt-fa5-1-0.gw.umass.edu (128.119.3.145) 1 ms 1 ms 2 ms
3 cht-vbns.gw.umass.edu (128.119.3.130) 6 ms 5 ms 5 ms
4 jn1-at1-0-0-19.wor.vbns.net (204.147.132.129) 16 ms 11 ms 13 ms
5 jn1-so7-0-0-0.wae.vbns.net (204.147.136.136) 21 ms 18 ms 18 ms
6 abilene-vbns.abilene.ucaid.edu (198.32.11.9) 22 ms 18 ms 22 ms
7 nycm-wash.abilene.ucaid.edu (198.32.8.46) 22 ms 22 ms 22 ms
8 62.40.103.253 (62.40.103.253) 104 ms 109 ms 106 ms
9 de2-1.de1.de.geant.net (62.40.96.129) 109 ms 102 ms 104 ms
10 de.fr1.fr.geant.net (62.40.96.50) 113 ms 121 ms 114 ms
11 renater-gw.fr1.fr.geant.net (62.40.103.54) 112 ms 114 ms 112 ms
12 nio-n2.cssi.renater.fr (193.51.206.13) 111 ms 114 ms 116 ms
13 nice.cssi.renater.fr (195.220.98.102) 123 ms 125 ms 124 ms
14 r3t2-nice.cssi.renater.fr (195.220.98.110) 126 ms 126 ms 124 ms
15 eurecom-valbonne.r3t2.ft.net (193.48.50.54) 135 ms 128 ms 133 ms
16 194.214.211.25 (194.214.211.25) 126 ms 128 ms 126 ms
17 * * *
18 * * *
19 fantasia.eurecom.fr (193.55.113.142) 132 ms 128 ms 136 ms
```

trans-oceanic link



* means no response (probe lost, router not replying)



- 큐의 용량이 무한대가 아님으로 트래픽 강도가 1에 접근하면 패킷 지연이 무한대가 되지 않는다.
- 패킷이 큐에 도착 했을때 큐가 꽉 차있다면 패킷은 버려진다.
- 손실된 패킷의 처리는 재전송 되거나 무시되거나 교정 될 수 있다.

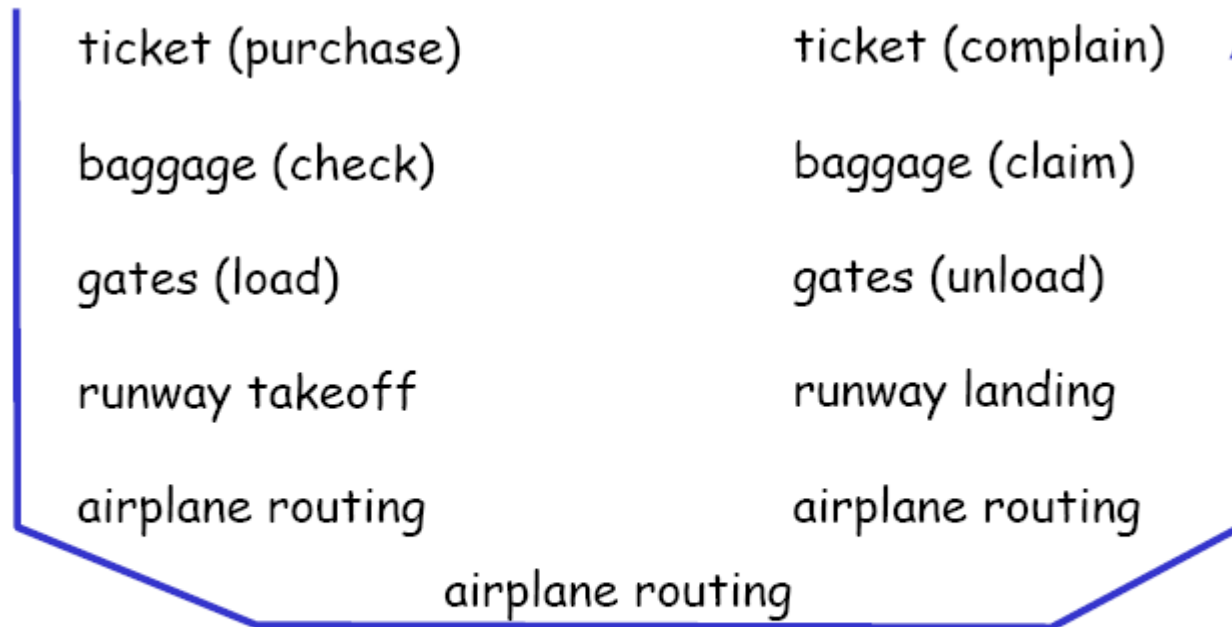
- 1.1 인터넷이란 무엇인가
- 1.2 네트워크의 가장자리(edge)
- 1.3 네트워크의 코어 (core)
- 1.4 네트워크 접속과 매체
- 1.5 ISP와 인터넷 백본
- 1.6 패킷교환 네트워크에서 지연과 손실
- 1.7 프로토콜계층과 서비스 모델
- 1.8 역사

Network are complex

- 많은 구성 요소로 이루어 져있다.
 - 호스트
 - 라우터
 - 여러가지 매체
 - 응용 프로그램
 - 프로토콜
 - H/W, S/W

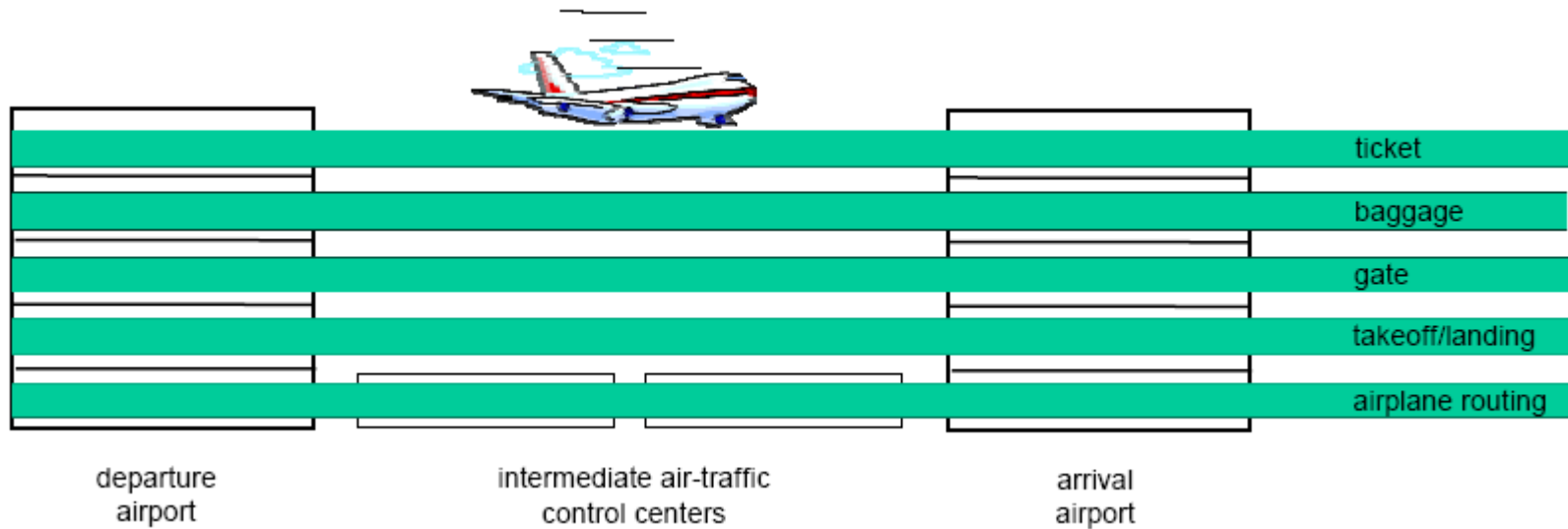
Question

네트워크 구조를 어떻게 조직하는가?
또는
네트워크 구조에 대한 논의는 가능 한
가?



□ a series of steps

공항 탑승 과정의 계층화

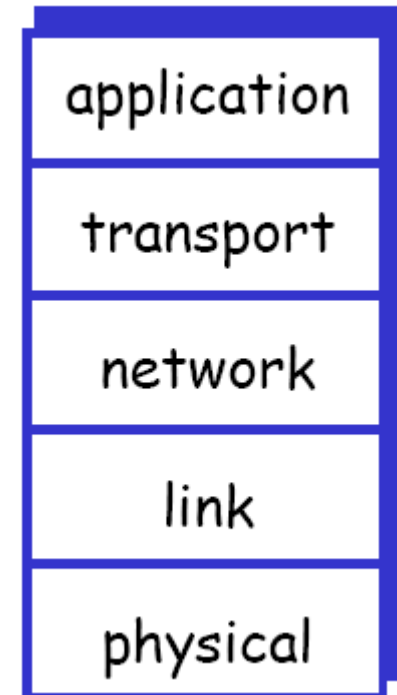


- Layers(계층) : 각 layer마다 서비스를 구현한다.
 - 내부 계층을 통해서 전체 서비스가 이루어진다.
 - 각 계층은 아래 계층과 연계하여 어떤 기능(서비스) 구현한다.

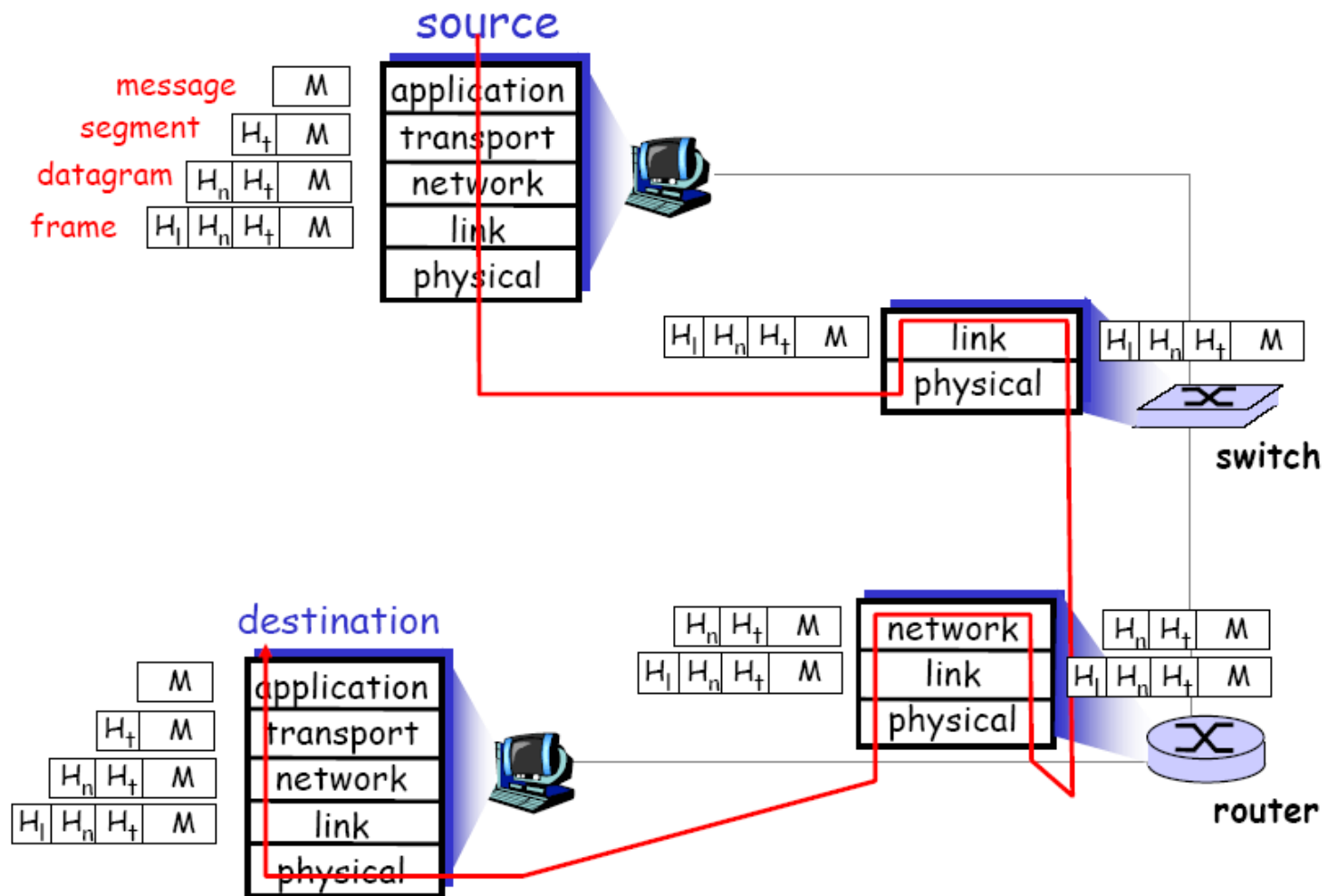
복잡한 시스템에 유용하다.

- 크고 복잡한 시스템을 잘 정의된 특정부분으로 논의 할 수 있게 해준다.
- 시스템의 유지 및 개선이 modularization으로 쉬워진다.
 - 각 layer를 개선하는 작업은 다른 layer에 대해서 투명하다.
 - 예를 들어 탑승 기능을 변화시켜도 탑승과정의 다른 계층에 대해서는 영향을 주지 않는다.
- Layering considered harmful?

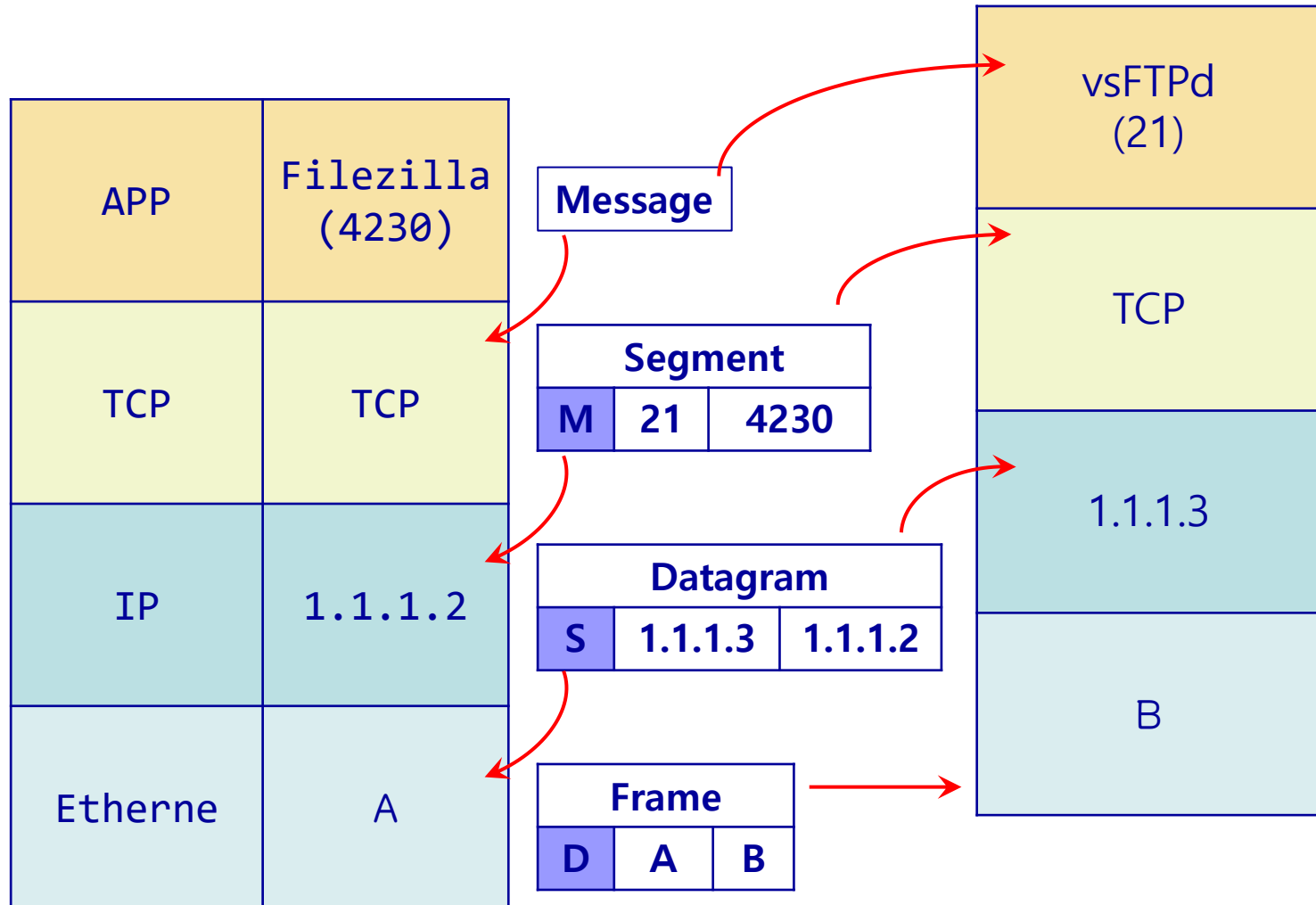
- Application : 네트워크 응용 프로그램
 - FTP, SMTP, HTTP
- Transport : 프로세서-프로세서간에 데이터 전송을 담당
 - TCP, UDP
- Network : 출발지(Host)에서 목적지(Host) 까지 datagram을 routing을 담당
 - IP, routing protocol
- Link : 네트워크 노드들 간에 data를 전송하는 기능을 담당
 - PPP, Ethernet
- Physical : bit “on the wire”



Encapsulation

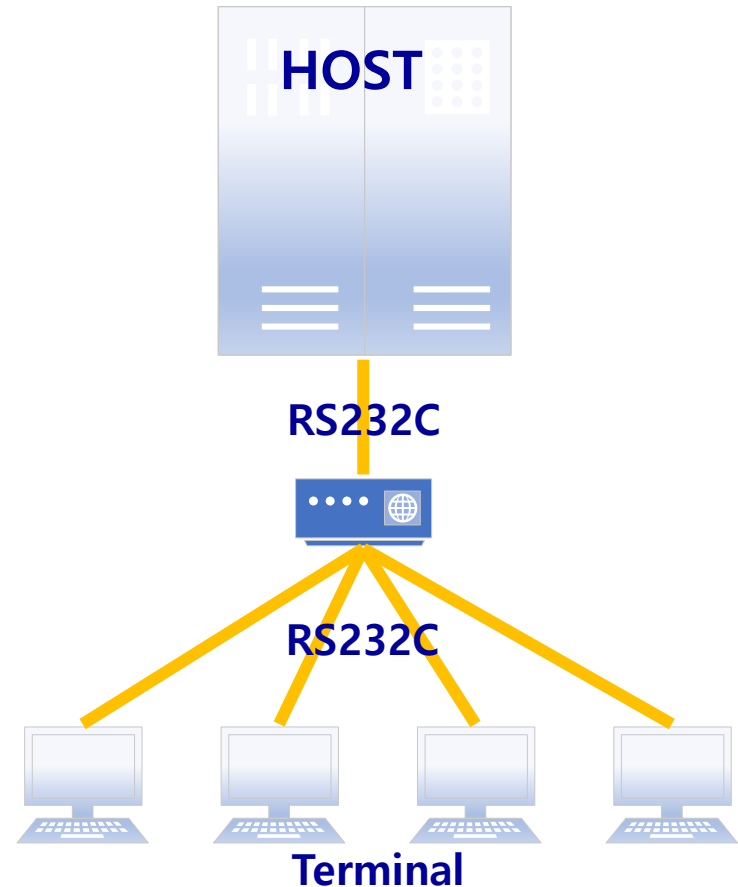


Encapsulation



응용 시스템의 구조 - 1 tier

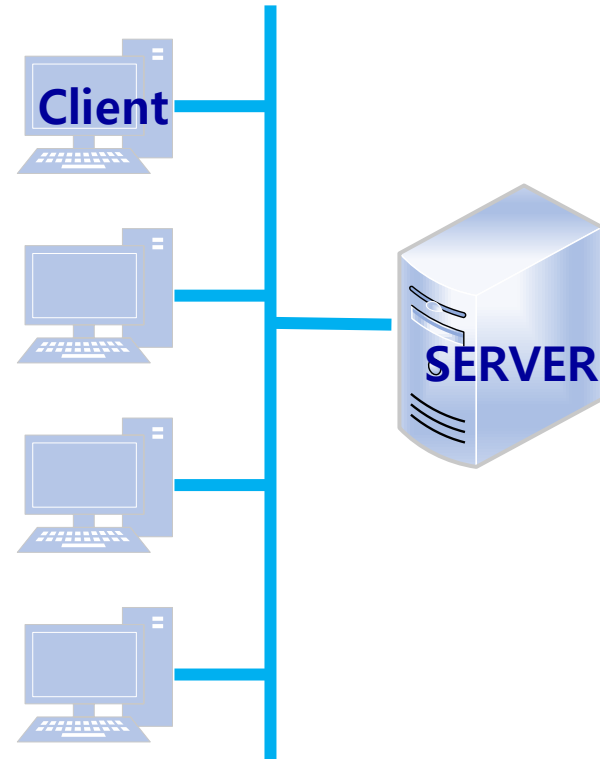
- 현재는 물리적인 구성으로 거의 사용되지 않지만 시뮬레이션 환경으로 많이 사용된다.
 - telnet, ssh 등은 1 tier의 시뮬레이션 환경이다.
- 1대의 Host에 다량의 terminal이 serial을 통해 연결된 환경이다.
 - Serial(RS232C)이 LAN(Ethernet)으로 바뀜
 - Host : 컴퓨터
 - Terminal : 모니터, 키보드 등 I/O 장치



➤ Client-Server 환경

- Server process와 Client process간에 상호 작용으로 네트워크가 구성된다.
- Server나 Client는 H/W가 아니라 S/W를 의미한다.
- Client : 서비스 요청
ex) chrome, 알ftp ...
- Server : 서비스 응답
ex) apache, vsftp...

➤ 인터넷의 기본 서비스 모델

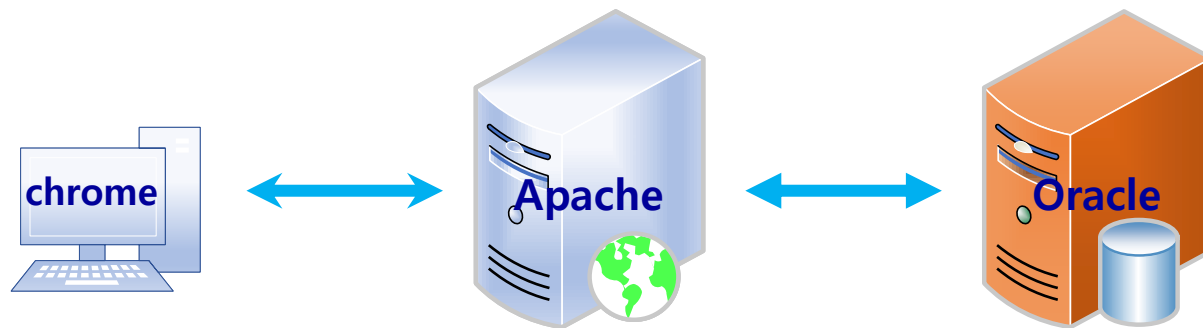


응용 시스템의 구조 - 3 tier

➤ Client-Server 환경의 확장

- 각 시스템은 Client이거나 Server이거나 둘 모두로 작동한다.
- Server Process가 기능의 일부를 외부 Server에 의존하는 경우 3 tier구조가 된다.
- H/W 구조 보다는 S/W 구조를 의미한다.
- 구축 유지 보수 비용이 2 tier에 비해서 획기적으로 줄어들어 대부분의 업무 환경이 3 tier 환경으로 구성된다.

ex) Web 서버(Application 서버)와 DB 서버



- 1.1 인터넷이란 무엇인가
- 1.2 네트워크의 가장자리(edge)
- 1.3 네트워크의 코어 (core)
- 1.4 네트워크 접속과 매체
- 1.5 ISP와 인터넷 백본
- 1.6 패킷교환 네트워크에서 지연과 손실
- 1.7 프로토콜계층과 서비스 모델
- 1.8 역사

1961 – 1972 : 패킷 교환기 개발

1972 – 1980 : 독점 네트워크와 인터넷워킹

Cerf 와 Kahn의 Internetworking 원칙

- minimalism : autonomy no internal changes required to interconnect network
- best effort service model
- stateless routers
- decentralized control

오늘날 인터넷의 구조

2장 Ethernet과 H/W

2.1 OSI 7Layer와 TCP/IP Layer

2.2 Network Topology

2.3 Ethernet

2.4 Ethernet의 확장

2.1 OSI 7Layer와 TCP/IP Layer

➤ TCP/IP model

- 1973년 9월 Sussex(서섹스) 대학에서 열린 컨퍼런스 조직된 INWD(International Network Working Group)에서 처음 발표됨

➤ OSI model

- 1983년 OSI(Open System Interconnection) 참조모델로 만들어지고 1984년 ISO 7489로 출간
- 단일 프로토콜이 아니라 다양한 프로토콜들의 표준 모델로 개발됨.

OSI model	TCP/IP model
Application	Application
Presentation	
Session	
Transport	Transport(TCP)
Network	Network(IP)
Data Link	Link(Physical)
Phyiscal	

- 모델별로 각 레이어에 대한 설명이 정확히 일치하지 않으나 대체적으로 비슷하다.
- Physical Layer
 - 각 개별 시스템간에 비트단위의 신호를 전송하는 책임을 담당한다. 그러나 개별 비트의 의미에 대해서는 관여하지 않으며 대체적으로 cable이나 물리적인 형태로 나타난다.
- Data Link Layer
 - 시스템을 H/W적으로 구별하며 frame의 형태로 Data를 전송하는 역할을 담당한다. CRC 에러 점검을 통해서 손상을 점검할 수 있으며 보통 랜카드, 브릿지, 스위치 등의 장비가 해당되는 계층이다.
 - 이 계층은 S/W와 H/W의 혼용으로 나타난다.
- Network Layer
 - 개별 시스템을 논리적으로 구별하는 역할을 담당한다. 인터넷에서 IP 가 이 계층에 속하며 여러 프로토콜의 Routing을 담당한다.- router

- Transport Layer
 - 상위 프로토콜을 구분하고 패킷의 순서와 손상을 보장한다.
 - TCP가 여기에 속하며 수신자가 발신자에게 응답을 전송하는 것이 여기서 담당한다.
 - 현재는 tran Layer까지 OS에 포함된다.
- Application
 - OSI에서는 session, presentation, application으로 나뉜다.
 - 응용 프로그램
 - : UI, socket 프로그래밍, OS 응용 프로그램

2.1 OSI 7Layer와 TCP/IP Layer

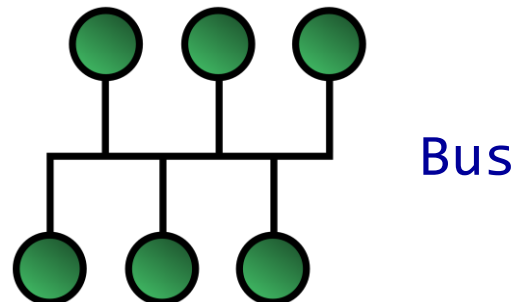
2.2 Network Topology

2.3 Ethernet

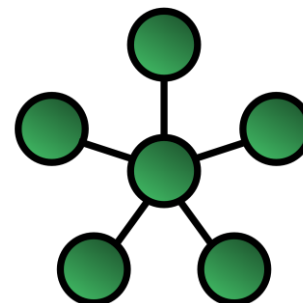
2.4 Ethernet의 확장

2.2 Network Topology

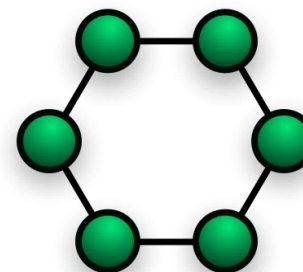
- Network Topology는 네트워크의 주요 구성 요소들을 물리적으로 연결된 것이거나 연결 방식을 의미한다.
- 가장 기본적인 토폴로지는 버스형, 스타형, 링형으로 구분한다.
- 케이블의 물리적인 구성으로 이들을 구분 할 수 없다.



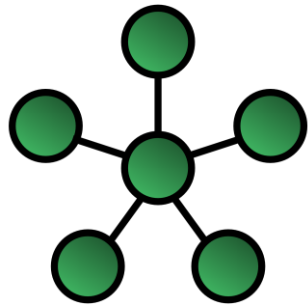
Bus



Star



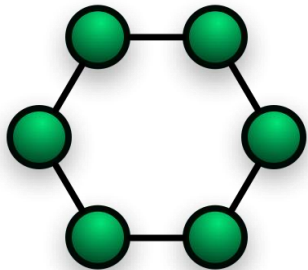
Ring



Star

➤ 스타형

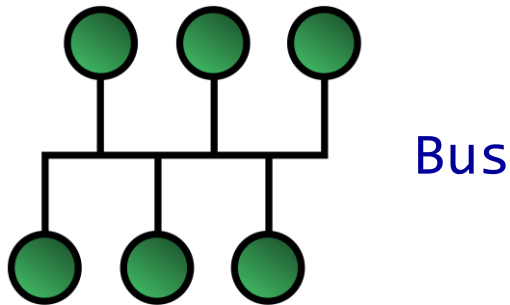
- 각 node에서 다른 node로 전달되는 경로가 단일 경로인 네트워크 구조이다.
- 대표적인 구조로 ATM이 있다.
- 90년대 까지 하위 단위 LAN에서 많이 사용되었으나 현재는 거의 사용되지 않는다.
- 25M, 155M, 2.1G ATM등의 다양한 규격이 제공된다.



Ring

➤ 링형

- 충돌 네트워크가 아님으로 동일 스펙의 버스형보다 성능이 우월하다.
- 국내에서 많이 사용된 링형은 FDDI가 있다.
- 버스형이나 스타와 달리 node 구성에 단말기가 거의 없다.



➤ Bus형

- 가장 대표적인 패킷망 네트워크이다.
- 현대에 가장 많이 사용되며 LAN망의 99% 이상이다.
- 복잡한 장비 없이도 구성됨으로 가성비가 매우 우수하다.
- Ethernet

➤ 세가지 토폴로지의 특성이 현대 네트워크에서 별 의미가 없다.

- 장애가 다른 노드에 영향을 끼치는 정도
- 장애 node에 대한 관리 비용
- 구현 회선의 길이
- 위 사항들이 큰 의미를 갖지 않는다.

2.1 OSI 7Layer와 TCP/IP Layer

2.2 Network Topology

2.3 Ethernet

2.4 Ethernet의 확장

➤ Unicast

- 1:1 통신 패킷
- Sender(src)와 destination(dst)주소가 특정 node나 host의 주소인 경우

➤ Broadcast

- 1:N 통신
- Destination 주소가 불특정 다수인 패킷이다.
- FF:FF:FF:FF:FF:FF, IP 마지막 주소 등

➤ Multicast

- 1:N 통신
- 대상이 미리 지정한 그룹인 경우.
- D class IP

➤ Ethernet

- 로버트 밥 메트칼프(Robert Bob Metcalfe)에 의해서 최초 설계되었다.
(10Base5, 10Base2)
- CSMA/CD
- Ethernet는 IEEE(Institute of Electrical and Electronics Engineers)의 공식 용어가 아니다.
- Bus형의 대표적인 형식이다.
- Link layer 에 해당한다.(Physical Layer를 포함한다.)

➤ CSMA/CD(Carrier Sense Multiple Access with Collision Detect)

- Carrier Sense
- Multiple Access
- Collision Detect

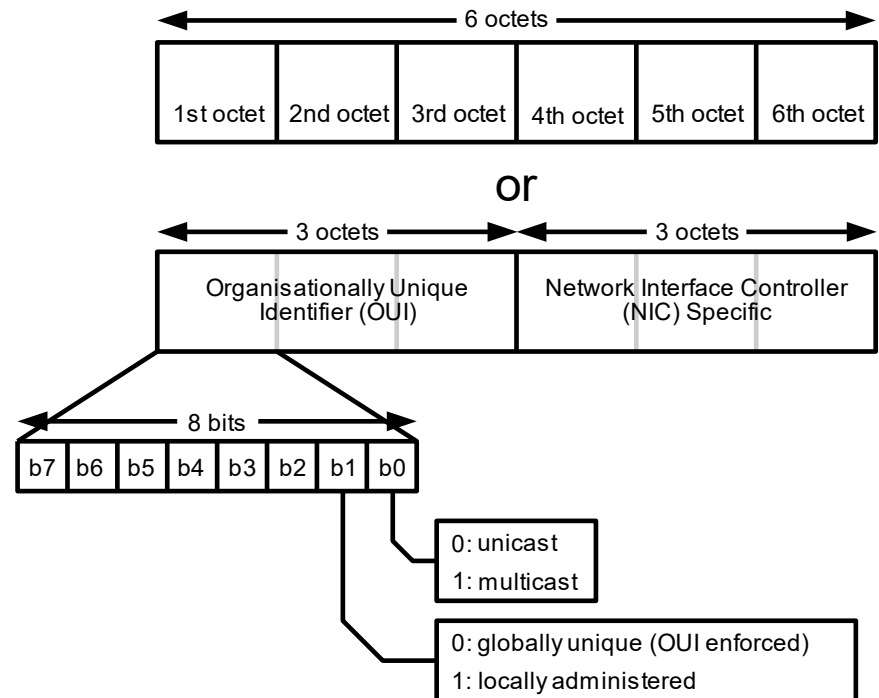
IEEE 802.3 네트워크에서 매체 사용을 위한 제어 구조 방식. 즉, 어떻게 데이터 패킷을 통신 케이블에 올려 놓느냐 하는 방법이다. 연결된 케이블의 전송신호를 감시하고 있다가 충돌이 없으면 통신하는 방식이다. 10Mbps 통신속도에 2.8km 통신 선로에서는 최소 패킷의 길이는 64바이트여야 충돌을 알 수 있다. 만일 충돌이 일어나면 보낸 신호를 취소하고 임의의 시간을 기다린 다음에 다시 전송을 시도한다. 만약에 많은 수의 노드가 전송을 시도하여 계속해서 충돌이 일어나면 전송 효율이 대폭 떨어지므로, 이런 현상이 방지하기 위해서는 통신 케이블을 규칙에 따라 세심하게 연결해야 하며, 이런 네트워크는 브리지 또는 라우터로 분할(segmenting)하여 사용(filter)해야 한다.

선로의 모든 data는 Multiple Access 방식으로 전달됨으로 각각 시스템의 네트워크 인터페이스들은 통신을 하지 않을 때도 늘 통신 데이터를 filtering 해야 하는 문제를 가지고 있고 단일 네트워크에 일정 수준의 데이터량을 넘어가면 네트워크를 사용할 수 없는 단점 있지만 구현이 매우 쉽고 단순해서(가격이 저렴) 현재 인터넷에 연결된 네트워크의 거의 대부분이 ethernet으로 구성되어 있다. 확장규격으로 100M를 전송하는 100BaseTx, 100BaseFx등과 1G를 전송하는 1000BaseT 등이 사용된다.

MAC (Media access control)

➤ MAC address

- node를 구분하는 고유 주소
- 48bit , 6byte,
00:00:00:00:00:00
- 3byte OUI와 3byte NIC로 구성된다.
- NIC에 할당된다.
- 영구적이고 전역적인 고유 식별을 위해 고안되었으나 사실 각 디바이스의 진단 프로그램을 이용하면 변경 가능하다.
- FF:FF:FF:FF:FF:FF



MAC (Media access control)

```
CA 명령 프롬프트
c:\>ipconfig /all

Windows IP 구성

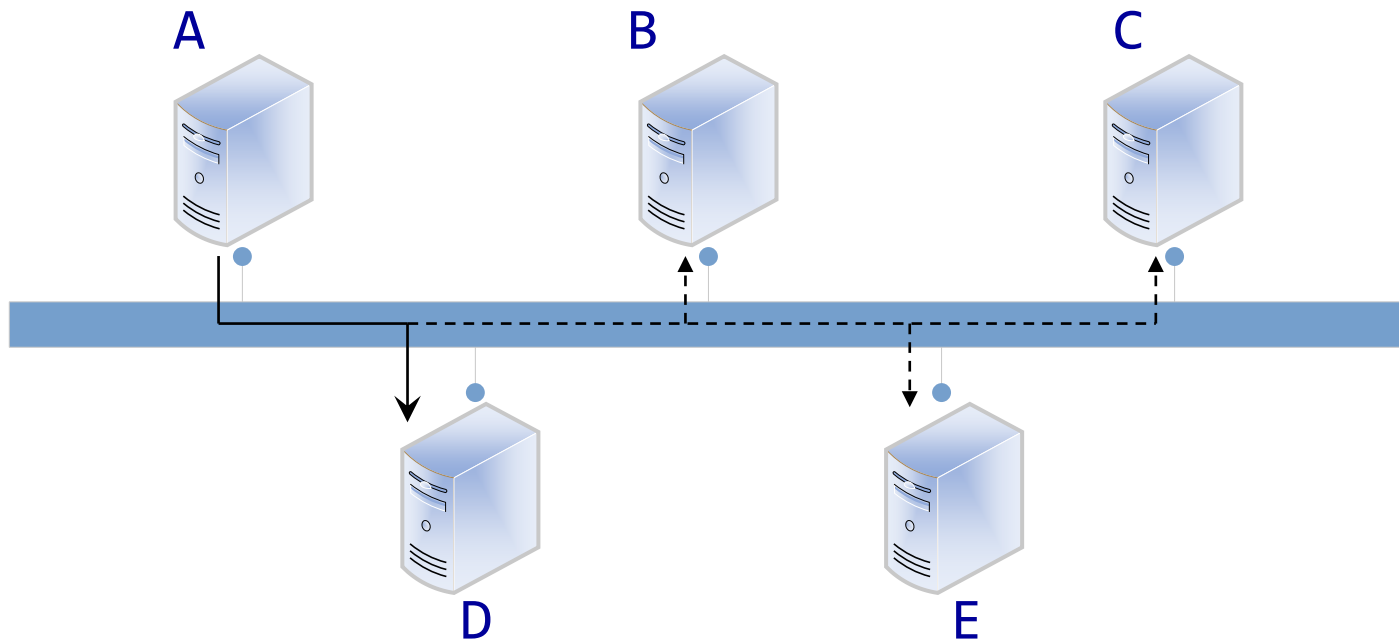
호스트 이름 . . . . . : birochya
주 DNS 접미사 . . . . . :
노드 유형 . . . . . : 혼성
IP 라우팅 사용 . . . . . : 아니요
WINS 프록시 사용 . . . . . : 아니요

이더넷 어댑터 이더넷:

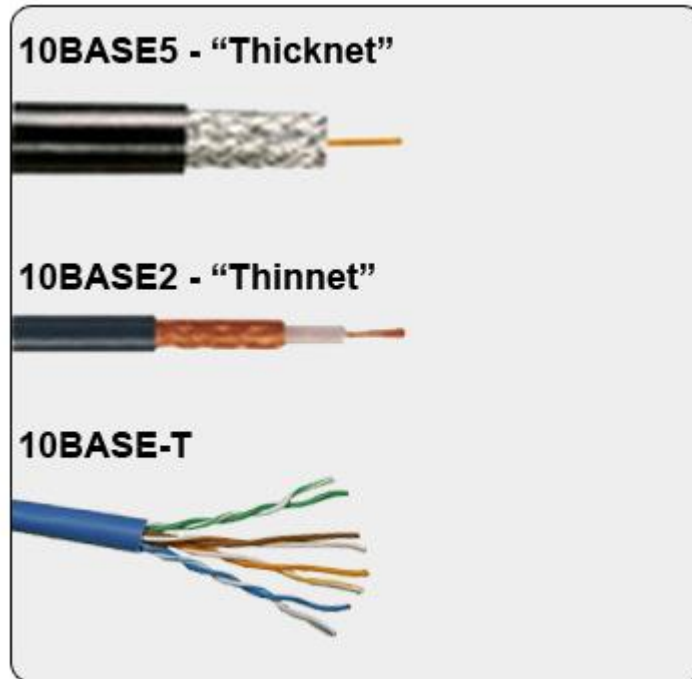
연결별 DNS 접미사 . . . . :
설명 . . . . . : Realtek PCIe GbE Family Controller
물리적 주소 . . . . . : A8-5E-45-57-85-EB
DHCP 사용 . . . . . : 아니요
자동 구성 사용 . . . . . : 예
링크-로컬 IPv6 주소 . . . : fe80::4469:b9db:f26e:c772%7(기본 설정)
IPv4 주소 . . . . . : 192.168.10.3(기본 설정)
서브넷 마스크 . . . . . : 255.255.255.0
기본 게이트웨이 . . . . . : 192.168.10.1
DHCPv6 IAID . . . . . : 111697477
DHCPv6 클라이언트 DUID . . : 00-01-00-01-26-F2-6A-6E-A8-5E-45-57-85-EB
DNS 서버 . . . . . : 203.248.252.2
                  192.168.10.1
Tcpip를 통한 NetBIOS . . . : 사용

c:\>
```

➤ 데이터 전송방식

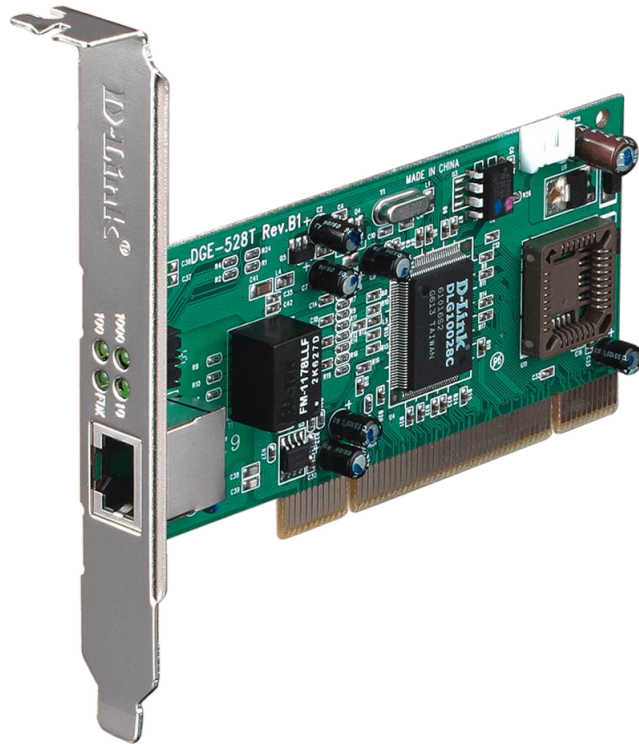


➤ 케이블

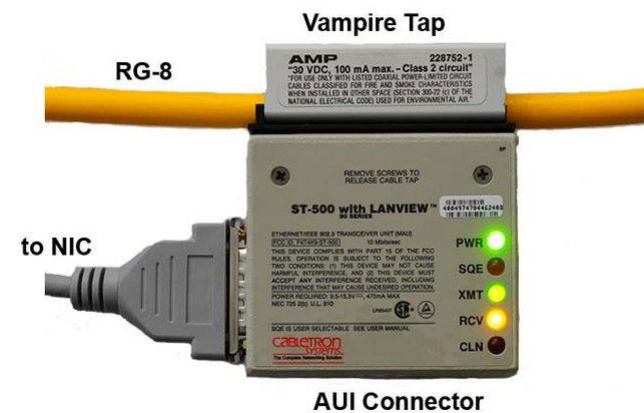
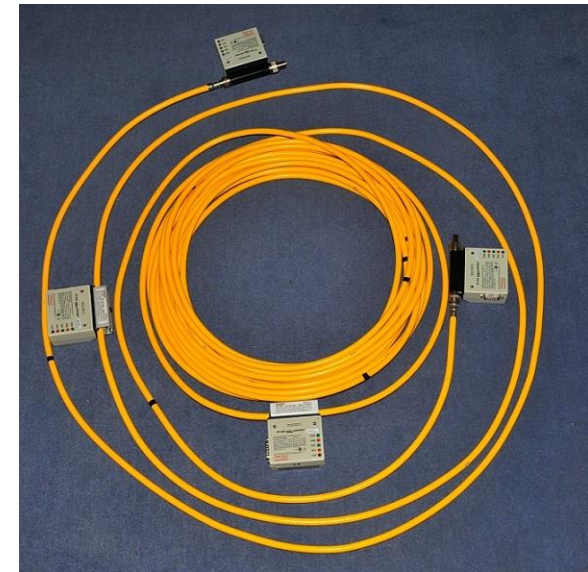
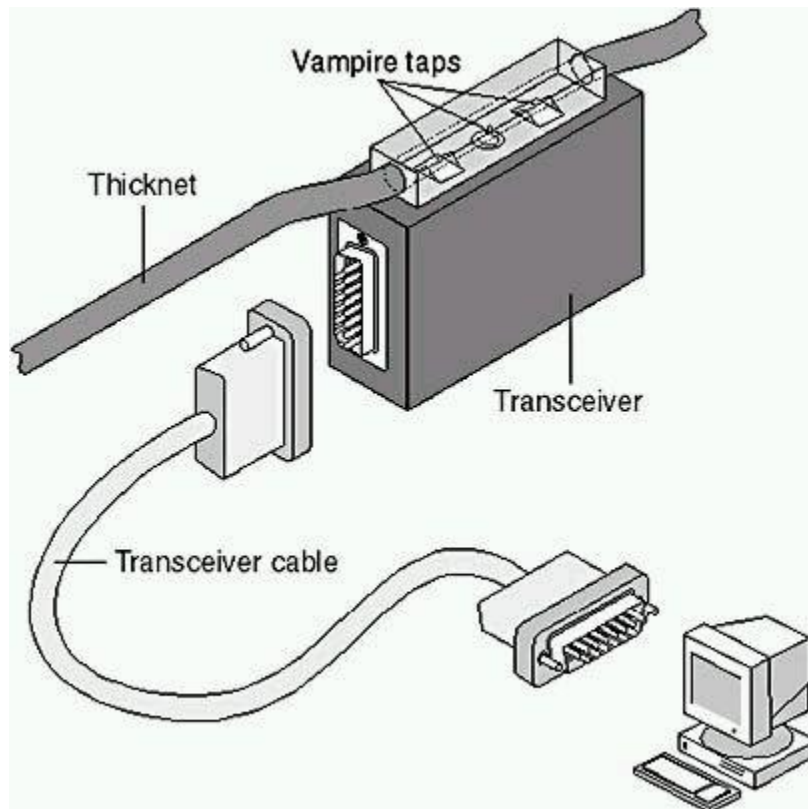


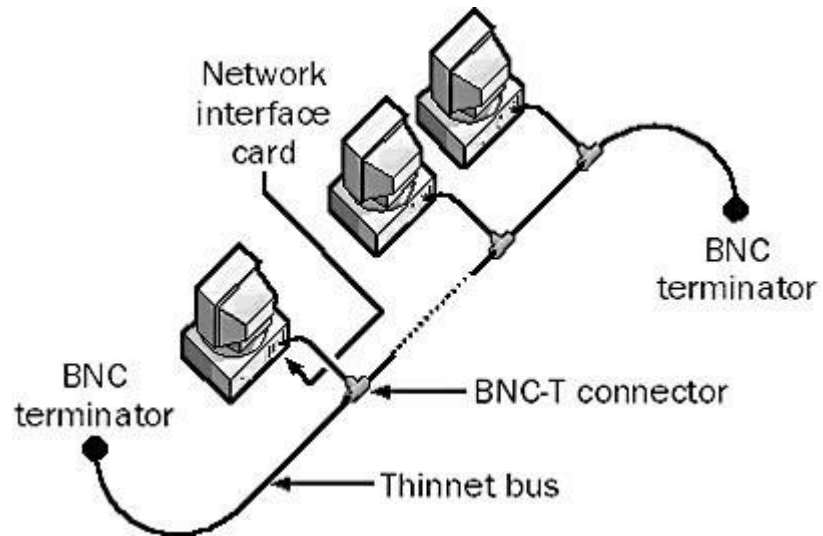
NIC(Network Interface Card)

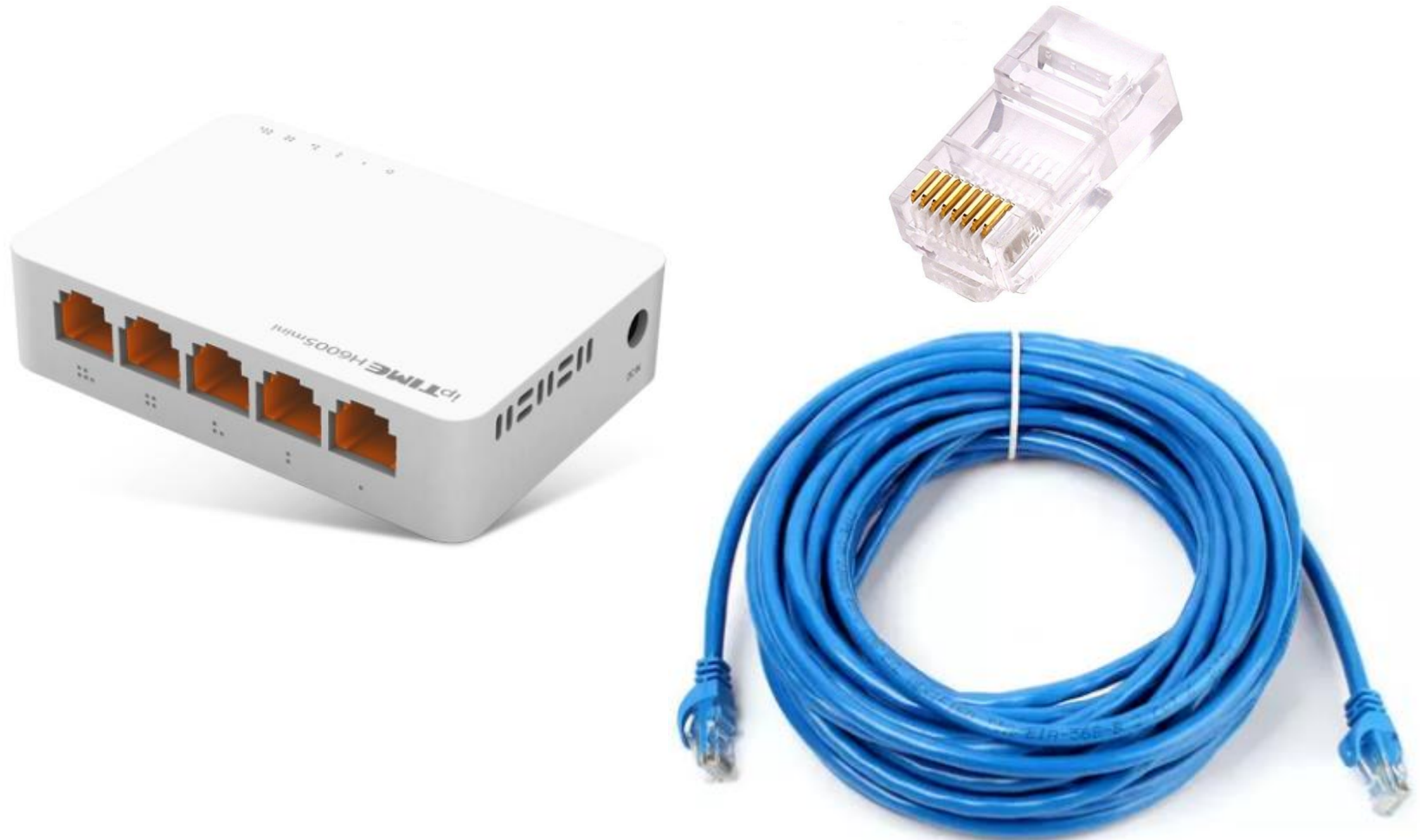
➤ NIC



10Base5







➤ Ethernet의 성능

- 일정 수준 이상으로 node가 증가하면 각 node가 사용가능한 대역폭은 줄어든다.
- node가 일정 수준을 넘어서면 네트워크의 가용성이 심각하게 저하된다.

Total Packets



of node

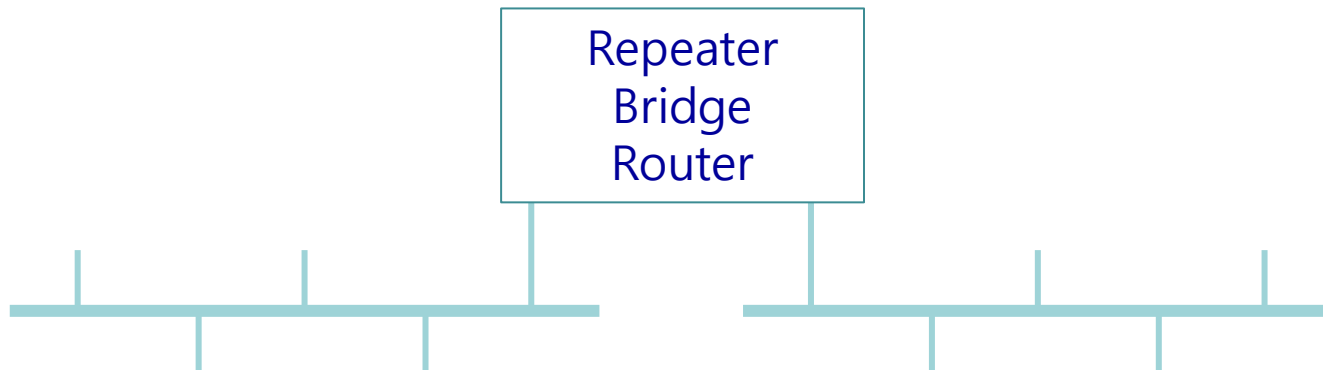
2.1 OSI 7Layer와 TCP/IP Layer

2.2 Network Topology

2.3 Ethernet

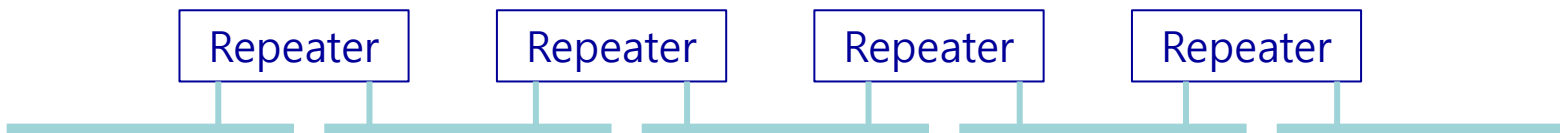
2.4 Ethernet의 확장

- 이더넷은 단일 segment만으로 모든 node의 수용이 불가능함으로 필연적으로 확장되어야 한다.
 - Repeater
 - Bridge(Switch)
 - Router



➤ Repeater

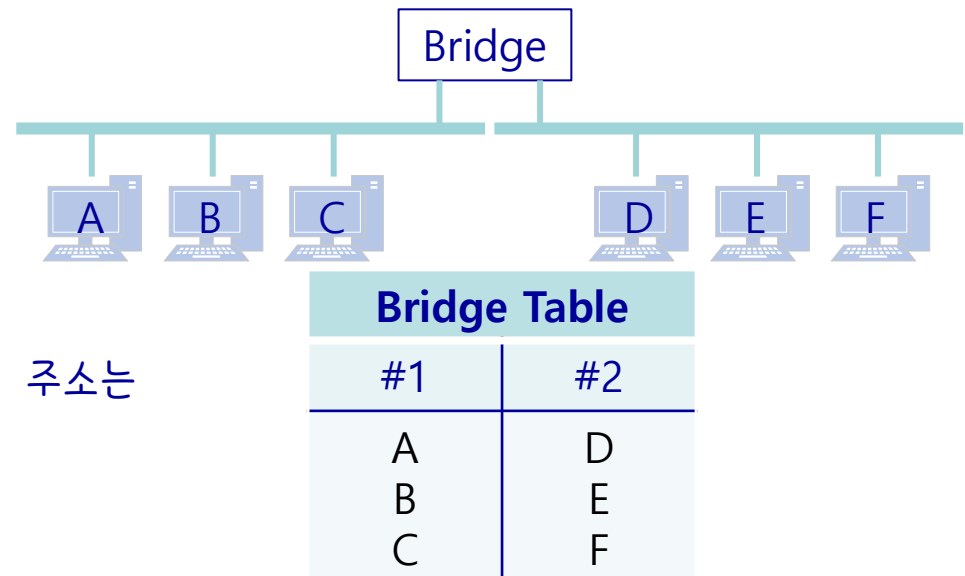
- 신호를 증폭하는 physical 장비이다.
- 90년대까지 주로 LAN에서 segment를 연결하는데 쓰였다.
- 더미 허브(Dummy HUB)도 동일한 기능을 제공한다.
 - 허브란 용어는 90년대까지 더미허브를 의미했지만 시장에서 더미허브가 없어지고 이제는 스위치 허브를 허브라고 부른다.
- 5,4,3 규칙
 - 80,90년대 제조사의 test 테스트 환경일 뿐 스펙이나 규격이 아니다.
 - 5개의 segment를 4개의 리피터로 연결하고 패킷이 전송이 정상적이면 장치에 이상은 없다. 단 이때 3개의 segment만 node를 갖는다.
 - 단지 TEST 환경일 뿐이다. 이외 아무런 의미는 없다.



- 썬 네트워크를 연결할 때 이렇게 연결하면?? DIJUM!!

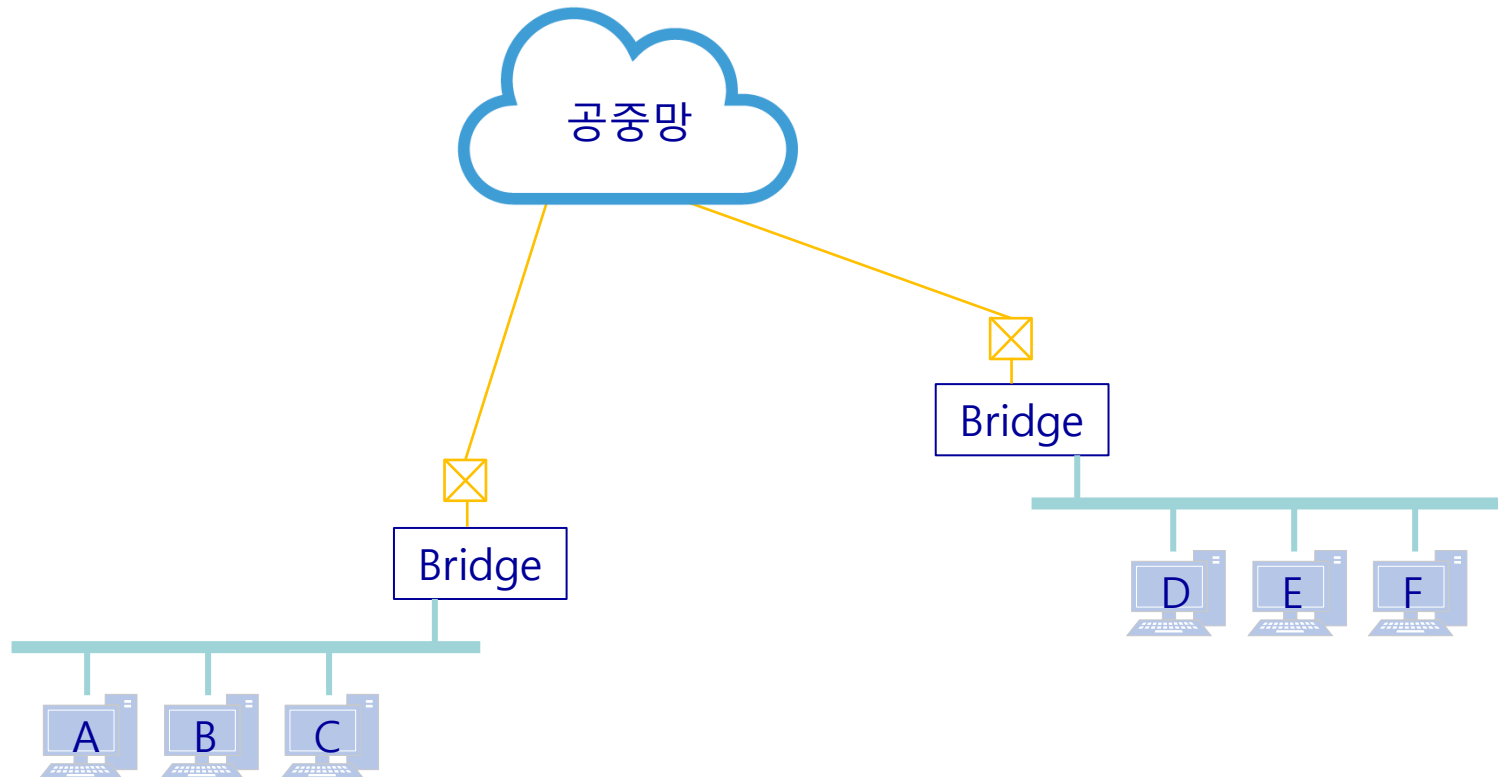
➤ Bridge

- Mac address 기반으로 네트워크의 패킷을 필터링한다.
- 네트워크를 매우 크게 확장 가능하다. - 대규모 단일 네트워크 구성이 가능하다.
- WAN 망으로 구성할 때 Broadcast storm 현상이 발생 한다.
 - LAN 내부에서 발생하기는 어렵다.
- 기능
 - Learning
 - Filtering
 - Forwarding
 - Flooding
 - Aging
- Multi Port Bridge
= Switch HUB
- Bridge Table에 없는 MAC 주소는
어떻게 처리될까?



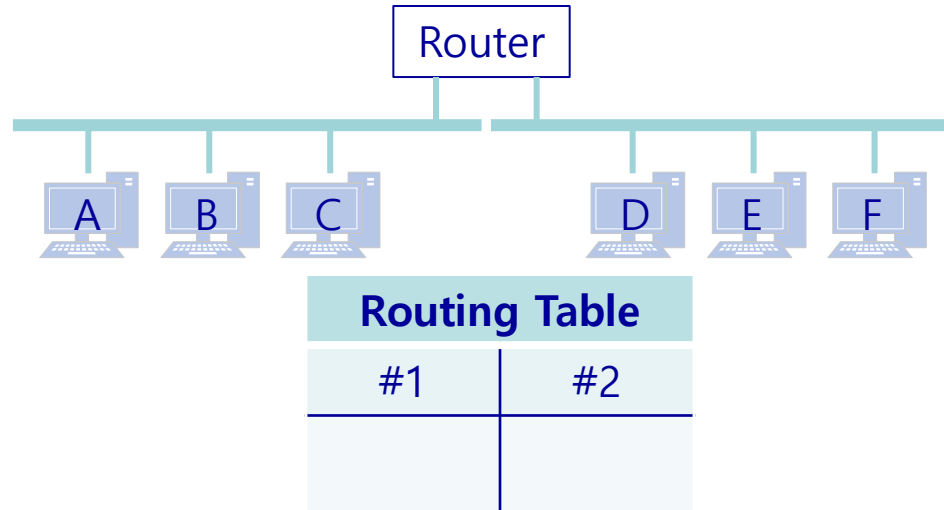
Broadcast storm

- Broadcast storm은 주로 LAN TO WAN 구성에서 나타나는 현상으로 Serial 병목 구간에 forwarding되는 broadcast packet로 인해 발생한다.
 - 70~80년대 흔한 문제였으나 이후는 네트워크 구성이 바뀌어 보기 어렵다.



➤ Router

- IP와 같이 routing 이 지원되는 network layer protocol인 경우에 router를 이용해서 네트워크 연결하고 분할하는 것이 가능하다.
- 장치의 메모리를 훨씬 효율적으로 이용한다.
- 처리지연이 증가하지만 전체 성능에 영향은 거의 없다.
- 경로를 배정하는 알고리즘이 포함된다. (RIP, OSPF,)



3장 IP(Network Layer)

3.1 Network Layer와 IP

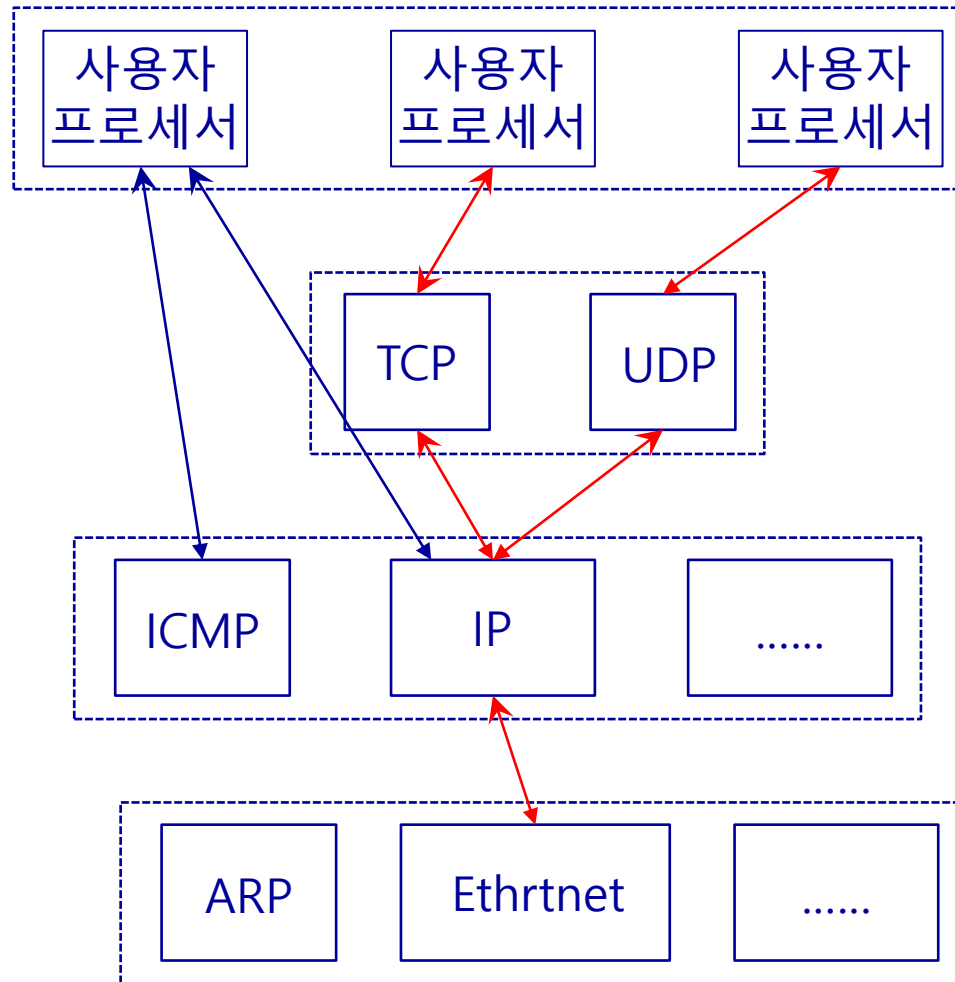
3.2 네트워크 분할

3.3 VLSM

3.4 NAT

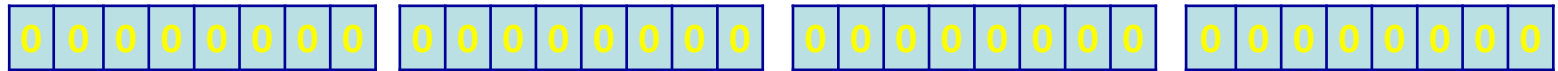
4.4 관련 protocol

3.1 Network Layer와 IP



➤ IP

- host를 구분하는 고유 주소
 - 인터넷에서 유일해야 한다.
- 32bit로 구성됨



00000001 00000001 00000001 00000001 → 1. 1. 1. 1

00000001 10000001 00000011 00000001 → 1. 129. 3. 1

- IP는 단독으로 설정되지 않으며 netmask(subnetmask), gateway와 함께 구성된다.

```
CA. 명령 프롬프트
c:\>ipconfig /all

Windows IP 구성

호스트 이름 . . . . . : birochya
주 DNS 접미사 . . . . . :
노드 유형 . . . . . : 혼성
IP 라우팅 사용 . . . . . : 아니요
WINS 프록시 사용 . . . . . : 아니요

이더넷 어댑터 이더넷:

연결별 DNS 접미사 . . . . :
설명 . . . . . : Realtek PCIe GbE Family Controller
물리적 주소 . . . . . : A8-5E-45-57-85-EB
DHCP 사용 . . . . . : 아니요
자동 구성 사용 . . . . . : 예
링크-로컬 IPv6 주소 . . . : fe80::4469:b9db:f26e:c772%7(기본 설정)
IPv4 주소 . . . . . : 192.168.10.3(기본 설정)
서브넷 마스크 . . . . . : 255.255.255.0
기본 게이트웨이 . . . . . : 192.168.10.1
DHCPv6 IAID . . . . . : 111697477
DHCPv6 클라이언트 DUID. . : 00-01-00-01-26-F2-6A-6E-A8-5E-45-57-85-EB
DNS 서버 . . . . . : 203.248.252.2
                  192.168.10.1
Tcpip를 통한 NetBIOS. . . : 사용

c:\>
```

- Class는 현재 기술적인 스펙이 아니라 관리 스펙으로 사용되지만 별 다른 의미는 없다.

Class	범위	# of network	# of host	ex
A	0~127	128	256^3	110.0.0.0 ~ 110.255.255.255
B	128~191	64×256	256^2	180.10.0.0 ~ 180.10.255.255
C	192~223	32×256^2	256	203.12.34.0 ~ 203.12.34.255
D	224~239			

- Class를 기반으로 IP를 할당하는 관리 체계는 IP 낭비로 인해 실패했다.
- ISP, 합당한 규모의 조직이 아니면 class기반의 IP는 할당 받기 어렵다.
- 이런 관리 체제변화와 초고속 통신망의 발달로 개별 edge 네트워크에서 라우터의 수요가 급감하고 있다.

➤ IP 주소의 구조

IP 주소는 일반적인 주소와 동일하게 sequence number가 아니라 address이다

.

경기도 파주시 문향로 11 520동 5230호

동네 주소

집 주소

0000000000 0000.....

.....00000

Network ID(주소)

host ID(주소)

➤ Netmask

IP 주소내에서 network 주소와 host 주소를 구분해준다.

- IP는 문자 주소와 달리 숫자만으로 구성되어 있어 netmask 없이는 host 주소를 구분할 방법이 없다.

➤ Gateway(게이트웨이)

- host는 인터넷의 모든 다른 host와 통신이 가능하지만 자신이 직접 패킷을 보내는 대상은 동일한 네트워크에 속한 host 까지로 제한된다.
- 자신과 다른 네트워크 주소를 갖는 host에 패킷을 보내는 경우 이를 대신해주는 시스템 즉 외부와 연결된 시스템이 필요한데 이를 gateway라고 한다.
- 네트워크에 여러 개의 gateway가 있는 경우라도 default gateway는 반드시 1개만 지정 가능하다.
- 2000년대 이후 다중 gateway는 네트워크 설계에서 고려되는 경우가 드물다.
 - 인터넷 인프라의 신뢰성 향상으로 Backup 라인의 중요도가 이전에 비해서 매우 낮다.
- 게이트웨이역할은 라우터나 공유기가 담당한다.

➤ 인터넷 사용을 위한 3가지 설정

IP, Netmask, Gateway

➤ Network ID

- IP 주소내에 netmask가 지정된 곳까지의 주소
- Netmask는 Network ID를 1로 마킹한다.

```
0 0 0 0 0 0 0 1  0 0 0 0 0 0 0 1  0 0 0 0 0 0 0 1  0 0 0 0 0 0 0 1
-----
1 1 1 1 1 1 1 1  1 1 1 1 1 1 1 1  1 1 1 1 1 1 1 1  0 0 0 0 0 0 0 0
```

IP : 1.1.1.1에서 1.1.1.0이 network id다.

1.1.1.1 인 host는 1.1.1.0에서 1.1.1.255 까지의 host들과 직접 통신이 가능하며 이외의 host와는 gateway를 통해서만 통신 할수 있다.

- Network ID는 IP와 Netmask를 &(and) 연산해서 계산한다.

ex)

203.248.224.12/255.255.255.0의 네트워크 ID를 계산한다.

203.248.224.12 → 11001011 11111000 11100000 00001100

255.255.255.0 → 11111111.11111111.11111111.00000000

1 1 0 0 1 0 1 1 1 1 1 1 1 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 1 1 0 0

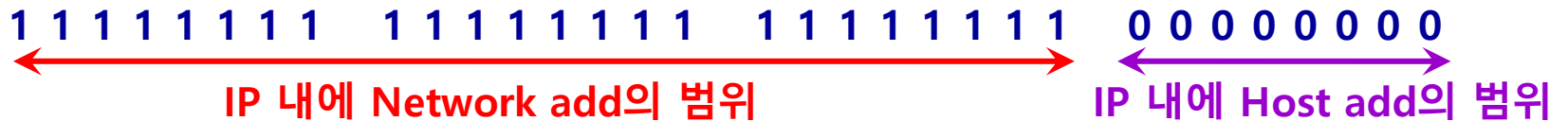
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0

1 1 0 0 1 0 1 1 1 1 1 1 1 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0

11001011 11111000 11100000 000000 → 203.248.224.0

Network ID는 203.248.224.0이다.

203.248.224.0 ~ 203.248.224.255 까지가 네트워크 범위다.



- Netmask는 IP 32bit중에 1로 마킹된 부분까지 네트워크 주소임을 의미한다.
- IP에 Network ID부분이 동일한 IP는 모두 동일한 네트워크에 속해 있다.
 - 경기도 파주시 문향로 11 520동 5230호 주소내에 ' 경기도 파주시 문향로 11 520동' 까지 동일한 주소를 갖는다면 같은 동네인 것과 동일한 개념이다.
- Netmask는 숫자가 아니라 위치 개념으로 이해한다.

Network ID와 Broadcast 주소

- 네트워크 주소내에 IP중에 맨처음 IP와 맨 마지막 IP는 사용 할 수 없다.
 - 맨 처음 IP : Network ID
 - 맨 마지막 IP : Broadcast IP

- 1.1.1.0 ~ 1.1.1.255
 - Network ID : 1.1.1.0
 - Broadcast IP : 1.1.1.255

- 주소 표기법
 - 1.1.1.2/255.255.255.0 → 1.1.1.2/24
 - 1.1.2.3/255.255.0.0 → 1.1.2.3/16

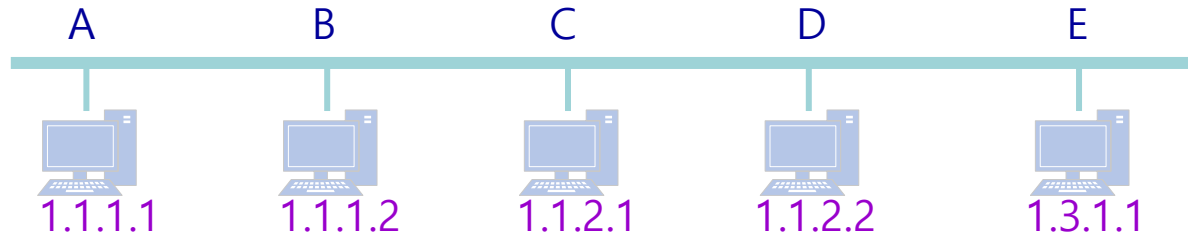
- Netmask를 bit 개수로 표현

➤ 인터넷에서 특수 목적으로 정의되어 사용하면 안되는 IP

Zero add	0.0.0.0 ~ 0.255.255.255	특수 목적으로 예약됨 0.0.0.0 : any
Network add Direct broadcast add	네트워크 첫 번째 IP 네트워크 마지막 IP	
Local broadcast add	255.255.255.255	인터넷 전체의 broadcast add 255.255.255.255 : host
Multicast add	D class	Multicast를 위해 할당
Loopback add	127.0.0.0 ~ 127.255.255.255	자기 자신을 의미하는 IP
Zero configuration	169.254.0.0 ~ 169.254.255.255	DHCP나 RARP 서버가 없는 PtP또는 Wireless 환경에서 자동 설정되는 IP
사설	A : 10.0.0.0 ~ 10.255.255.255 B : 172.16.0.0~172.31.255.255 C : 192.168.0.0 ~ 192.168.255.255	사설 IP

➤ 각 IP가 속한 네트워크 범위를 계산한다.

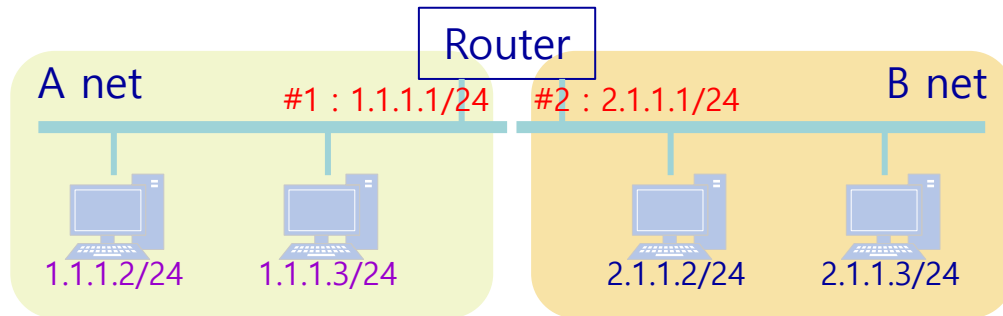
- | | |
|------------------------|-------------------------|
| ① IP : 210.113.29.45 | netmask : 255.255.255.0 |
| ② IP : 29.1.210.35 | netmask : 255.255.0.0 |
| ③ IP : 2.113.1.1 | netmask : 255.0.0.0 |
| ④ IP : 10.35.3.9 | netmask : 255.255.0.0 |
| ⑤ IP : 110.194.129.199 | netmask : 255.255.0.0 |
| ⑥ IP : 9.213.229.204 | netmask : 255.255.255.0 |
| ⑦ IP : 8.1.2.4 | netmask : 255.0.0.0 |
| ⑧ IP : 190.13.48.245 | netmask : 255.255.255.0 |



➤ Netmask 설정과 통신 범위

- 255.255.255.0
- 255.255.0.0
- 255.0.0.0

Router와 Network ID



Routing Table	
#1	#2
1.1.1.0/24	2.1.1.0/24

	Network ID	Broadcast add	Gateway IP
A network	1.1.1.0	1.1.1.255	1.1.1.1
B network	2.1.1.0	2.1.1.255	2.1.1.1

3.1 Network Layer와 IP

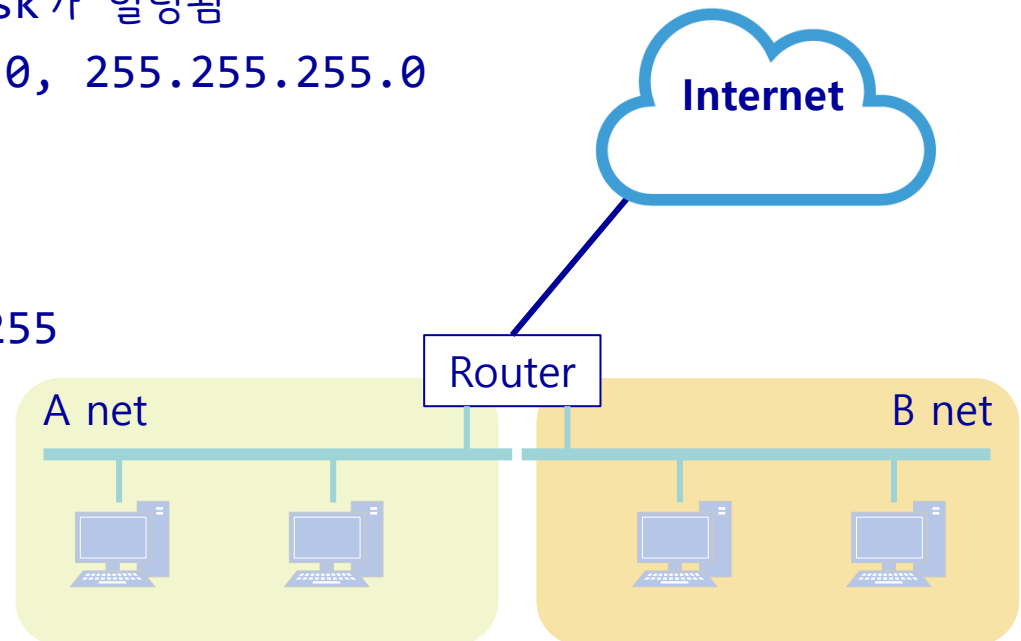
3.2 네트워크 분할

3.3 VLSM

3.4 NAT

4.4 관련 protocol

- 네트워크 분할과 병합은 동일한 기술을 이용한다.
- Netmask는 네트워크의 범위를 결정하는 정보이다. 초기 Netmask는 Class oriented netmask였으나 네트워크 구조가 복잡해지고 IP의 낭비가 심해져 이를 보완하기 위해 bitmasking을 이용 네트워크를 분할(병합)한다.
- Class oriented Netmask
 - IP에 따라 자동으로 Netmask가 할당됨
 - 255.0.0.0, 255.255.0.0, 255.255.255.0
 - 0, 255만이 사용된다.
- **BUT !!**
 - 할당 : 1.1.1.0~1.1.1.255
 - A network : 100ea
 - B network : 98ea



bitmasking을 통한 네트워크 분할

00000001.00000001.00000001.00000001 : 1.1.1.1
11111111.11111111.11111111.00000000 : 255.255.255.0

할당된 네트워크 범위 : 1.1.1.0~1.1.1.255



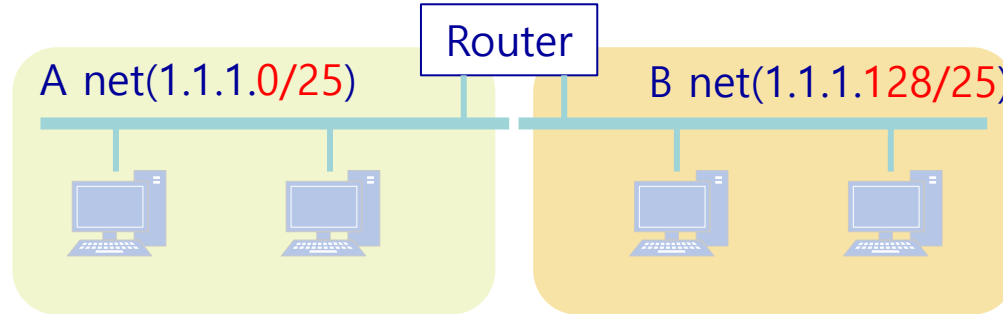
11111111.11111111.11111111.00000000 : 255.255.255.0
Network add Host add

11111111.11111111.11111111.10000000 : 255.255.255.128
Network add Host add

0	00000000 11111111	00000000 01111111	0 127	1.1.1.0 ~ 1.1.1.127	A net
1	00000000 11111111	10000000 11111111	128 255	1.1.1.128 ~ 1.1.1.255	B net

➤ 네트워크 분할

- 1.1.1.0 ~ 1.1.1.255
- Netmask : 255.255.255.128
- 1.1.1.0/25, 1.1.1.128/25 두개의 네트워크로 분할됨



➤ 분할 확인

- 1.1.1.0~1.1.1.127 → 1.1.1.1 → 1.1.1.0
- 1.1.1.128~1.1.1.255 → 1.1.1.129 → 1.1.1.128
 - 두 IP를 netmask 255.255.255.128과 & 연산한다.

bitmasking 을 이용한 네트워크 분할

➤ 1.1.1.0 ~ 1.1.1.255 를 4개의 네트워크로 분할한다.

00	000000 111111	00000000 00111111	0 63	1.1.1.0 ~ 1.1.1.63	A net
01	000000 111111	01000000 01111111	64 127	1.1.1.64 ~ 1.1.1.127	B net
10	000000 111111	10000000 10111111	128 191	1.1.1.128 ~ 1.1.1.191	C net
11	000000 111111	11000000 11111111	192 255	1.1.1.192 ~ 1.1.1.255	D net

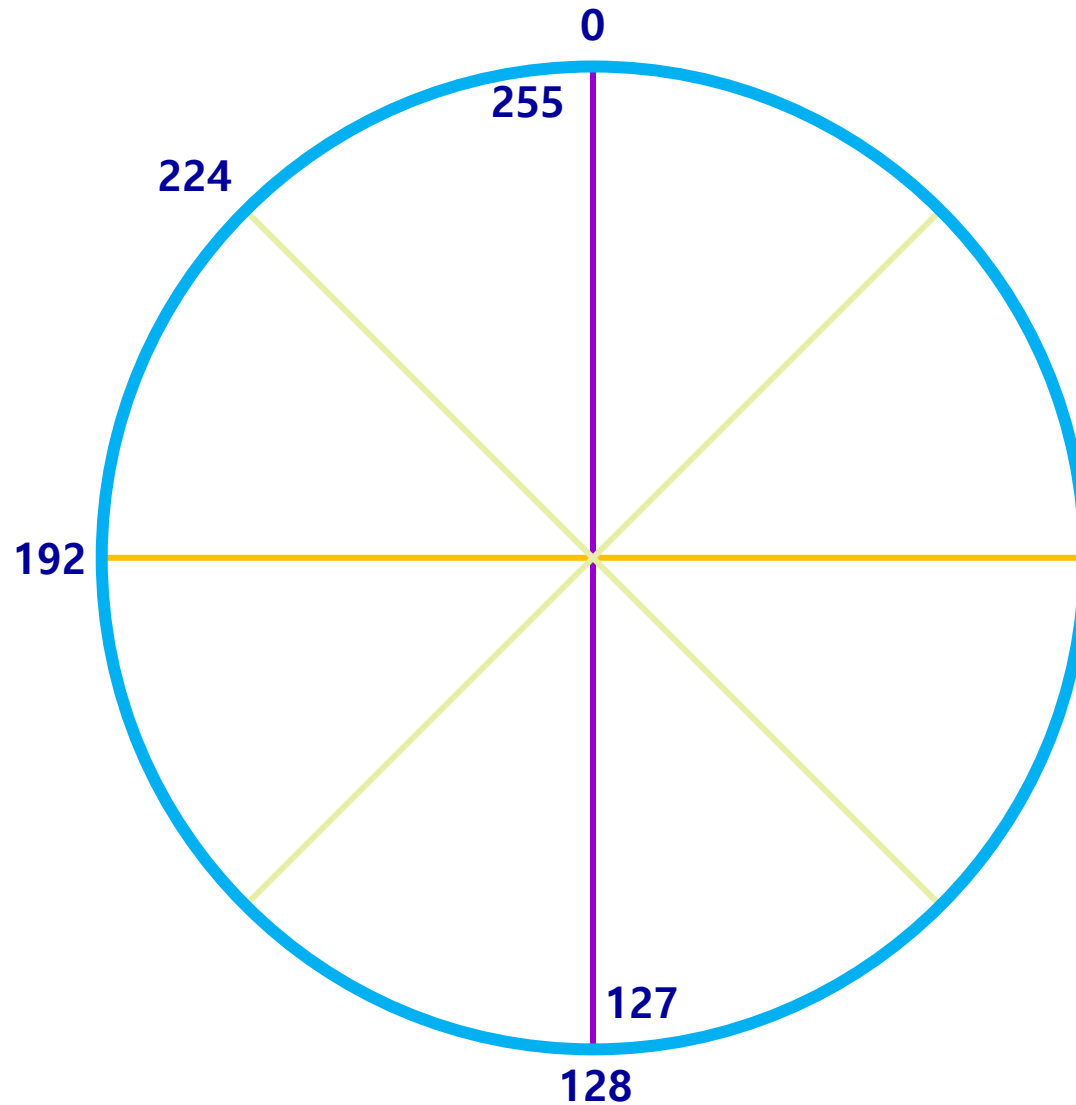
- 255.255.255.192
- 4개의 네트워크는 각 64개의 IP를 가진다.
- 각 네트워크는 network id, broadcast add를 제외한 62개의 IP를 host에 할당 가능하다.

bitmasking 을 이용한 네트워크 분할

255.255.255.

netmask		# of network	# of host
0	00000000	1	256(254)
128	10000000	2	128(126)
192	11000000	4	64(62)
224	11100000	8	32(30)
240	11110000	16	16(14)
248	11111000	32	8(6)
252	11111100	64	4(2)
254	11111110	128	2(0)
255	11111111	256	0

bitmasking 을 이용한 네트워크 분할



3.1 Network Layer와 IP

3.2 네트워크 분할

3.3 VLSM

3.4 NAT

4.4 관련 protocol

VLSM (Variable Length subnet Masks)

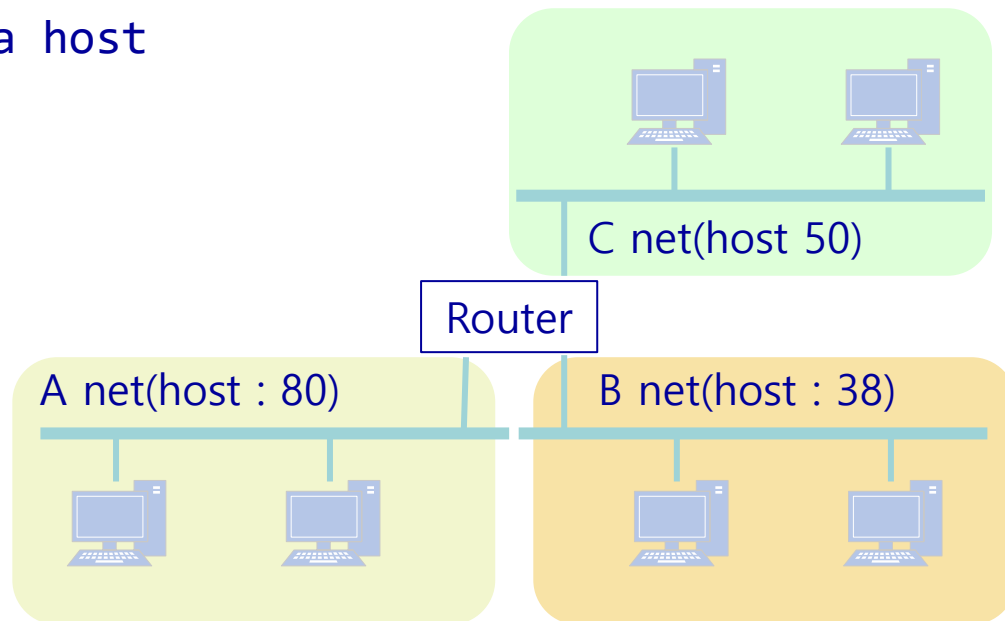
- VLSM은 네트워크를 나눌 때 동일한 규모가 아니라 서로 다른 크기의 네트워크로 분할 하는 것을 의미한다.
- IP낭비를 일부 막을 수 있으나 IP 낭비로 인한 문제를 해결해 주지 못한다.
- Class oriented protocol 에는 사용 불가능하다.
 - RIP, IGRP

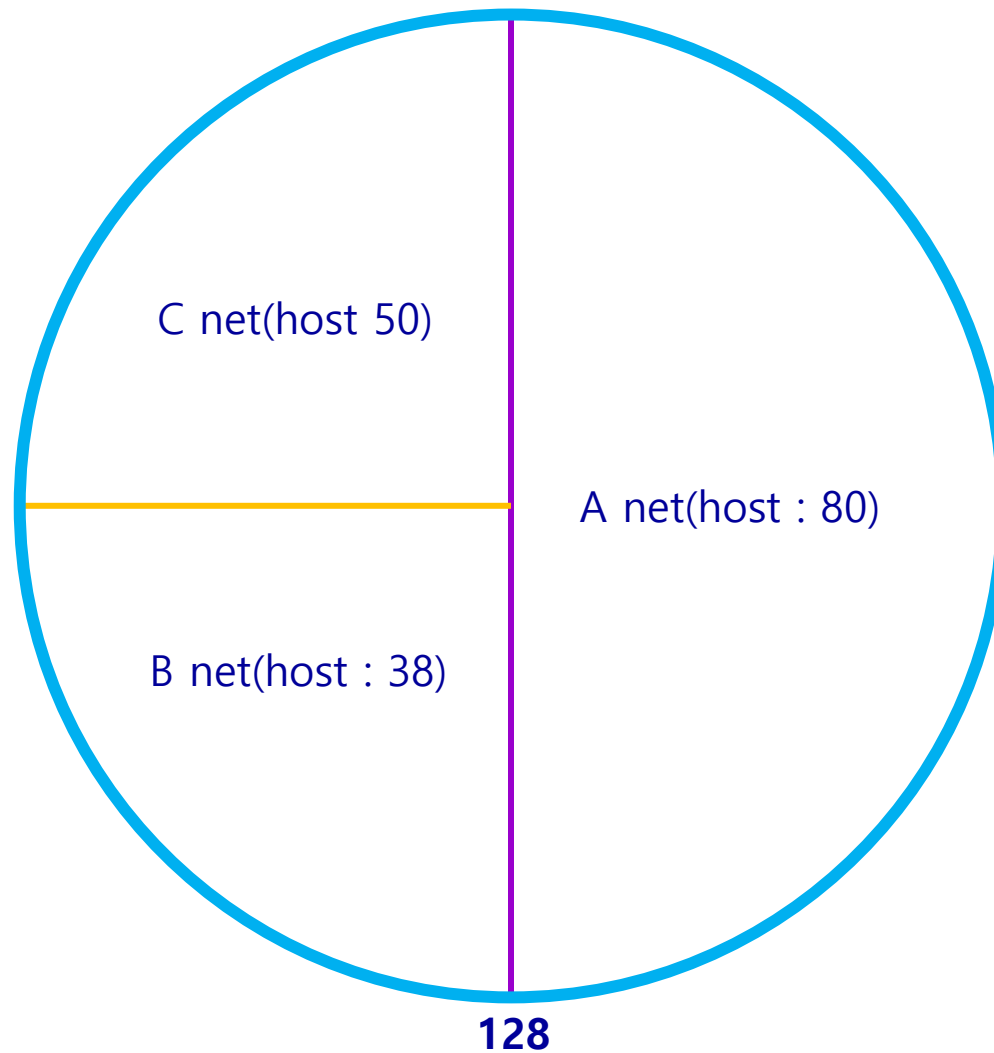
- 1.1.1.0/24 네트워크를 다음 조건에 맞는 A, B, C 세개의 네트워크로 분할 한다.

A 네트워크 : 80ea host

B 네트워크 : 38ea host

C 네트워크 : 50ea host





➤ 네트워크 분할

- A Net : 1.1.1.0/25 : 1.1.1.0 ~ 1.1.1.127
- B Net : 1.1.1.128/26 : 1.1.1.128 ~ 1.1.1.191
- C Net : 1.1.1.192/26 : 1.1.1.192 ~ 1.1.1.255

또는

- A Net : 1.1.1.128/25 : 1.1.1.128 ~ 1.1.1.255
- B Net : 1.1.1.0/26 : 1.1.1.0 ~ 1.1.1.63
- C Net : 1.1.1.64/26 : 1.1.1.64 ~ 1.1.1.127

- 각 서브네트워크는 서로 완전히 독립적이므로 구성이 어떻게 되든지 아무 상관이 없다.

서브넷에 대한 여러 문제

➤ 1.1.1.0 ~ 1.1.1.255 IP를 할당 받은 경우.

- ① 50ea 전후의 host를 갖는 네트워크 몇 개를 수용 할 수 있는가?
- ② 네트워크를 8개로 분할하면 각 네트워크의 네트워크 주소와 넷마스크는 무엇인가?
- ③ 현재 사용중인 3 개의 네트워크가 각각 29, 30, 50 ea의 host를 갖는다면 네트워크 할당은 어떻게 이뤄져야 하는가?

➤ 1.1.0.0~1.1.255.255 IP를 할당 받은 경우.

- ① 200ea, 140ea, 300ea의 host를 갖는 서브넷에 IP를 할당해보자.

3.1 Network Layer와 IP

3.2 네트워크 분할

3.3 VLSM

3.4 NAT

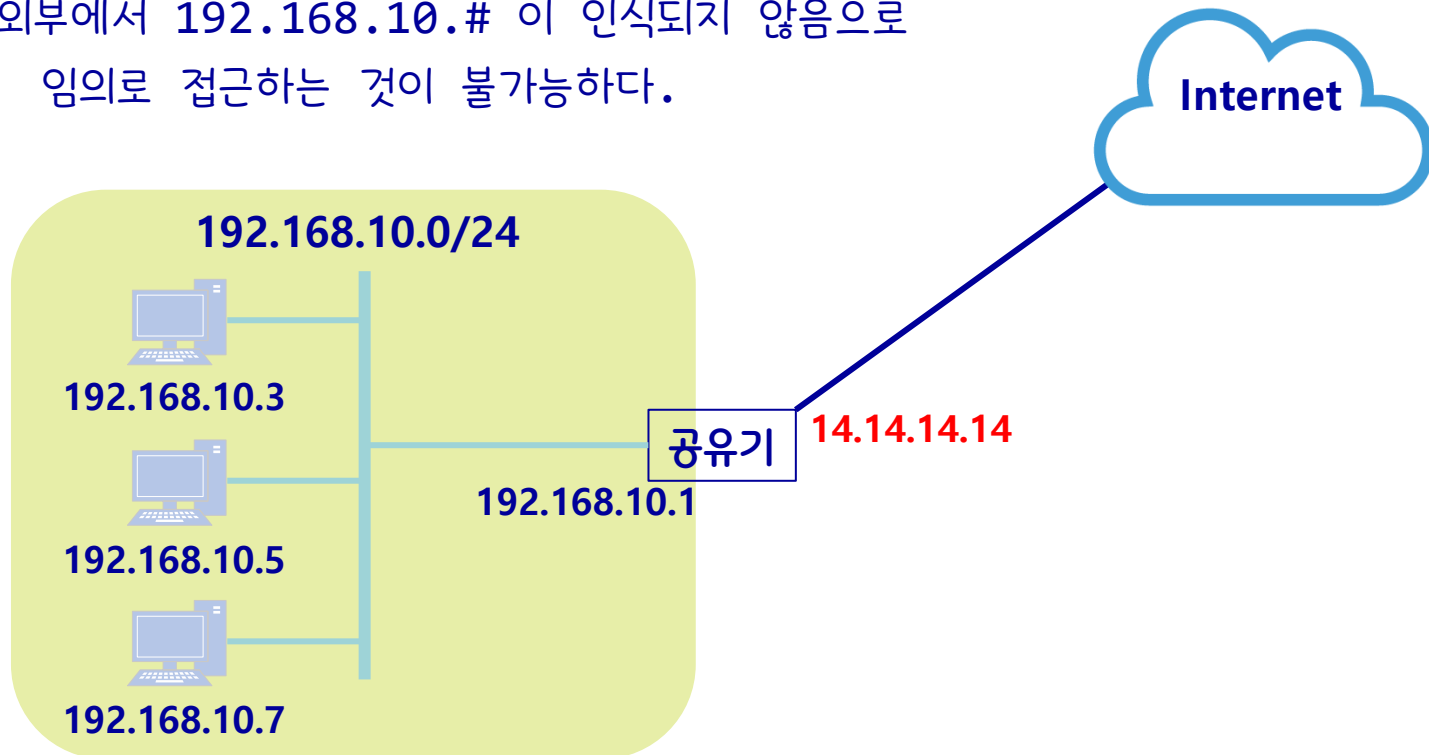
4.4 관련 protocol

NAT(Network Address Translation)

- 실재 구현이나 역할은 약간씩 다르지만 NAT, PAT, Masquerading으로 불리던 여러 기술이 NAT라고 쓰인다.
- 모두 공인 IP의 부족 현상을 해결하기 위해 사용된 기술이다.
- NAT의 특징
 - 사설 IP와 공인 IP 1:1, 1:N 모두 가능하지만 1:1 구현은 현재 쓰이지 않는다.
 - 리눅스와 같은 UNIX에서 masquerading(마스커레이딩)이란 명칭으로 firewall 프로그램에 의해 제공된다.
 - 기능을 제공하는 단독형 장비를 공유기라고 부른다.
 - NAT는 원래 IP부족을 해결하기 위한 장비이지만 부가적으로 외부 침입에 대해서 근본적인 접근을 허용하지 않음으로 현재는 방화벽의 일부로 인정된다.
 - 네트워크 성능에 영향을 끼쳤으나 현재는 이와 별 상관이 없다.

➤ 공유기를 이용한 Network 구성

- gateway : 192.168.10.1
- 192.168.10.# 을 가진 host들이 외부로 패킷을 보내면 모두 **14.14.14.14**가 보낸 것으로 인식된다.
- 외부에서 192.168.10.# 이 인식되지 않으므로 임의로 접근하는 것이 불가능하다.



인터넷 설정 정보

● 동적 IP 방식

○ PPPoE

○ 고정 IP 방식

외부 IP 주소

1236

사설 IP 할당 허용하기

서브넷 마스크

255.255.255.0

기본 게이트웨이

1236

기본 DNS 주소

210.220.163.82

보조 DNS 주소

219.250.36.130

DNS 주소 수동 입력

WAN MAC 주소

A8-5E-45-57-85-EB

MAC주소 변경

MTU

1500

수동 입력

내부 네트워크 설정

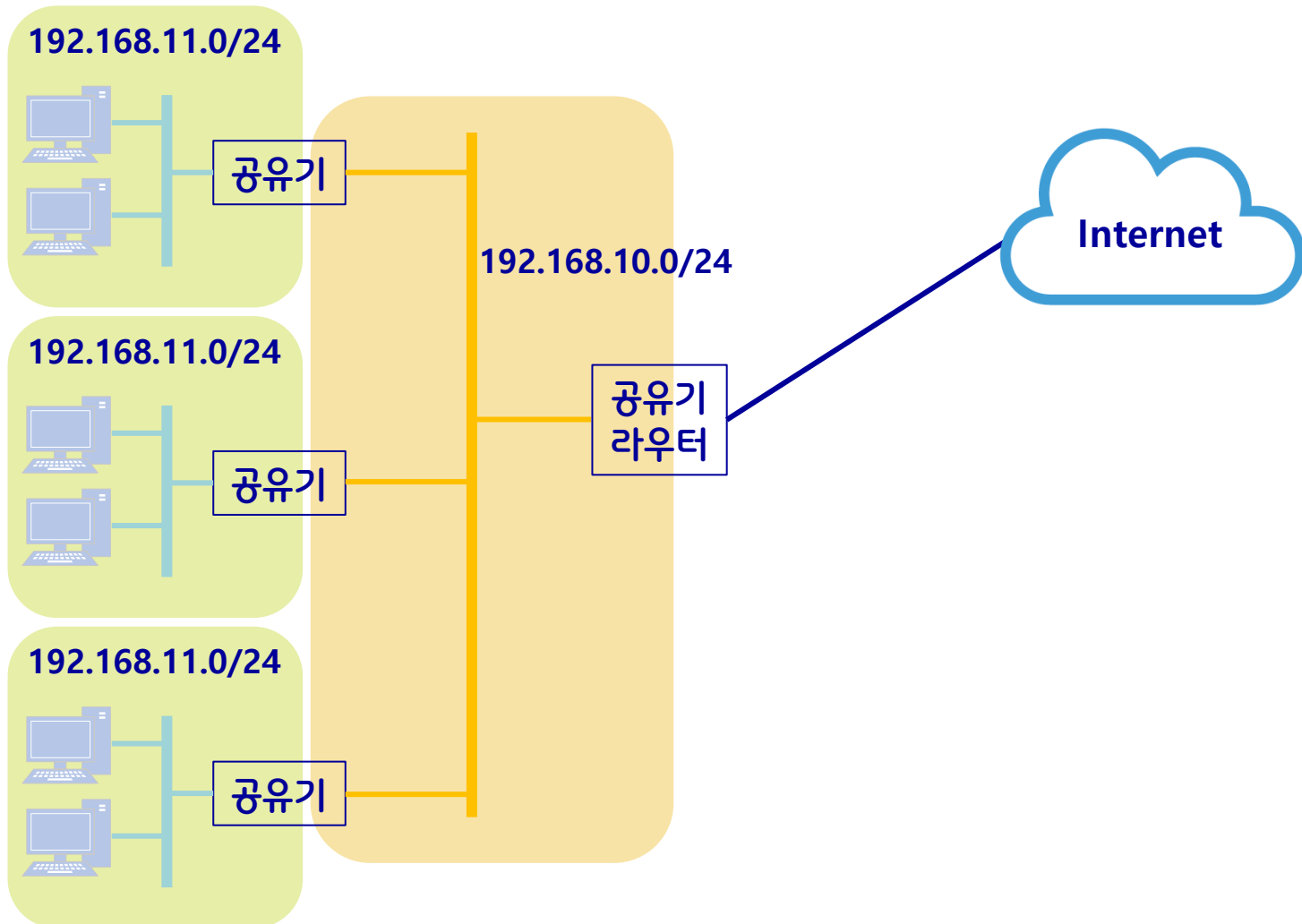
내부 IP주소

192.168.10.1

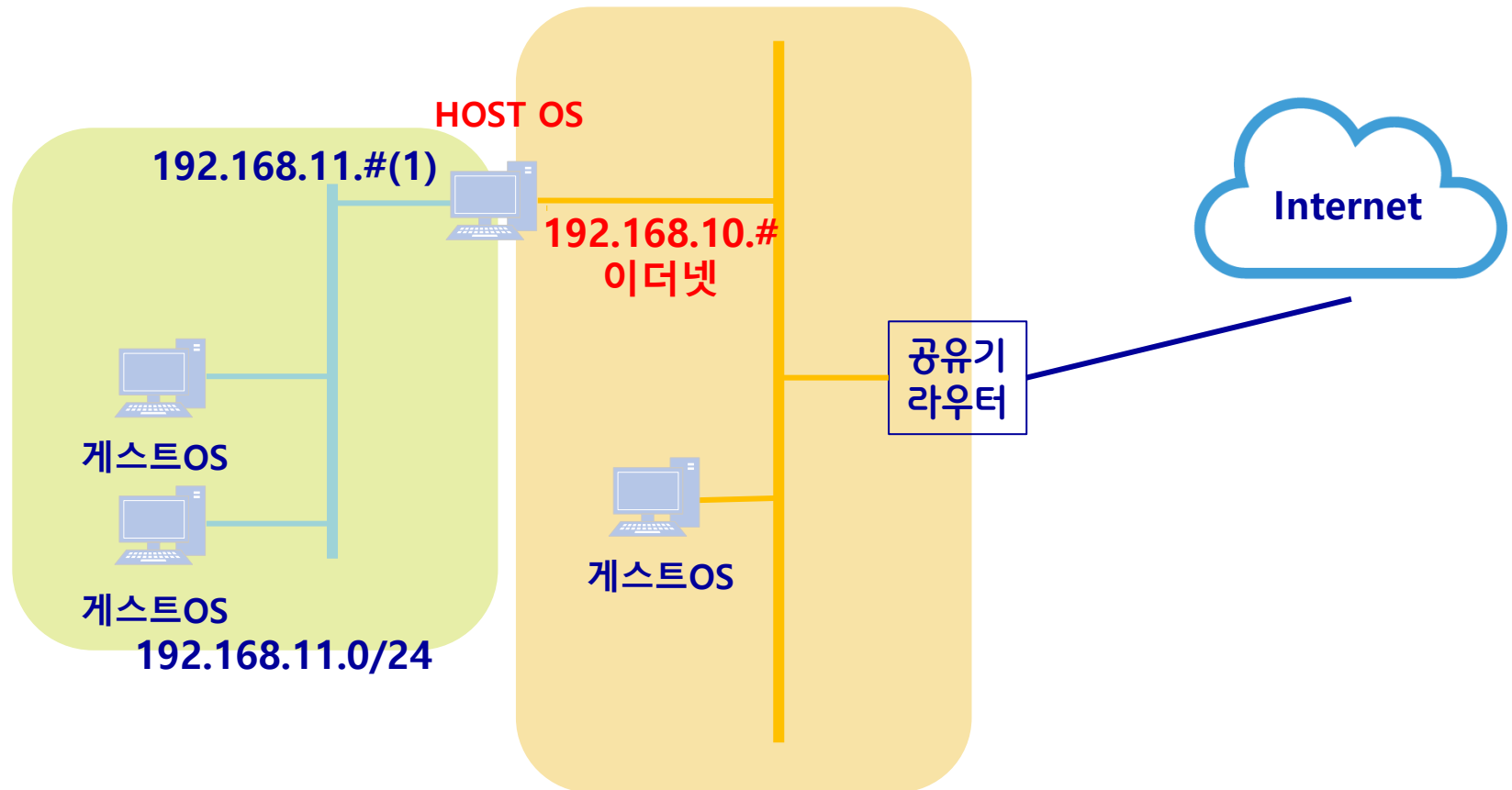
(00-26-66-FE-28-1A)

서브넷 마스크

255.255.255.0



➤ Vmware 구성



3.1 Network Layer와 IP

3.2 네트워크 분할

3.3 VLSM

3.4 NAT

4.4 관련 protocol

ICMP(Internet Control Message Protocol)

- 인터넷 제어 메시지로, 네트워크 관리 및 오류 보고를 위해 사용되며 IP의 비 신뢰성, 비 연결성의 단점을 보완하고자 사용되는 IP supported protocol이다. 특정 프로그램에 의해 호출되었을 때만 동작하며, Message 형식으로 정보를 알린다.
- 다양한 질의와 응답이 있으나 이를 통한 가용성 공격이 빈번함으로 현재는 대부분 방화벽 수준에서 DROP된다.
- 주로 시스템 및 경로 확인을 위해 사용된다.
 - ping
 - traceroute(tracert)

➤ ping

```
C:\ 명령 프롬프트

c:\>ping www.naver.com

Ping www.naver.com.nheos.com [125.209.222.141] 32바이트 데이터 사용:
요청 시간이 만료되었습니다.
요청 시간이 만료되었습니다.
요청 시간이 만료되었습니다.
요청 시간이 만료되었습니다.

125.209.222.141에 대한 Ping 통계:
    패킷: 보냄 = 4, 받음 = 0, 손실 = 4 (100% 손실),

c:\>ping www.google.com

Ping www.google.com [216.58.220.196] 32바이트 데이터 사용:
216.58.220.196의 응답: 바이트=32 시간=63ms TTL=115
216.58.220.196의 응답: 바이트=32 시간=63ms TTL=115
216.58.220.196의 응답: 바이트=32 시간=62ms TTL=115
216.58.220.196의 응답: 바이트=32 시간=63ms TTL=115

216.58.220.196에 대한 Ping 통계:
    패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실),
왕복 시간(밀리초):
    최소 = 62ms, 최대 = 63ms, 평균 = 62ms

c:\>
```

➤ tracert

```
CA. 명령 프롬프트
c:\>tracert 8.8.8.8

최대 30홉 이상의
dns.google [8.8.8.8] (으)로 가는 경로 추적:

  1    <1 ms    <1 ms    <1 ms    gw [192.168.10.1]
  2    25 ms    4 ms     6 ms    1.236.43.1
  3     3 ms    2 ms     3 ms    100.74.122.81
  4     2 ms    1 ms     1 ms    100.74.129.157
  5     2 ms    1 ms     1 ms    10.65.253.14
  6     5 ms    5 ms     6 ms    10.222.15.218
  7     *      3 ms     *      10.222.6.63
  8    33 ms    33 ms    34 ms    72.14.216.81
  9    36 ms    34 ms    35 ms    108.170.242.129
 10    34 ms    34 ms    35 ms    108.170.236.5
 11    35 ms    35 ms    36 ms    dns.google [8.8.8.8]

추적을 완료했습니다.
c:\>
```


- ARP는 시스템이 알고 있는 통신 상대방의 IP 주소를 이용해서 MAC 주소를 알아내는 프로토콜이다. 논리적인 IP 주소를 물리적인 MAC 주소로 변환해주는 역할을 수행하는 프로토콜, 호스트는 ARP 요청 메시지를 보낼 때 **자신의 IP 주소, 자신의 MAC주소, 그리고 수신측 IP 주소**는 알고 있지만 **수신측 MAC주소를** 모르고 있기 때문에 물리 계층 브로드캐스트를 통해 모든 호스트에게 수신측의 MAC 주소를 요청하는 패킷을 전송한다. **ARP 요청** 메시지를 수신한 호스트는 수신 IP 주소와 자신의 IP 주소를 검사하여 자신에 대해 MAC 주소를 요구하는 경우라면 **ARP 응답** 메시지 전송한다.

➤ RARP

RARP는 주로 diskless 시스템에서 자신의 IP를 알기 위해 자신의 MAC 주소를 브로드캐스트 로 전송해서 네트워크상에 존재하지만 알지 못하는 RARP 서버에게 자신이 사용할 IP를 질의하는 프로토콜이다.

RARP는 DHCP 서비스에 의해서 사용되는 사례를 찾기 어려우며 ARP 와 RARP 패킷의 구조는 동일하다.

ARP(Address Resolution Protocol)

➤ ARP의 특징

- IP에 의해 자동으로 호출된다.
- IP와 Ethernet뿐 아니라 다양한 링크계층과 네트워크 계층 프로토콜에서 사용된다.
- 보안 기능이 없기 때문에 스푸핑(spoofing) 공격에 취약하다.

➤ ARP 패킷의 종류

- 질의(Request) : broadcast
- 응답(Reply) : unicast

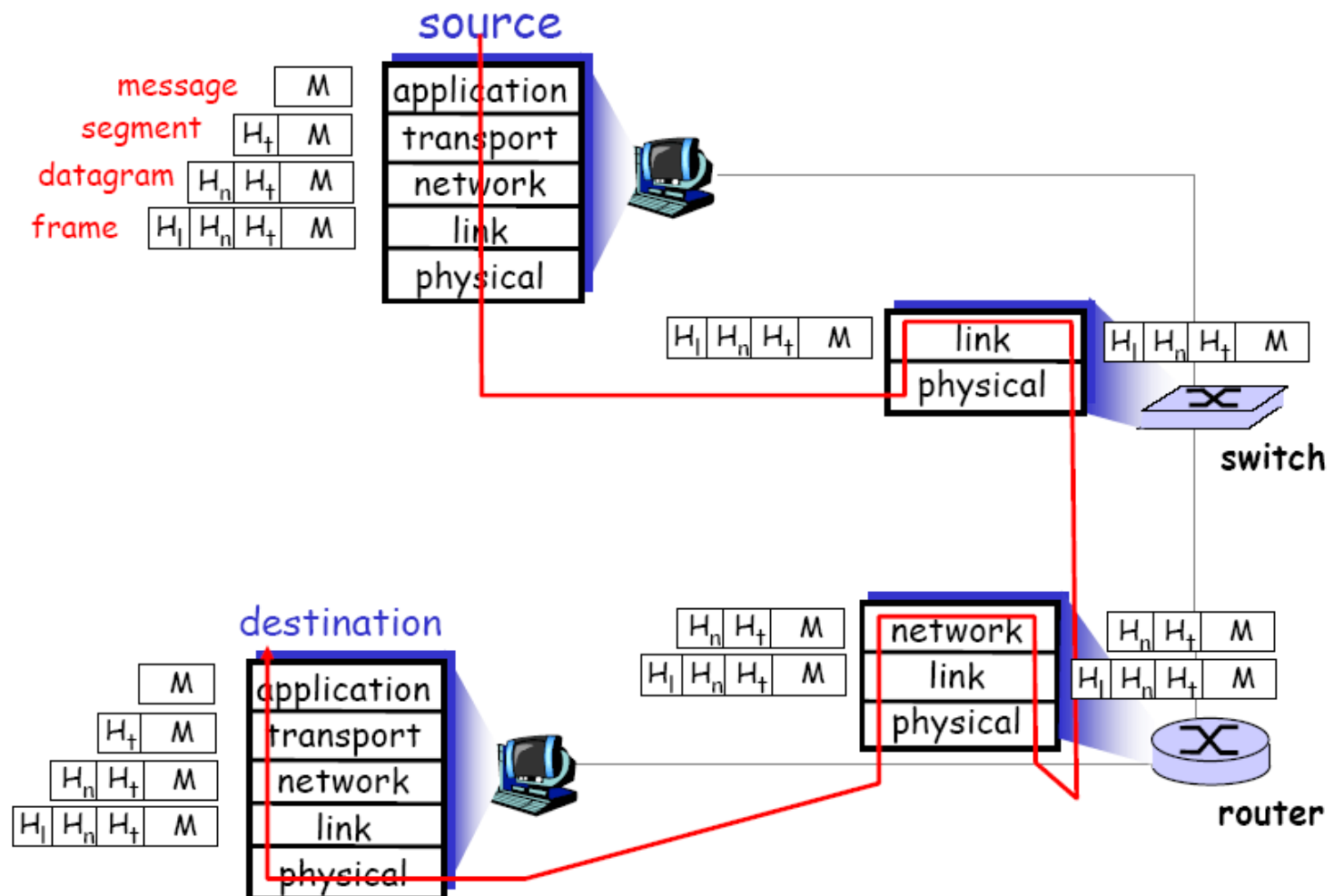
```

C:\> 명령 프롬프트

C:\Users\birochya>arp -a

인터페이스: 192.168.10.3 --- 0x3
  인터넷 주소      물리적 주소      유형
192.168.10.1      90-9f-33-18-93-8f  동적
192.168.10.11     70-4d-7b-b8-69-e3  동적
192.168.10.255    ff-ff-ff-ff-ff-ff  동적
224.0.0.2         01-00-5e-00-00-02  정적
224.0.0.22        01-00-5e-00-00-16  정적
224.0.0.251       01-00-5e-00-00-fb  정적
224.0.0.252       01-00-5e-00-00-fc  정적
239.255.255.250   01-00-5e-7f-ff-fa  정적
  
```

- 아래 그림을 이용해서 ARP에 대해서 설명한다.



4장 TCP/UDP(Transport Layer)

이장의 목적

- Transport layer의 배경과 원리 이해
 - multiplexing/demultiplexing
 - 신뢰적인 data transfer
 - flow control
 - congestion control

- ◆ 인터넷 transport layer 이해
 - UDP : connectionless transport
 - TCP : connection oriented transport
 - TCP congestion control

4.1 Transport Layer 서비스 개요

4.2 다중화와 역다중화

4.3 Connectionless transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

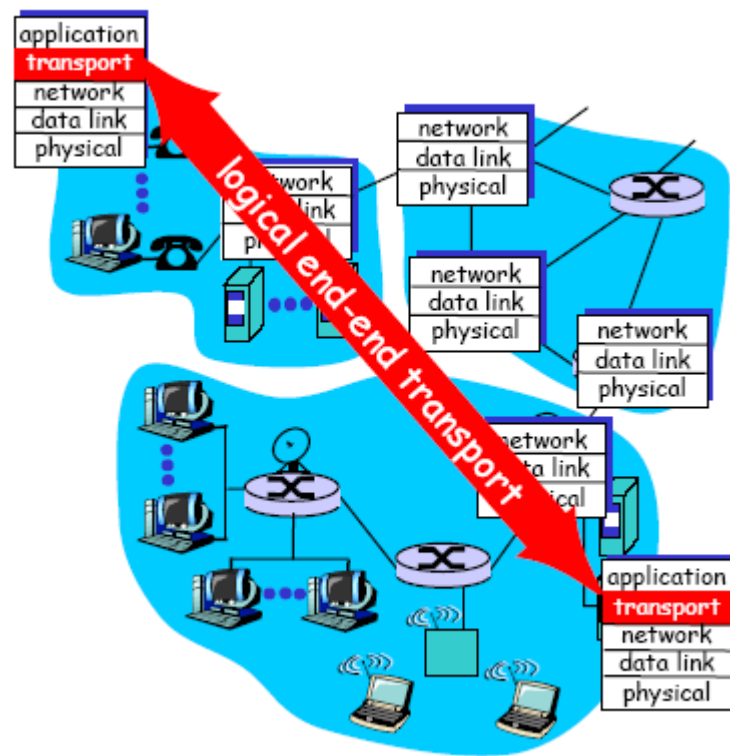
- segment structure
- reliable data transfer
- flow control
- connection control

4.6 혼잡제어의 원리

4.7 TCP의 혼잡제어

Transport services and protocol

- 서로 다른 host간에 작동하는 app process간에 논리적인 통신 (logical communication)을 제공한다.
- transport protocol은 end system에서 작동한다.
 - 송신측 : app message를 segment로 변환 network layer에 전달한다.
 - 수신측 : network layer로 부터 전달받은 segment로 부터 message를 추출 app layer에 전달한다.
- 네트워크 app는 하나이상의 transport protocol을 사용 할 수 있다
 - 인터넷 : TCP, UDP

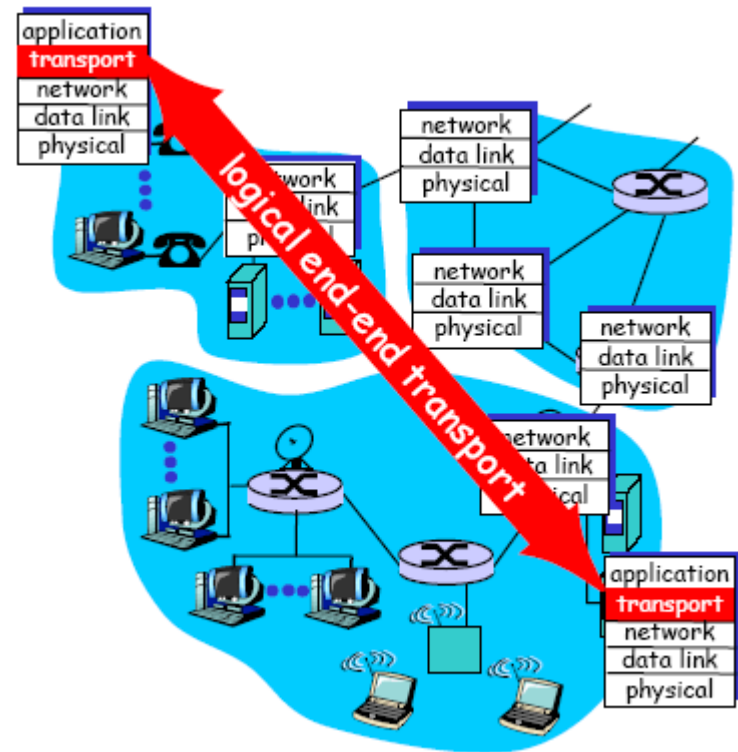


Transport vs Network layer

- network layer : host 간에 logical communication을 제공
- transport layer : process간에 logical communication을 제공
 - Network layer가 제공하지 못하는 신뢰적인 전송을 제공 할 수 있다.
 - 그러나 network layer의 제약을 받는 경우도 있다.
 - : 지연이나 대역폭에 대한 보장은 불가능하다.

인터넷에서 trans-layer의 개요

- 신뢰적인 연결지향 서비스(TCP)
 - 혼잡제어
 - 흐름제어
 - 연결지향
- 비신뢰적인 비연결지향 서비스 (UDP)
 - best-effort delivery service : IP
 - : data 전달에 최선을 다하지만 어떠한 보장도 하지 않는다.
- 보장하지 않는것
 - 최대 지연시간
 - 전달 대역폭



4.1 Transport Layer 서비스

개요

4.2 다중화와 역다중화

4.3 Connectionless

transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

- segment structure
- reliable data transfer
- flow control
- connection control

4.6 혼잡제어의 원리

4.7 TCP의 혼잡제어

Multiplexing/demultiplexing

Demultiplexing at rcv host

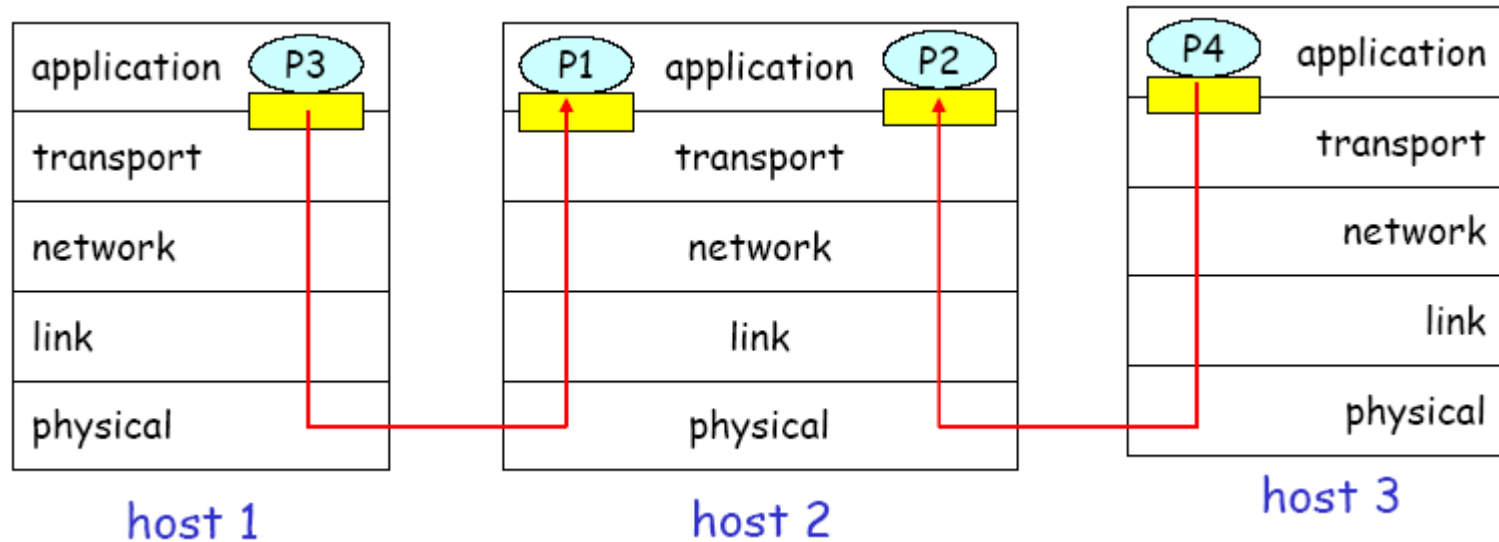
segment의 data를 올바른
socket으로 전달하는 작업

Multiplexing at send host

socket들로 부터 data를 모으고
각 data의 header정보로 캡슐화 하
고 하위 layer로 전달하는 작업

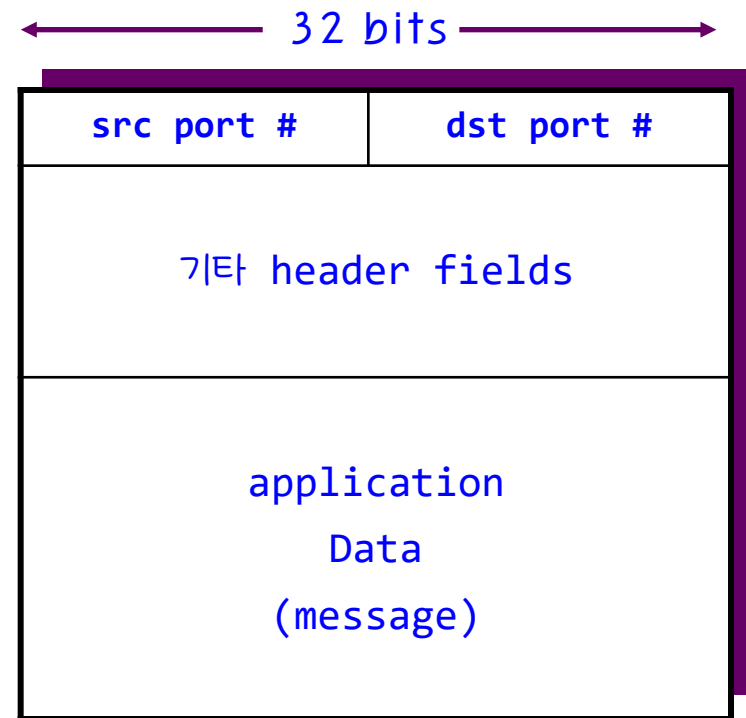
○ = process

■ = socket



Transport layer에서 demultiplexing

- Transport layer에서 demultiplexing을 위한 요구 사항
 - 각 socket은 유일한 식별자를 갖는다.
 - 각 segment는 segment가 전달될 적절한 socket을 가르키는 특별한 field를 갖는다.
 - 이들 특별한 field는 source와 destination의 port 번호이다.
(0~1023 까지의 port를 well-know port라고 한다.: RFC 1700에 명시되어있다.)
- Segment가 host에 도착하면 T/L는 segment안에 목적지의 port #를 검사하고 이에 상응하는 socket으로 data를 전달한다.



UDP/TCP segment format

Connectionless demultiplexing

- Port #를 이용 socket을 생성한다.

```
DatagramSocket mysocket1=new  
DatagramSocket(59111);
```

```
DatagramSocket mysocket1=new  
DatagramSocket(59222);
```

- UDP socket은 목적지 IP와 목적지 port #로 구성된 두 요소로 된 집합에 의해 식별된다.

- UDP segment 전달

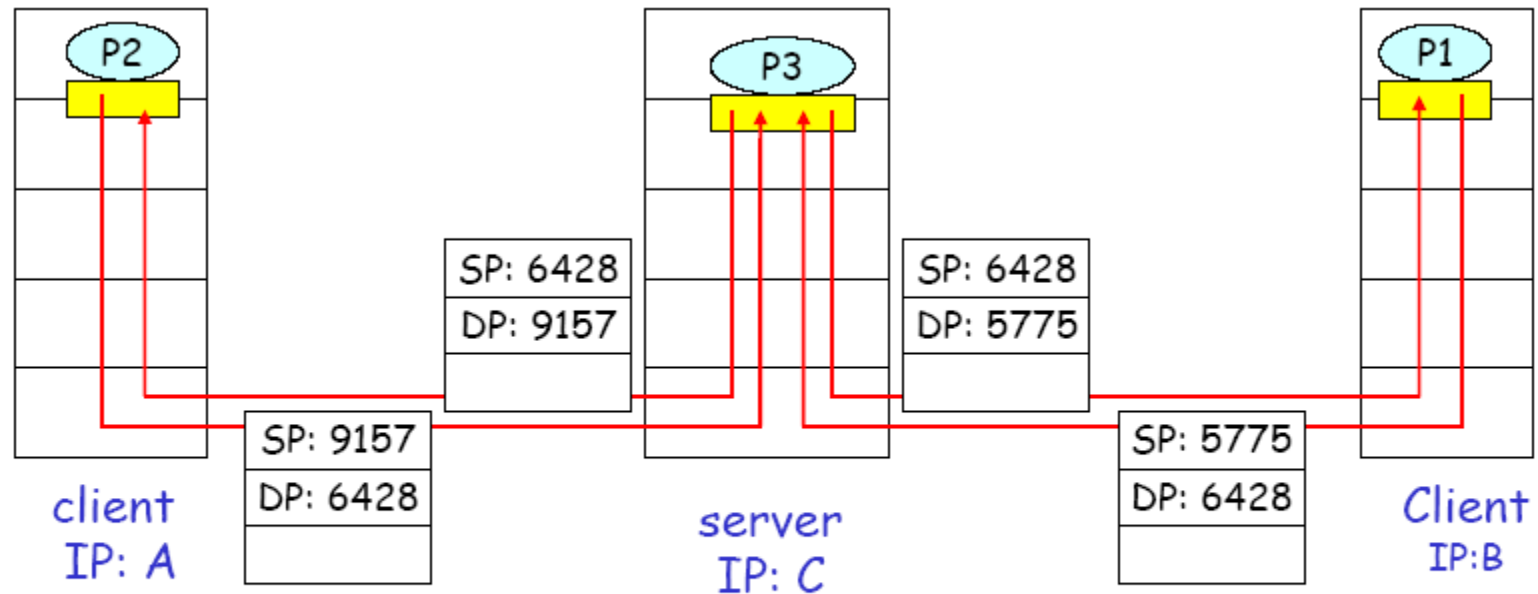
- Segment의 dest port #를 검사한다.

- Segment를 적절한 socket으로 전달한다.

- 두개의 UDP segment들이 출발지 IP 나 port # 가 둘다 다르거나 둘중 하나가 다르더라도 동일한 목적지 IP와 port #를 갖는다면 동일한 socket을 통해 process에 전달된다.

Connectionless demultiplexing

```
DatagramSocket serverSocket = new DatagramSocket(6428);
```

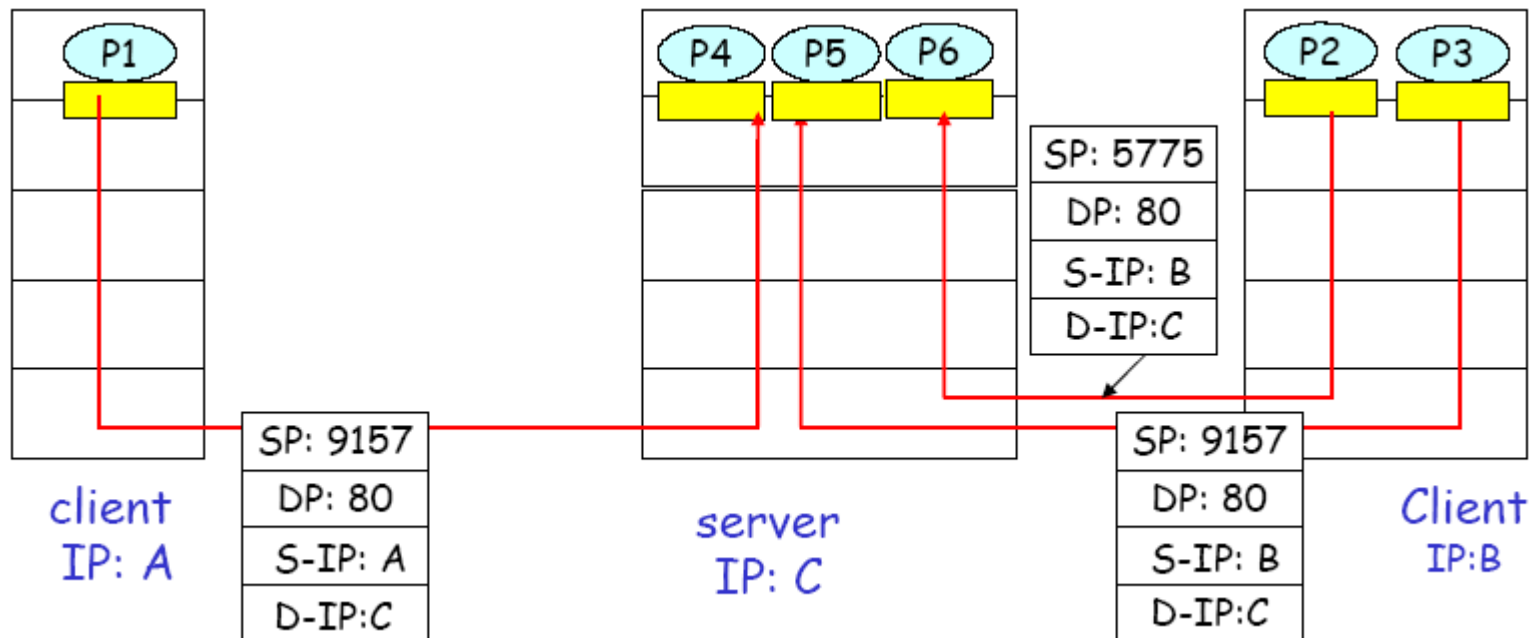


SP provides "return address"

Connection oriented demux

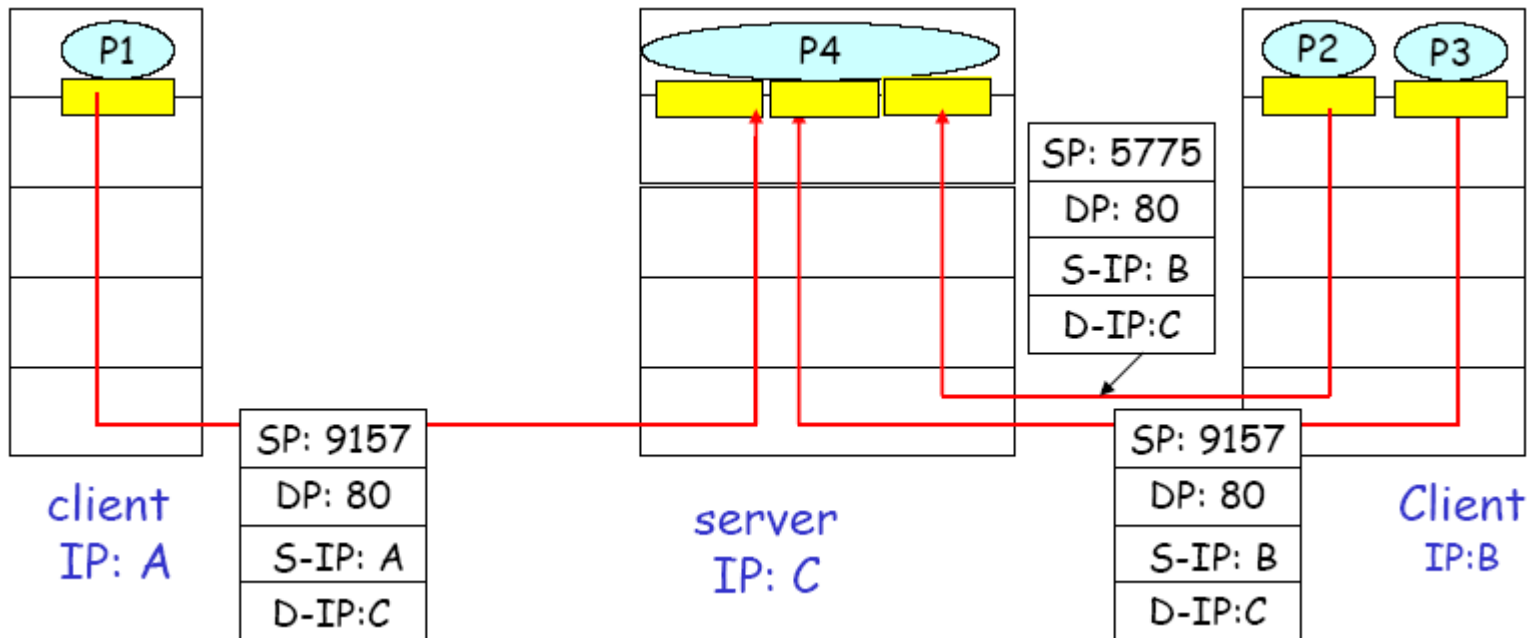
- TCP socket는 4개의 요소로 구성된 집합에 의해 식별된다.
 - source IP
 - source port #
 - dest IP
 - dest port #
- Host에 segment가 도착하면 host는 적절한 socket으로 message를 전달하기 위해 네개의 값을 모두 사용한다.
- Server host는 동시에 여러 개의 TCP socket을 지원한다.
 - 각 socket들은 앞예본 4개의 tuple에 의해 구별된다.
- Web server는 각각의 client의 접속에 대해 서로 다른 socket을이용한다.
 - non-persistent HTTP의 경우 모든 요청마다 다른 socket을 이용한다.
각각의 object마다 새로운 TCP connection을 이용한다.

Connection oriented demux



Connect oriented demux

Thread Web server



4장 Transport Layer

4.1 Transport Layer 서비스 개요

4.2 다중화와 역다중화

4.3 Connectionless transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

- segment structure
- reliable data transfer
- flow control
- connection control

4.6 혼잡제어의 원리

4.7 TCP의 혼잡제어

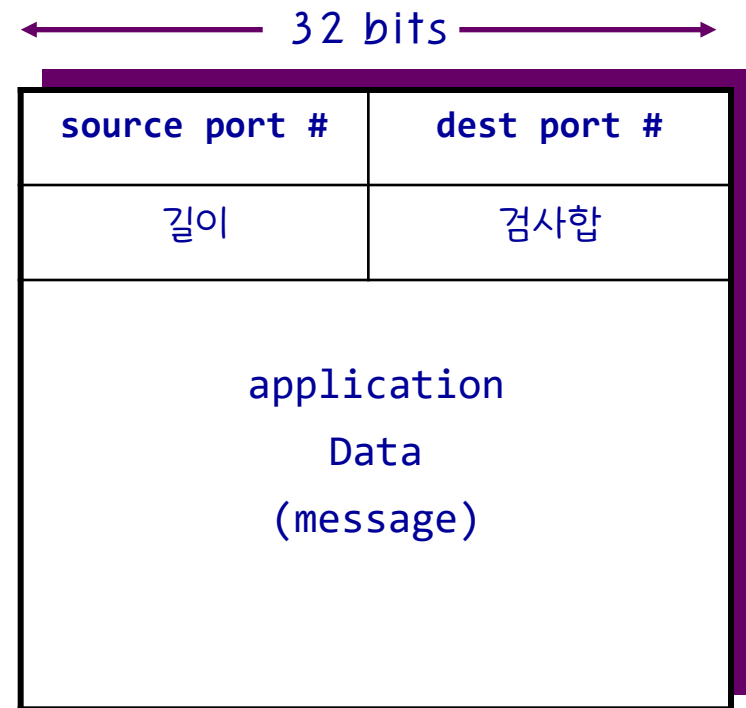
UDP : User Datagram Protocol [RFC 768]

- UDP는 Transport layer에서 할 수 있는 최소한의 기능으로 동작한다.
 - App가 거의 IP와 직접 통신하는 것이다.
 - mulx/demulx 기능만을 제공
- "best effort" (보장하지 않는다.)
 - 손실가능
 - 전송순서가 바뀔 가능
- **connectionless**
 - UDP sender와 receiver간에 handshaking하지 않는다.
 - 각 UDP segment들은 서로 독립적으로 전달된다.

UDP를 이용하는 이유

- ◆ 연결설정이 없다.
 - 예비동작 없이 전송함으로 설정에 따른 어떠한 지연도 없다
(DNS)
- ◆ 연결상태가 없다.
 - 연결상태 유지를 위한 변수가 없음으로 TCP보다 많은 app를 수용할 수 있다.
- ◆ header가 작다.
 - 8 byte (TCP : 20 byte)
- ◆ 혼잡제어하지 않는다.
 - 네트워크의 혼잡도를 고려하지 않음으로 app가 요구하는 전송을 전송량을 제한없이 전송한다.
(SNMP)

- often used for streaming multimedia apps
 - 손실 허용
 - Rate sensitive
- UDP를 사용하는 app
 - DNS
 - SNMP
- app가 신뢰성을 제공한다면 UDP상에서도 신뢰성 있는 통신이 가능하다.
 - app이 신뢰성 있는 통신을 위해 확인응답이나 재전송 등의 기능을 제공해야한다.



UDP/TCP segment format

목적 : 전송된 segment에 "errors"를 검출한다.

Sender

- Segment의 content는 순서에 따라 16bit integers로 간주한다.
- 모든 16bit(word)의 합을 가지고 1의 보수를 수행한다. (이때 오버플로우는 버린다.)
- 결과는 UDP segment checksum field에 삽입한다.
- overflow bit를 wraparound로 이용하는 경우도 있음

receiver

- Segment를 수신후 checksum을 포함한 모든 16bit word를 더한다.
- 값이 모두 1이면 에러가 없지만 하나라도 0이 나오면 오류가 있다.
- 참고 (checksum을 사용하는 방법과 bit sum에서 발생하는 overflow를 처리하는 방법이 다름. 과정은 다르지만 판단 결과는 동일)

➤ Example : 두개의 16bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

4장 Transport Layer

4.1 Transport Layer 서비스 개요

4.2 다중화와 역다중화

4.3 Connectionless transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

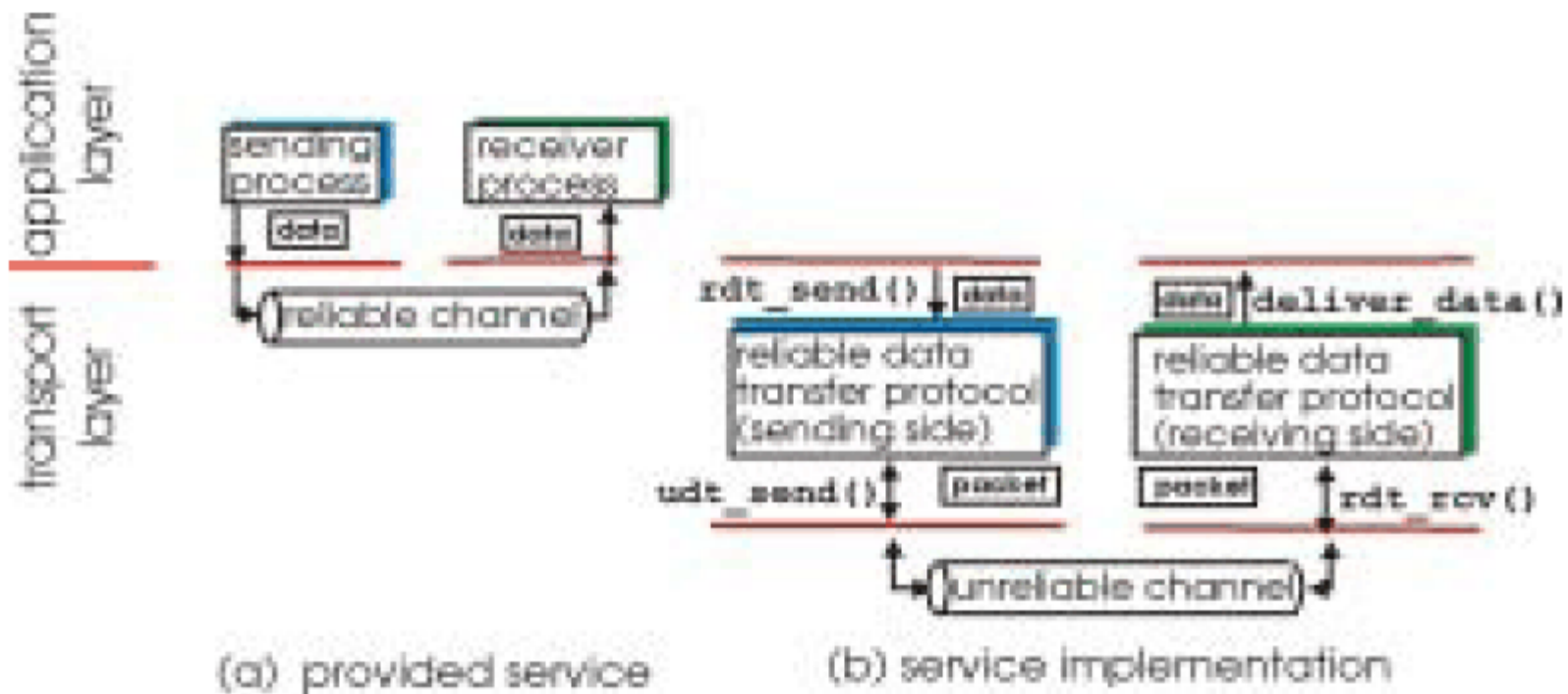
- segment structure
- reliable data transfer
- flow control
- connection control

4.6 혼잡제어의 원리

4.7 TCP의 혼잡제어

신뢰적인 data 전송의 원리

- ◆ T/L 뿐 아니라 A/L, L/L에서도 매우 중요한 문제이다.
- ◆ 매우 중요한 topics

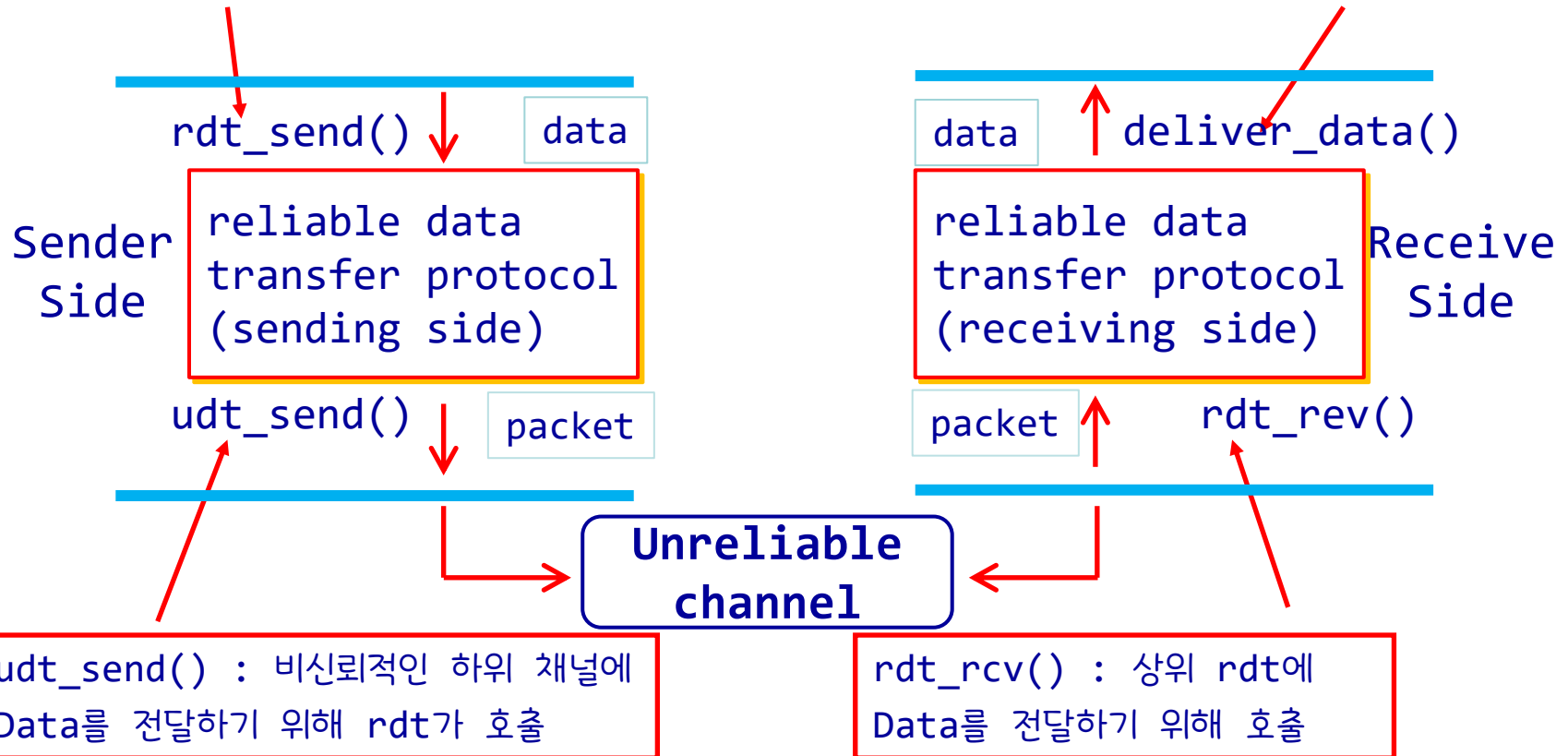


- 하위계층이 비 신뢰적인 채널인 경우 상위채널에서 신뢰적인 통신을 제공 해야 한다.

신뢰적 Data transfer

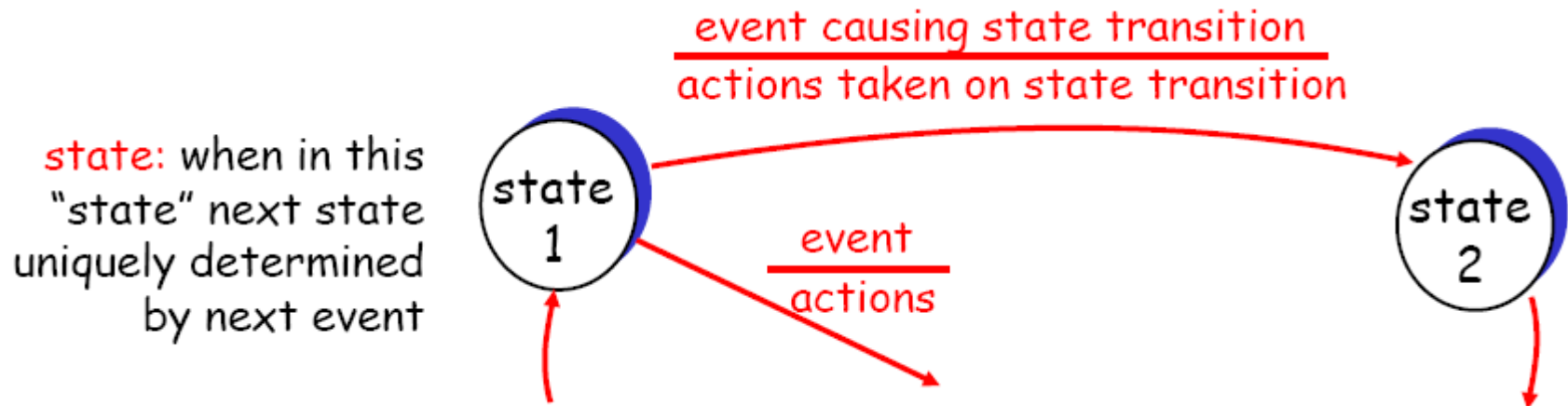
rdt_send() : 하위 rdt에 Data를 전달하기 위해 app가 호출

deliver_data() : rdt가 상위 layer에 Data를 전달하기 위해 호출



앞으로의 전개

- 점점 복잡해지는 하위채널을 고려 신뢰적 data 전송 protocol의 송신자 측면과 수신자 측면을 전개
- 단 방향 전송만을 고려 : 이 경우에도 제어 패킷은 양방향 전송이 필요
- FSM으로 표현



참고 : datagram이라는 용어보다는 일반적인 packet이라는 용어를 사용한다.

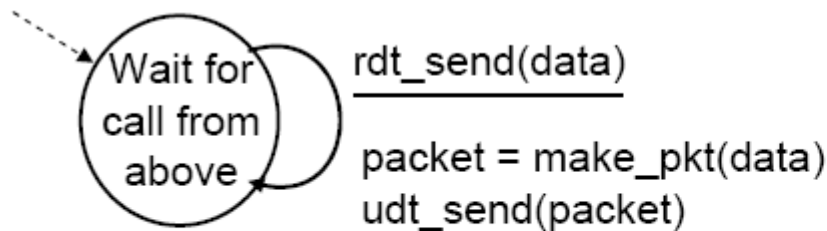
rdt 1.0 : 신뢰적인 채널에서 rdt

➤ 하위 채널이 완전히 신뢰적인 경우

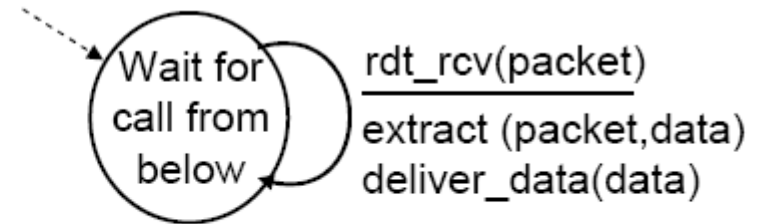
- no bit error
- no loss of packets

➤ FSM

- 송신측은 상위 layer의 data를 받아 하위 채널로 packet을 전달한다.
- 수신측은 하위 채널에서 packet을 수신하고 data를 상위 layer로 전달한다.



sender

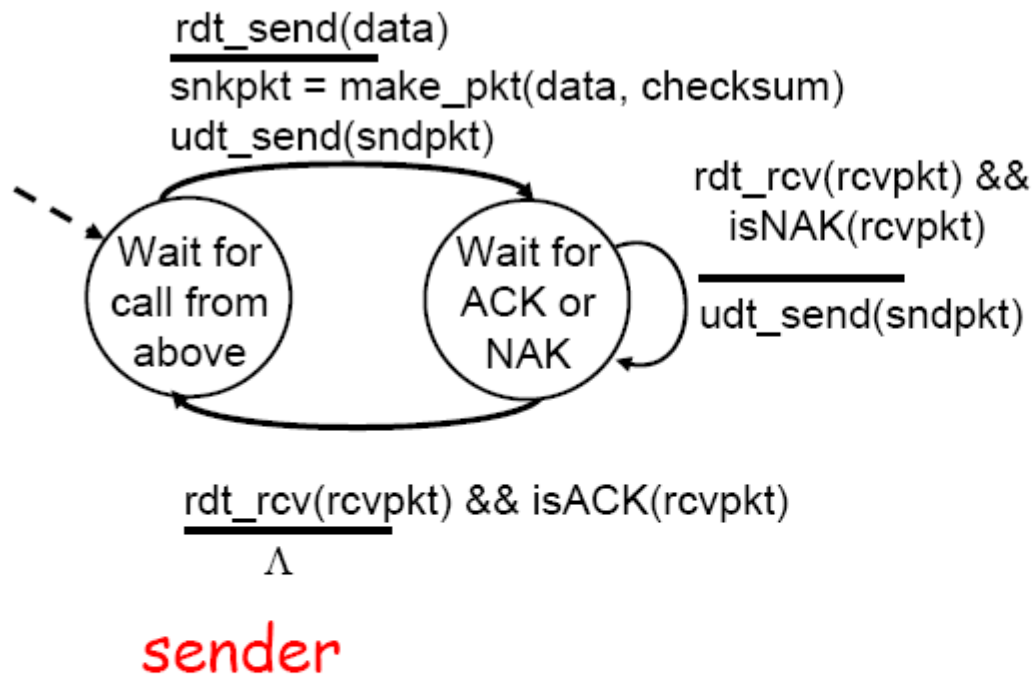


receiver

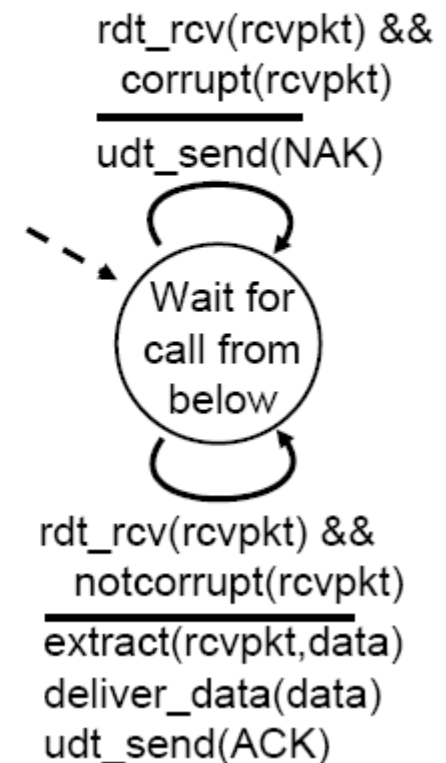
rdt 2.0 : 비트 오류가 있는 채널

- bit error이 가능한 채널에서 rdt : packet의 손실이나 순서는 보장
 - checksum을 통해 bit error감지
- 수신측의 feedback 필요
 - acknowledgement (ACKs)
 - : receiver 가 sender 에게 pkt를 잘 받았다는 응답
 - negative acknowledgement (NAKs)
 - : receiver 가 sender 에게 pkt에 error또는 장애가 있다는 응답
 - sender는 NAK인 경우 pkt 재전송
- rdt 2.0에 부가적으로 필요한 protocol (rdt 1.0에 비해서)
 - error 검출
 - receiver feedback : 제어 msg (ACK,NAK)
 - 재전송

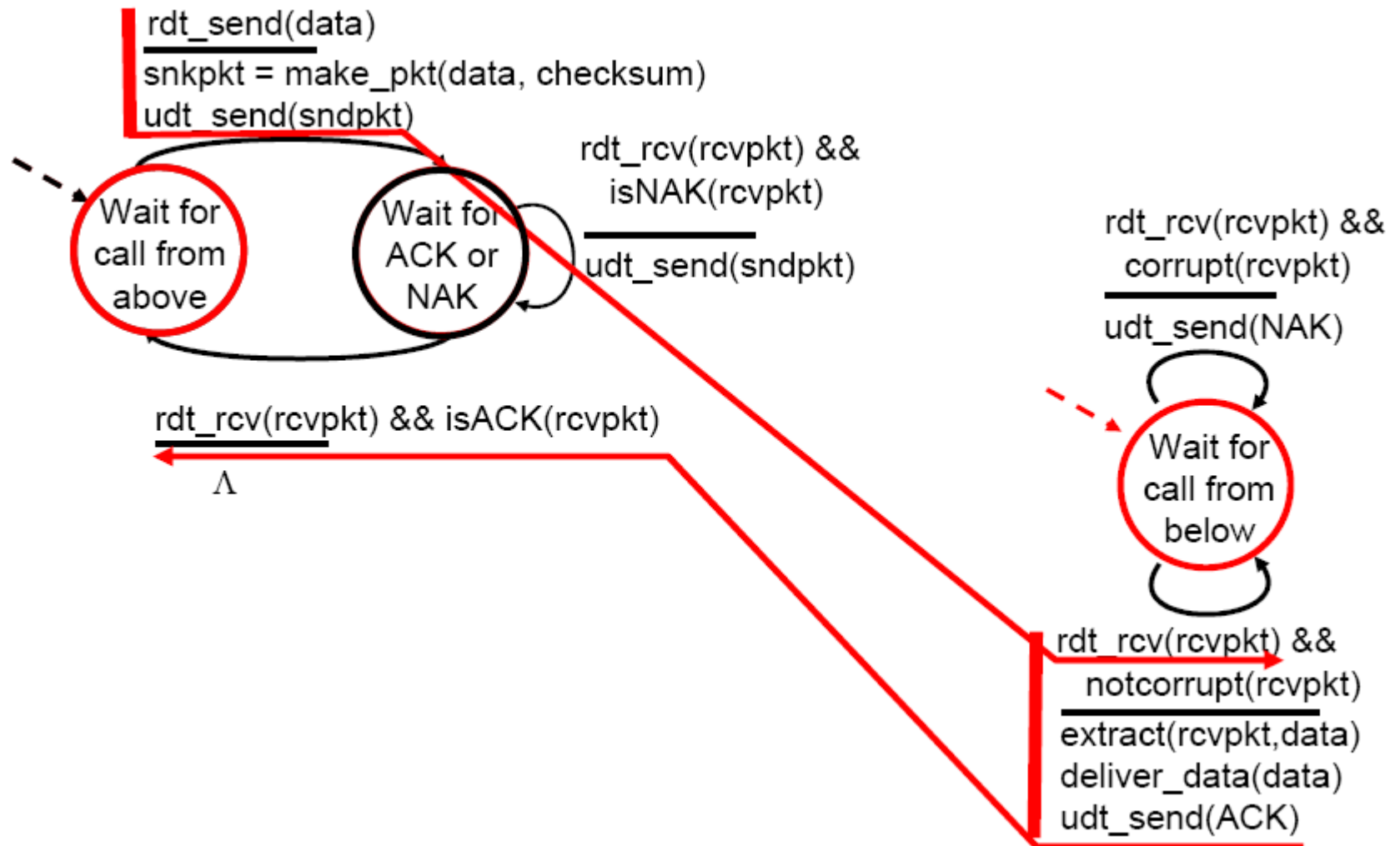
rdt 2.0 : FSM



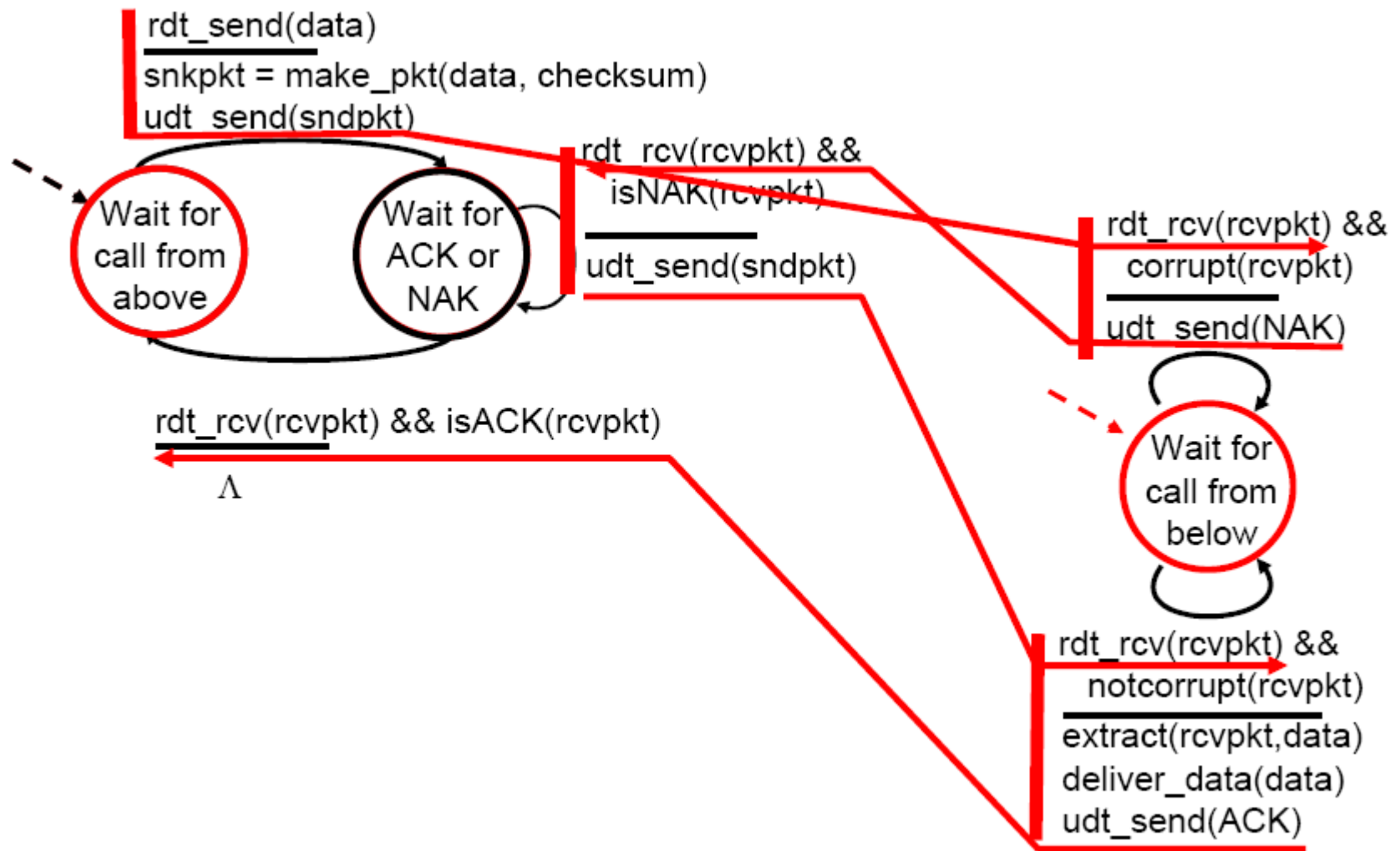
receiver



rdt 2.0 : rcvpkt OK



rdt 2.0 : rcvpkt error



rdt 2.0 : 치명적인 결함

ACK/NAK가 훼손되는 경우

- sender는 receiver쪽에 어떤일이 벌어졌는지 알 수 없다.
- 재전송으로 packet이 중복될 수 있다.

중복을 제어하는 방법

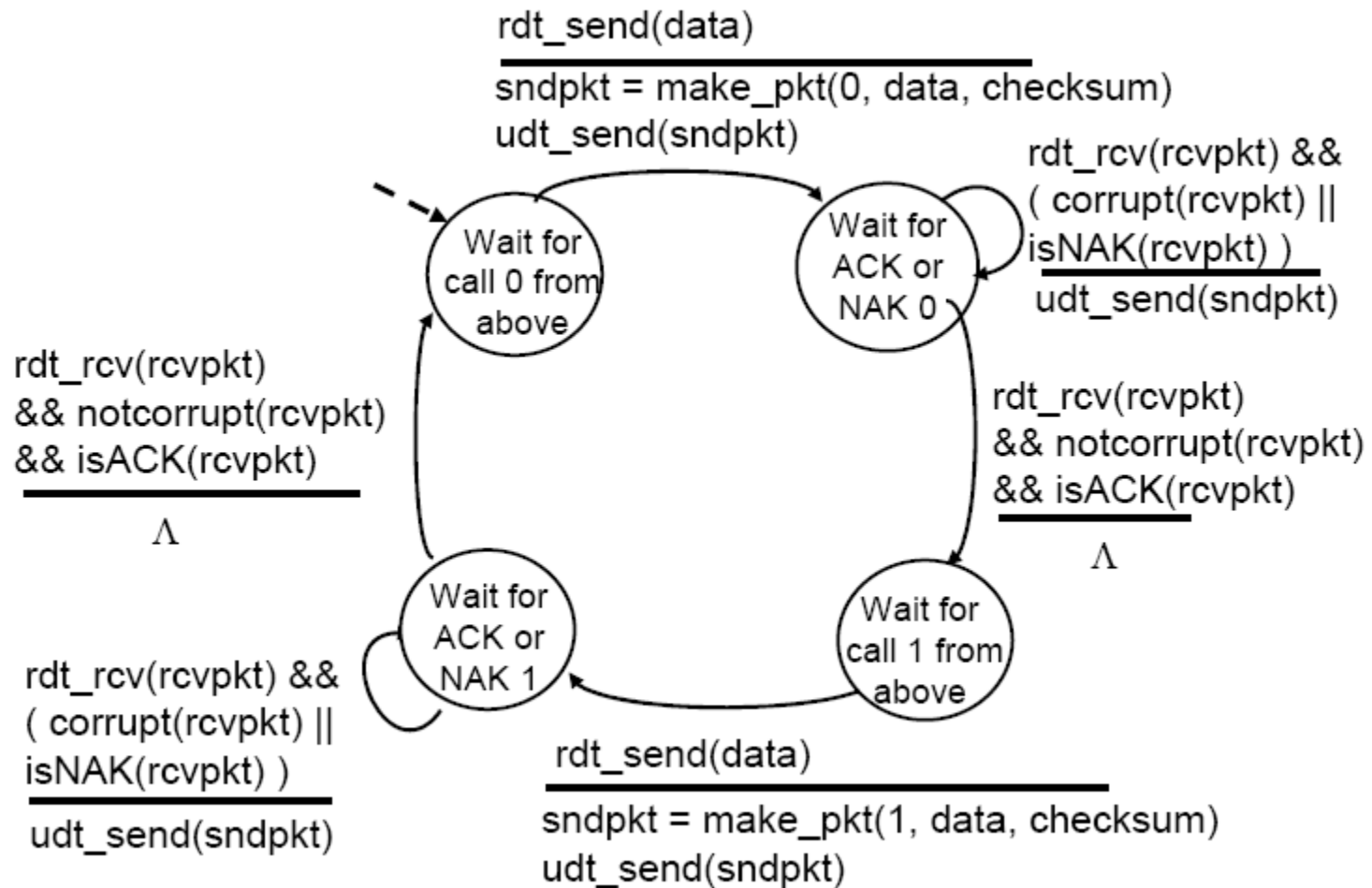
- sender는 pkt에 sequence number를 추가한다.
- ACK나 NAK가 변형되면 sender는 pkt를 재전송한다.
- receiver는 중복된 pkt을 버린다.
- rdt 2.1

stop and wait

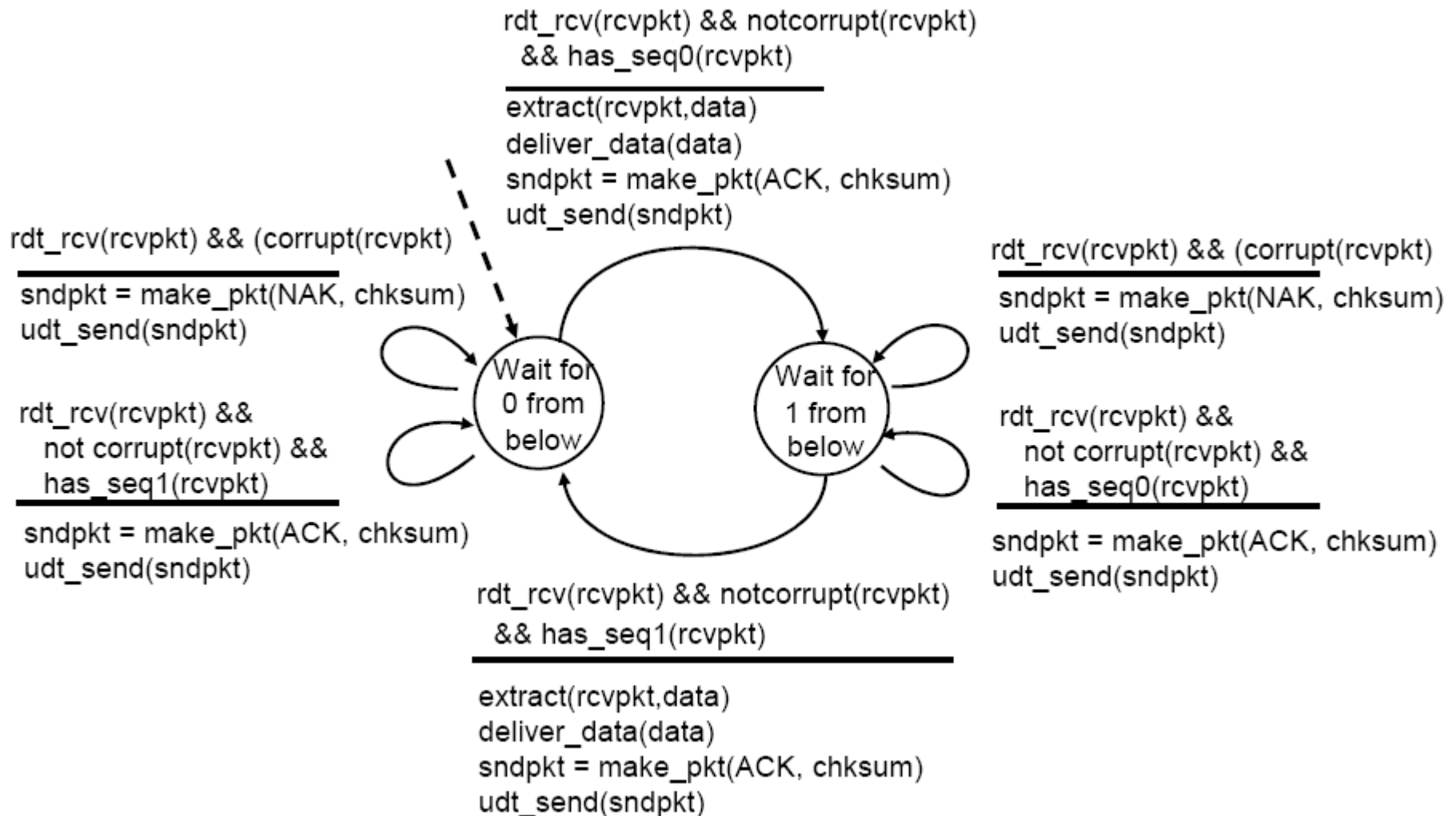
Sender는 pkt 하나를 전송한후 receiver 로 부터 response가 올때까지 대기한다.

-> response가 없으면 계속 기다린다.

rdt 2.1 : sender



rdt 2.1 : receiver



rdt 2.1

Sender

- pkt에 seq #를 추가한다.
- 두개의 seq #만으로 충분하다.
- (이전 송신된 pkt와의 중복만을 검사함으로..)
- ACK/NAK의 손상을 검사한다.
- 상태가 rdt 2.0보다 두배 많다.
 - 현재 전송 pkt의 seq #가 0인지 1인지 기억하고 있어야 한다.

receiver

- 수신된 pkt의 중복여부를 검사해야한다.
 - pkt의 seq #가 0/1인지 검사한다.
- receiver는 sender가 ACK/NAK를 정상적으로 수신했는지 알 수 없다.

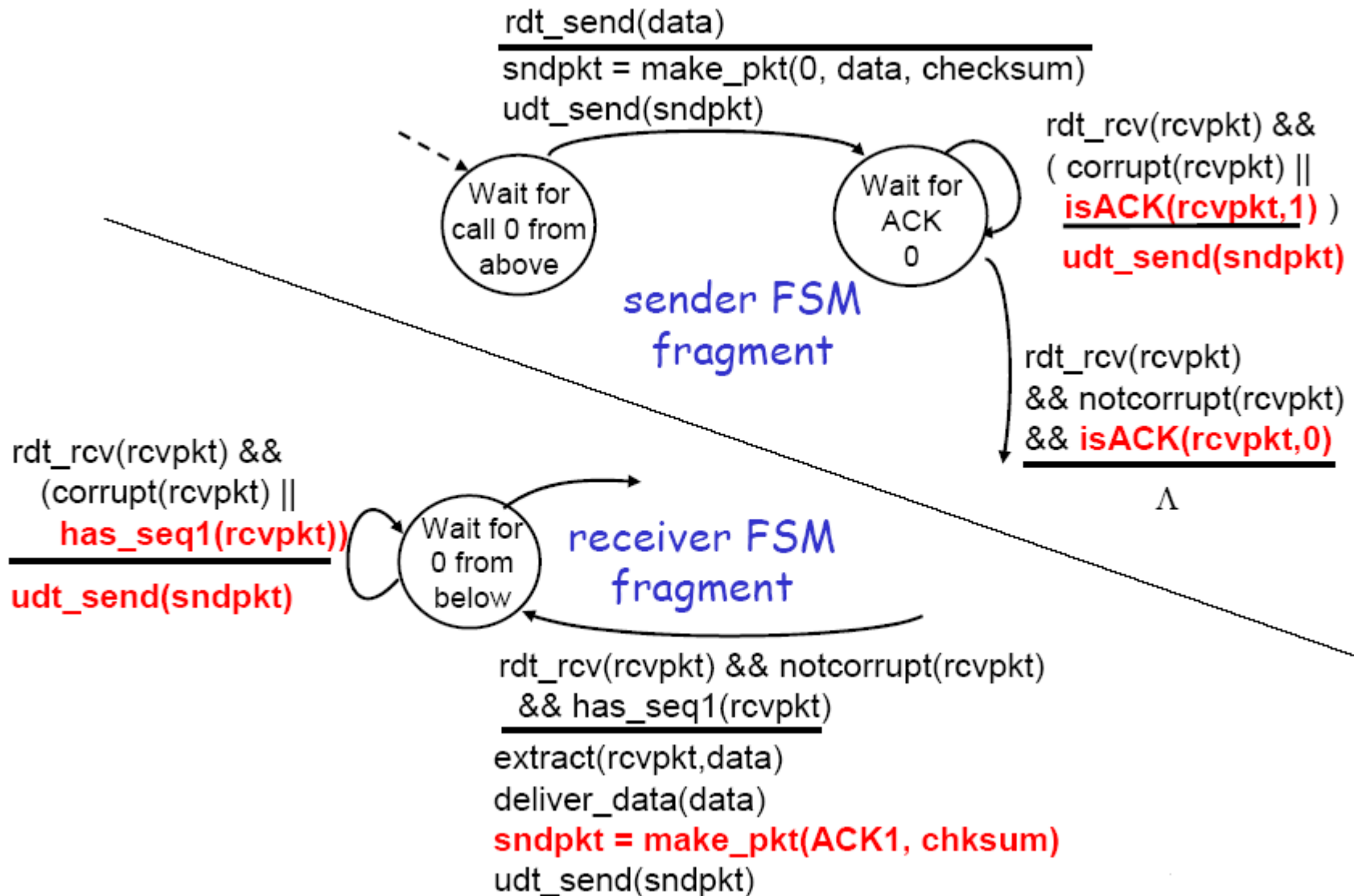
rdt 2.2 : NAK가 없는 rdt

- rdt 2.1과의 차이는 ACKs만을 사용하는 것이다.
- receiver는 NAK 대신 최근에 정확히 수신된 pkt에 대한 ACK를 전달한다.
- Sender는 중복된 ACK를 통해서 중복된 ACK pkt이후에 전송한 pkt이 정상적으로 수신되지 못했음을 안다.

ACK의 누적

누적된 ACK를 사용하는 경우 sender는 마지막 수신한 ACK의 seq # 이전의 pkt는 모두 정상 전달 되었음을 확인 할 수 있다.

rdt 2.2 : NAK가 없는 rdt



rdt 3.0 : 채널에 error나 loss가 가능

새로운 가정

하위 채널에 packet loss가 일어날 수 있는 경우 (data & ACKs)

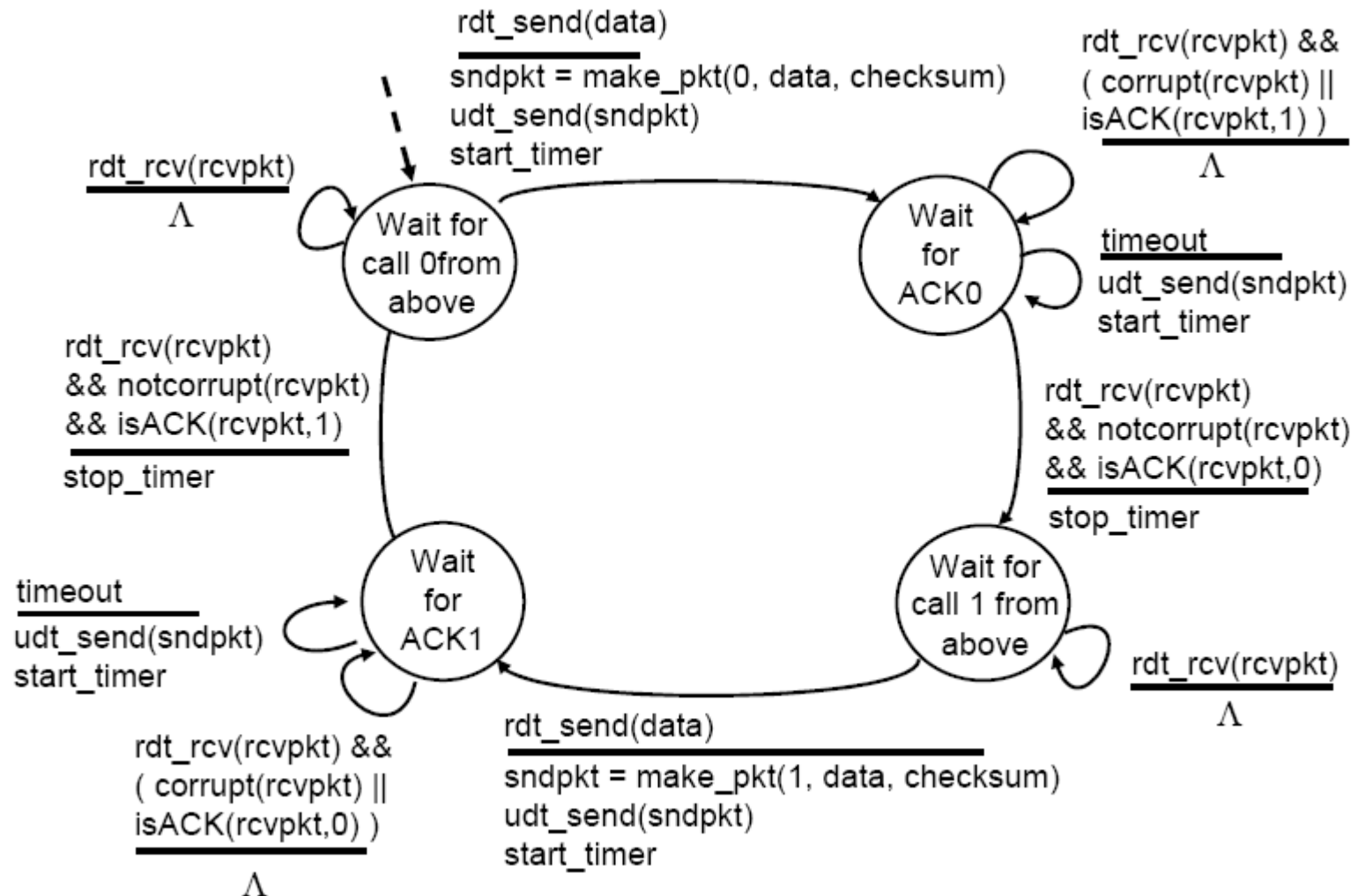
- checksum, seq #, ACKs만으로는 충분하지 않다.

새로운 접근 방식

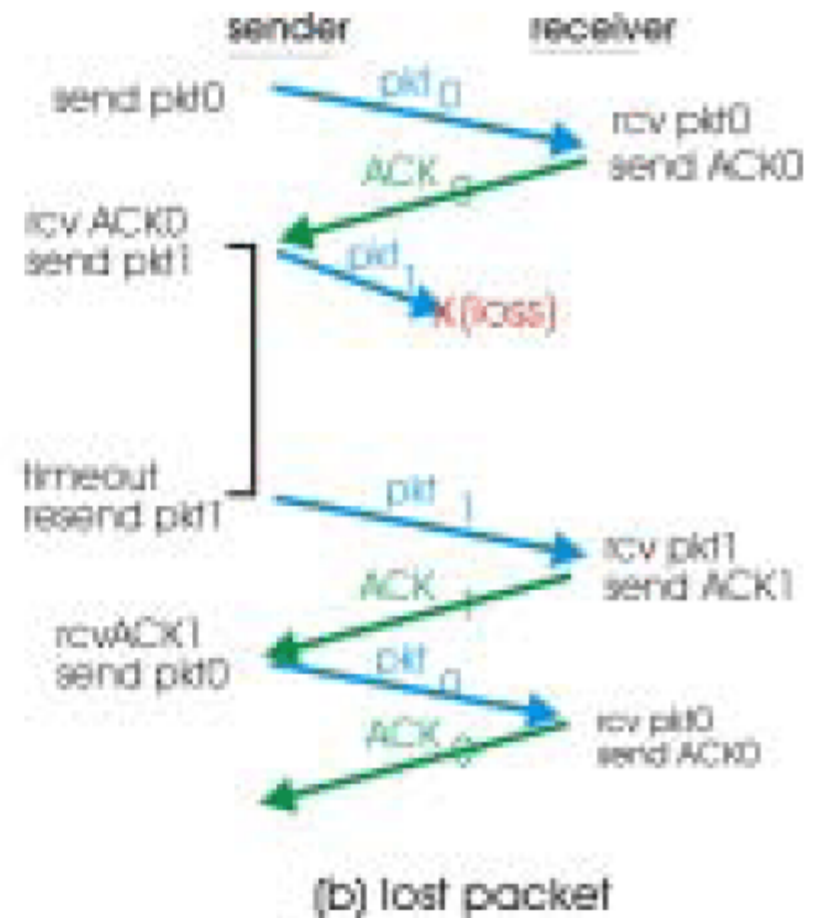
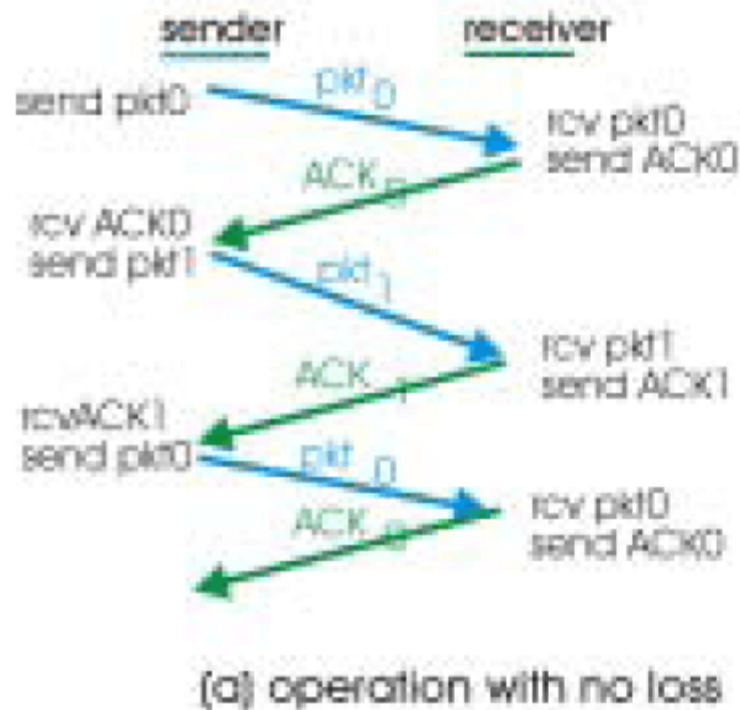
sender는 합리적인 시간 만큼 ack를 기다린다.

- 합리적인 시간만큼 기다려도 ACK가 없으면 재전송한다.
- 만일 ACK가 지연된 것이라면
 - 중복을 유발하는 재전송이 일어난다. – 이미 중복 처리에 대한 준비가 되어있다.
 - receiver는 중복된 pkt에 대한 ACK를 재전송한다.
 - 이러한 작업은 실제로 문제를 일으키지는 않는다.
- 위의 구현을 위해 **countdown timer**를 필요로 한다.

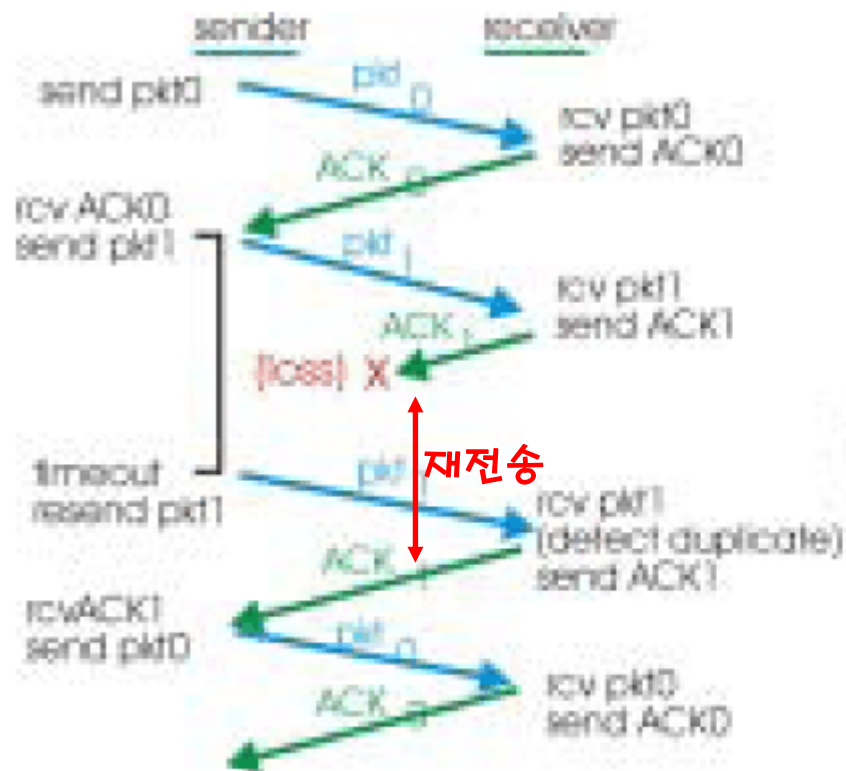
rdt 3.0 : sender



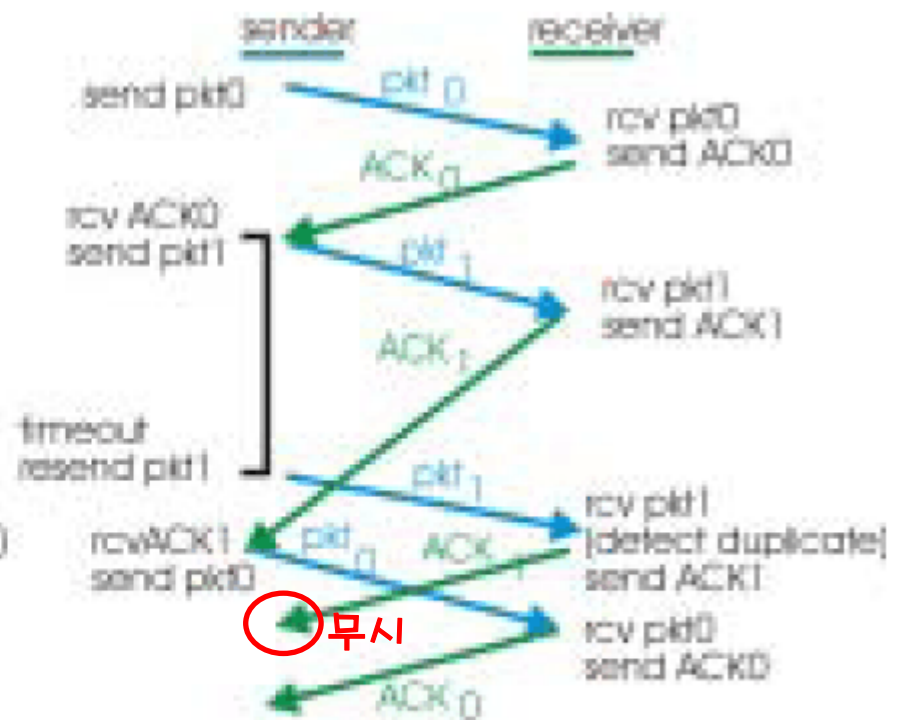
rdt 3.0 in action



rdt 3.0 in action



(c) lost ACK



(d) premature timeout

rdt 3.0 의 성능

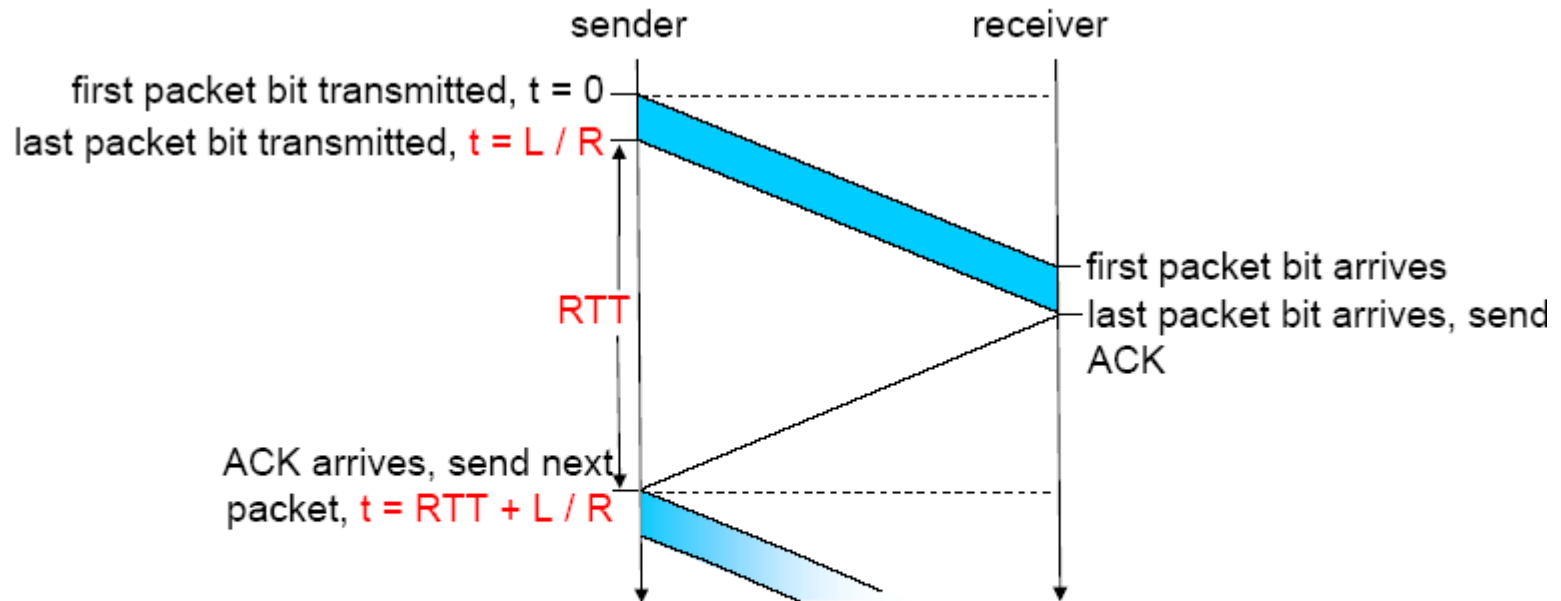
- rdt3.0 works, but performance stinks
- example: 1 Gbps link, 15 ms e-e prop. delay, 1KB packet:

$$T_{\text{transmit}} = \frac{L \text{ (packet length in bits)}}{R \text{ (transmission rate, bps)}} = \frac{8\text{kb/pkt}}{10^9 \text{ b/sec}} = 8 \text{ microsec}$$

$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

- U_{sender} : **utilization** - fraction of time sender busy sending
- 1KB pkt every 30 msec -> 33kB/sec thruput over 1 Gbps link
- network protocol limits use of physical resources!

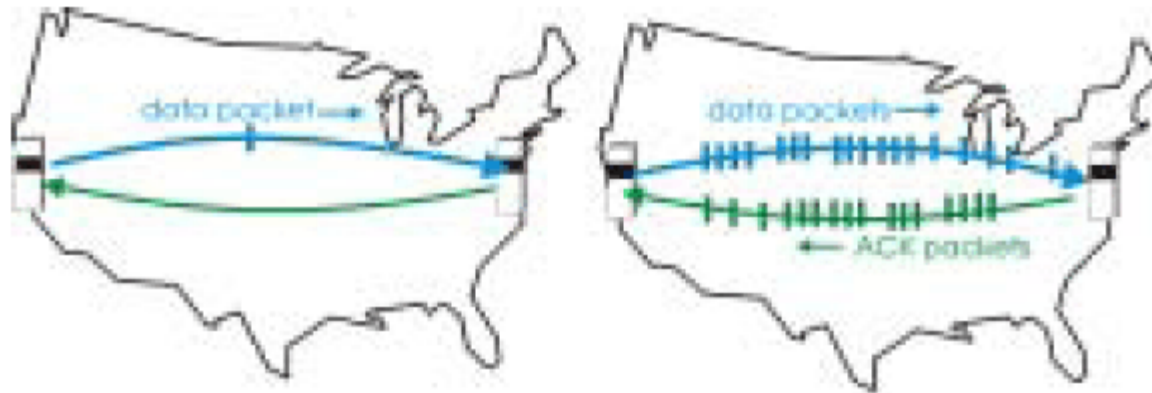
rdt 3.0 : stop – and - wait



$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

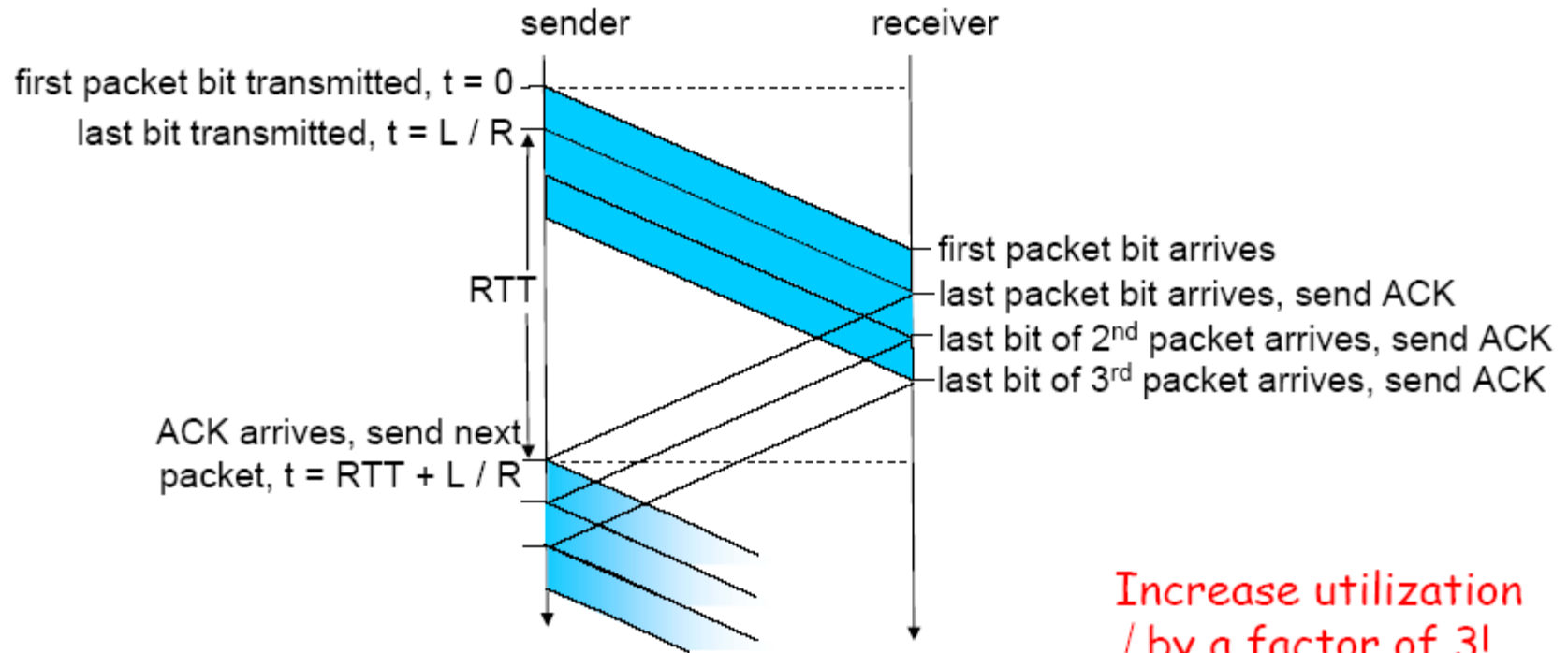
Pipelining : sender에게 ACK를 기다리지 않고 여러 개의 pkt를 전송하도록 하는 용하는 것

- seq #의 범위는 증가되어야 한다.
- sender와 receiver는 하나 이상의 pkt를 buffering해야 한다.



➤ piplining protocol : go-Back-N, selective repeat

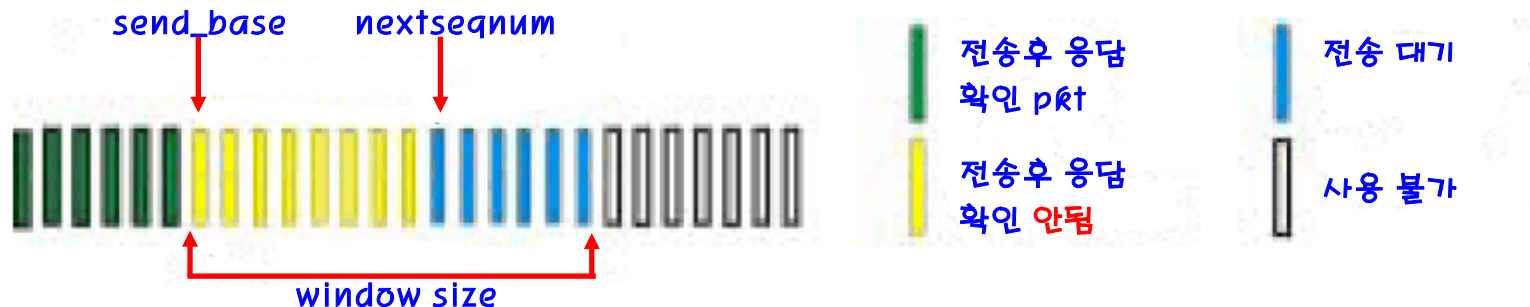
Pipelining : 성능의 향상



$$U_{\text{sender}} = \frac{3 * L / R}{RTT + L / R} = \frac{.024}{30.008} = 0.0008$$

sender

- pkt header에 k-bit seq # 를 갖는다.
- window : 전송되었지만 확인 안된 pkt를 위해 허용 할수있는 seq #의 범위



- ACK(n) : seq # n을 가진 ACK를 cumulative ACK로 인식한다.
 - 수신측에서 보면 n을 포함한 n까지의 모든 pkt에 대한 ACK이다.
- 가장 오래된 수신확인 안된 timer를 단일 timer로 사용한다.
- Timeout 발생시 : 송신되었으나 ACK가 없는 모든 pkt를 재전송한다.

(window내에 전송된 모든 pkt)

sender

timer

- sendbase에 단일 timer

상위 layer에서 Data가 수신되면

- pkt의 다음 seq#를 검사 window내에 있으면 pkt를 전송한다.

timeout(n)

- sendbase에서부터 윈도우 내에 모든 패킷을 재전송한다.

ACK(n) 이 수신

- $n < \text{sendbase}$: 무시.
- $n \geq \text{sendbase}$:
 - sendbase를 윈도우내에 미확인 패킷중 가장 오래된 패킷으로 이동
 - 윈도우내에 미발송 패킷 전송

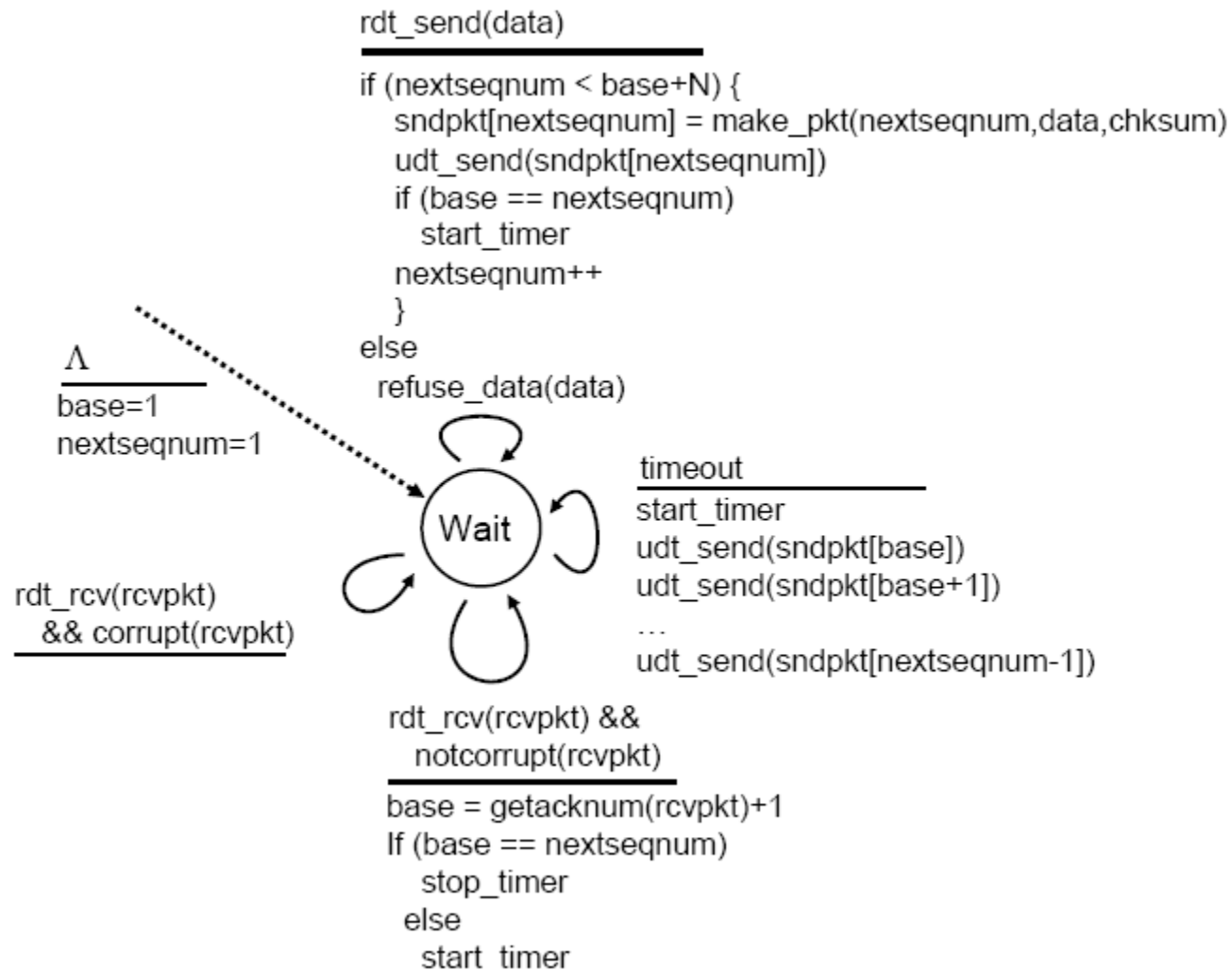
receiver

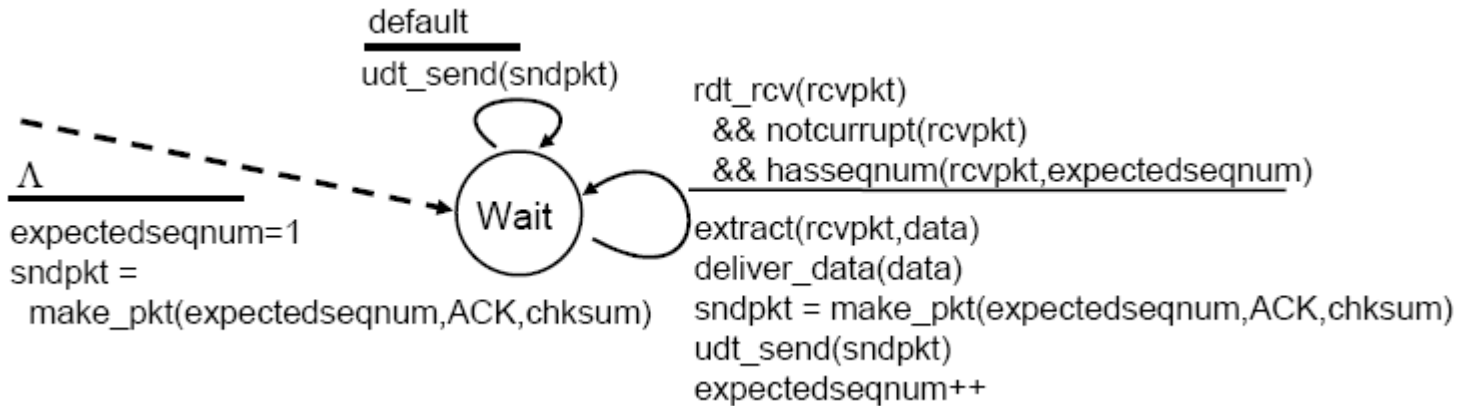
pkt n [n in rcvbase, rcvbase+N-1]

- 누적 ACK을 송신
- out-order인 경우 : 버림.
- in-order인 경우 : 정상 처리

이외의 경우

- 무시한다.





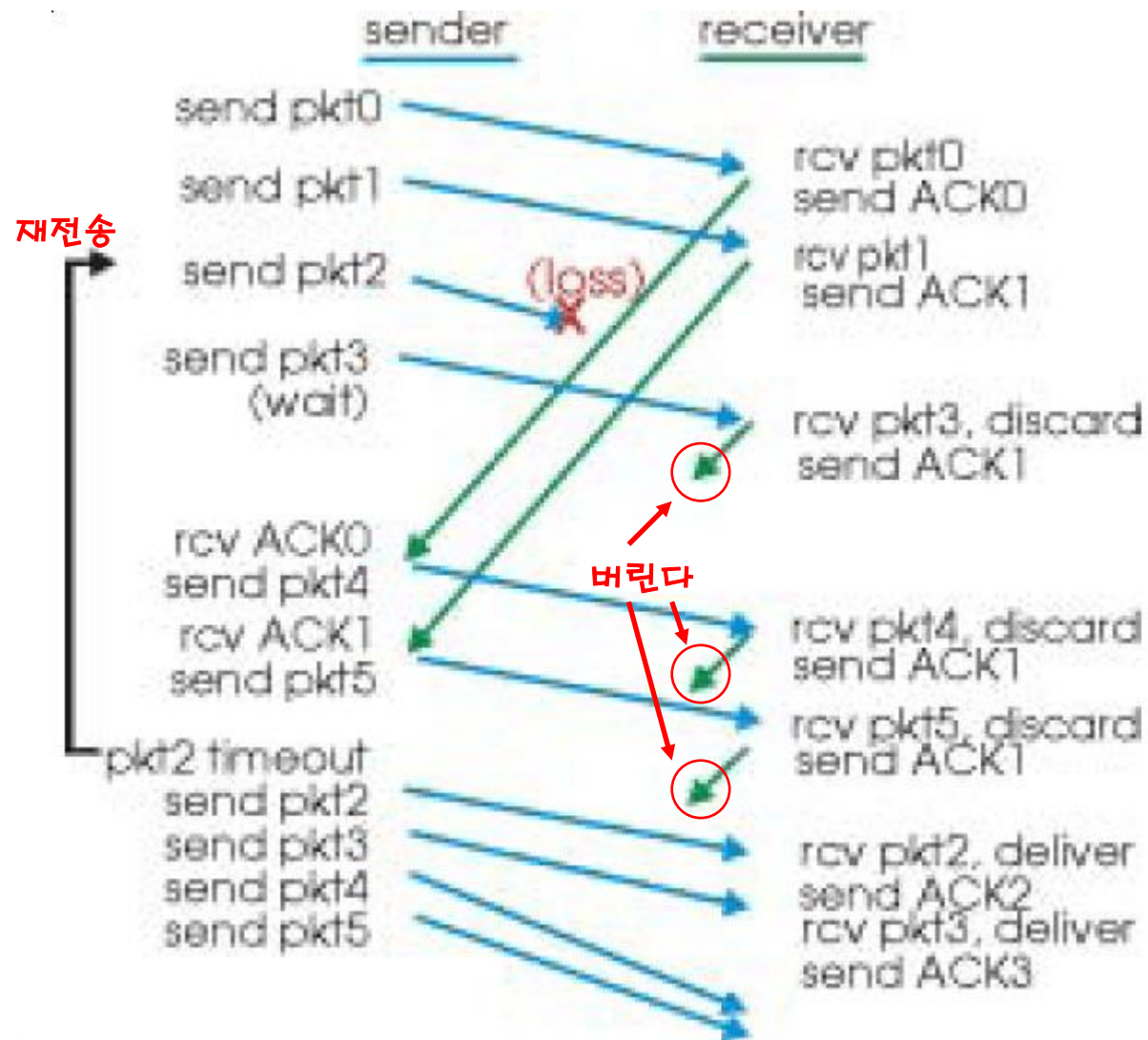
ACK만 사용 : pkt가 수신될때 마다 항상 현재까지 수신된 가장 높은 seq#를 가진 pkt에 대한 ACK를 전송한다.

- 중복된 ACK가 발생 할 수 있다.
- 단지 expectedseqnum만을 유지한다.

: 현재까지 수신된 pkt의 seq#의 다음 seq#

➤ 순서가 잘못 수신된 pkt의 처리

- 강 버린다. : 순서가 잘못된 pkt에 대해서 buffering 할 필요가 없다.
- 현재까지 수신된 가장 큰 seq#에 대한 ACK를 재전송한다.



- receiver가 수신한 pkt에 대한 개별적인 ACK가 요구 된다.
 - 순서가 틀린 pkt들은 하위 seq#를 가진 pkt가 수신 될때까지 buffering되었다가 seq# 순서대로 상위 layer에 전달된다.
- sender는 ACK가 없는 pkt에 대해서만 재전송한다.
 - sender의 timer는 ACK가 없는 각각의 pkt에 대해서 유지된다.
- sender 의 window
 - send_base pkt에 대한 ACK를 받아야 이동한다.
 - : GBN에서는 누적된 ACK를 이용 함으로 send_base pkt에 대한 ACK가 없어도 이동가능하다.

Selective repeat : sender receiver windows



sender

상위 layer에서 Data가 수신되면

- pkt의 다음 seq#를 검사 window내에 있으면 pkt를 전송한다.

timeout(n)

- pkt n을 재전송하고 timer를 restart한다.

ACK(n) 이 수신

- n이 윈도우에 있다면 pkt가 수신된 것을 확인한다.
- n이 send_base와 같다면 send_base는 가장 작은 seq#를 갖는 미확인 pkt로 이동하고 window내에 이 전송 pkt가 있으면 전송한다.

receiver

pkt n [n in rcvbase, rcvbase+N-1]

- ACK(n)을 송신
- out-order인 경우 buffering한다.
- in-order인 경우 필요하다면 buffer에 저장된 번호가 연속적인 pkt와 함께 상위 layer에 전달하고 rcv_base를 가장 낮은 seq#를 가진 미전송 pkt로 옮긴다.

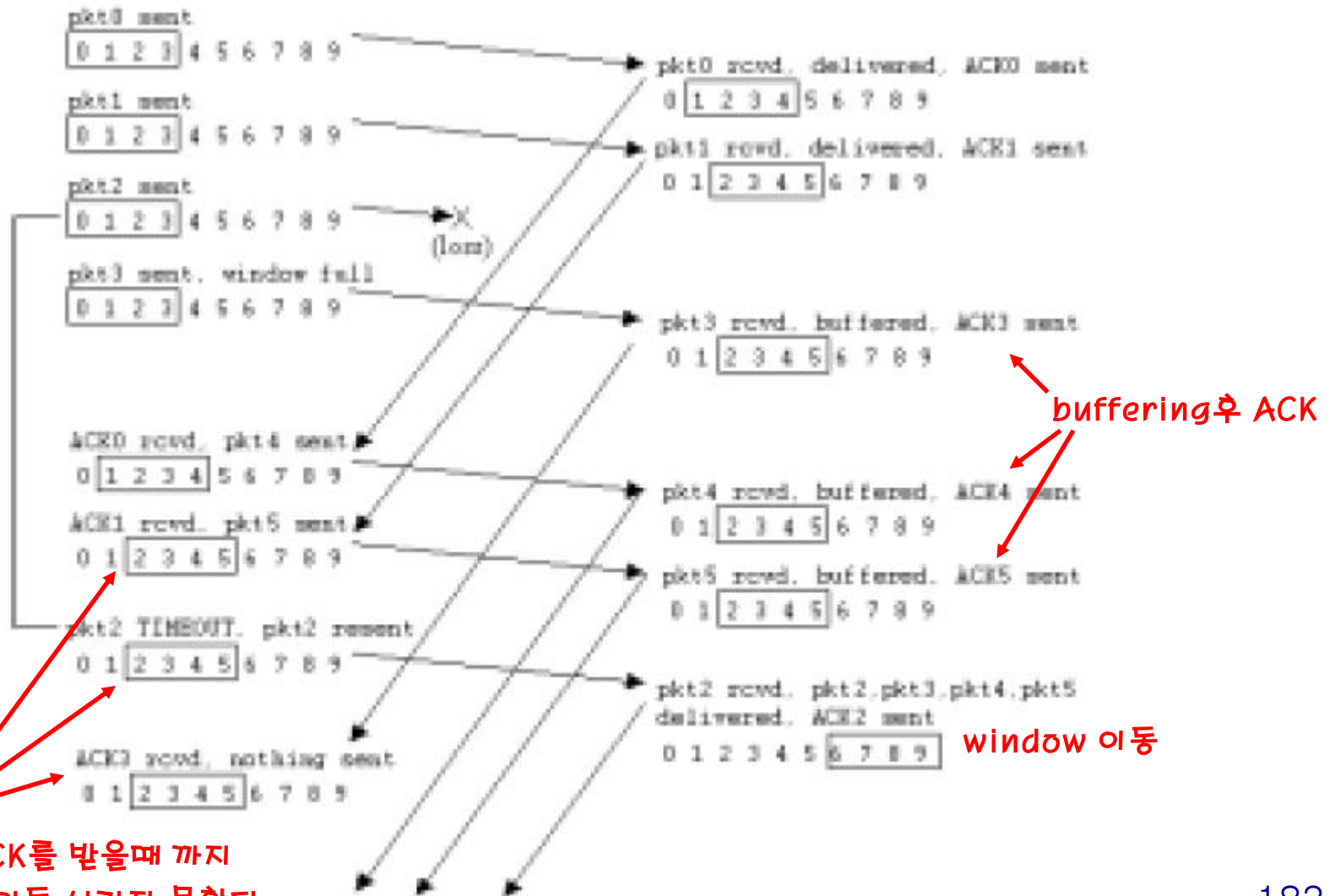
pkt n [n in rcvbase-N, rcvbase-1]

- ACK(n)

이외의 경우

- 무시한다.

Selective repeat in action



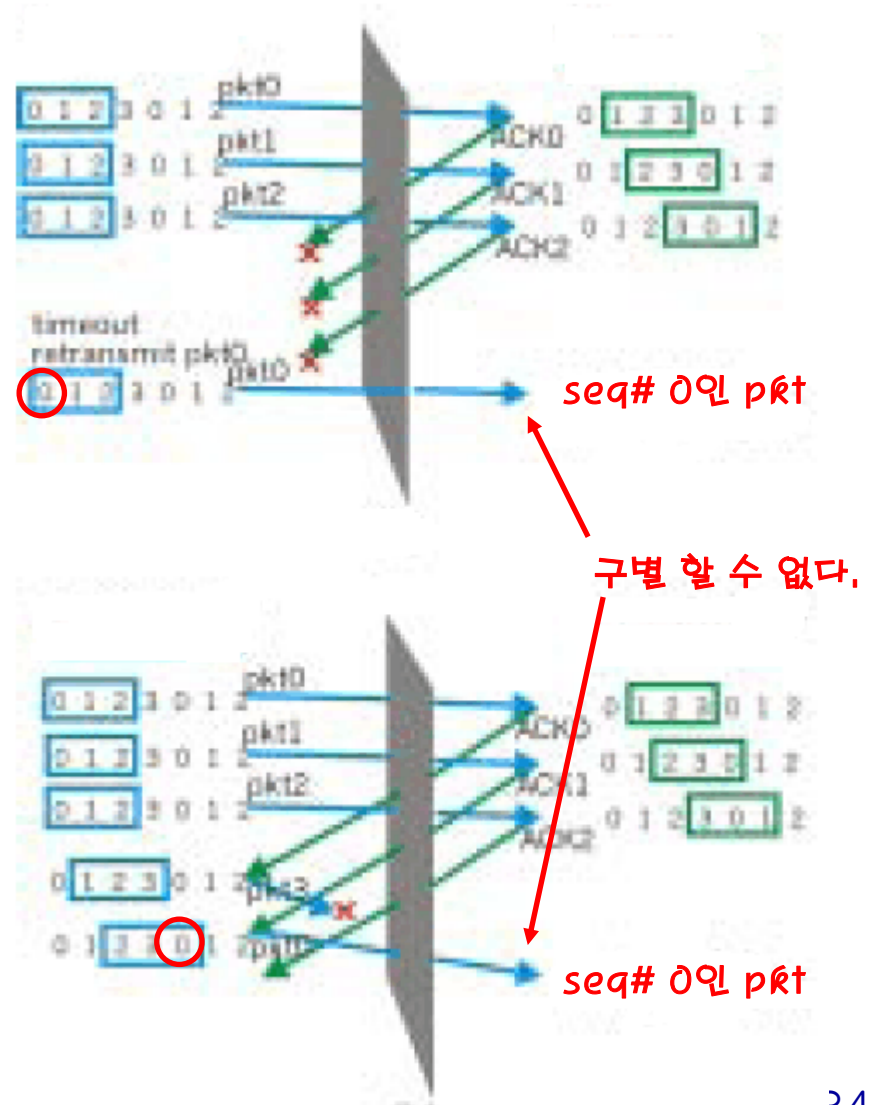
2에 대한 ACK를 받을때 까지
window를 이동 시키지 못한다.

Selective repeat 의 문제

Example

- Seq# : 0, 1, 2, 3
- windows size = 3
- 왼쪽 두개의 경우 모두 seq#가 0으로 구별

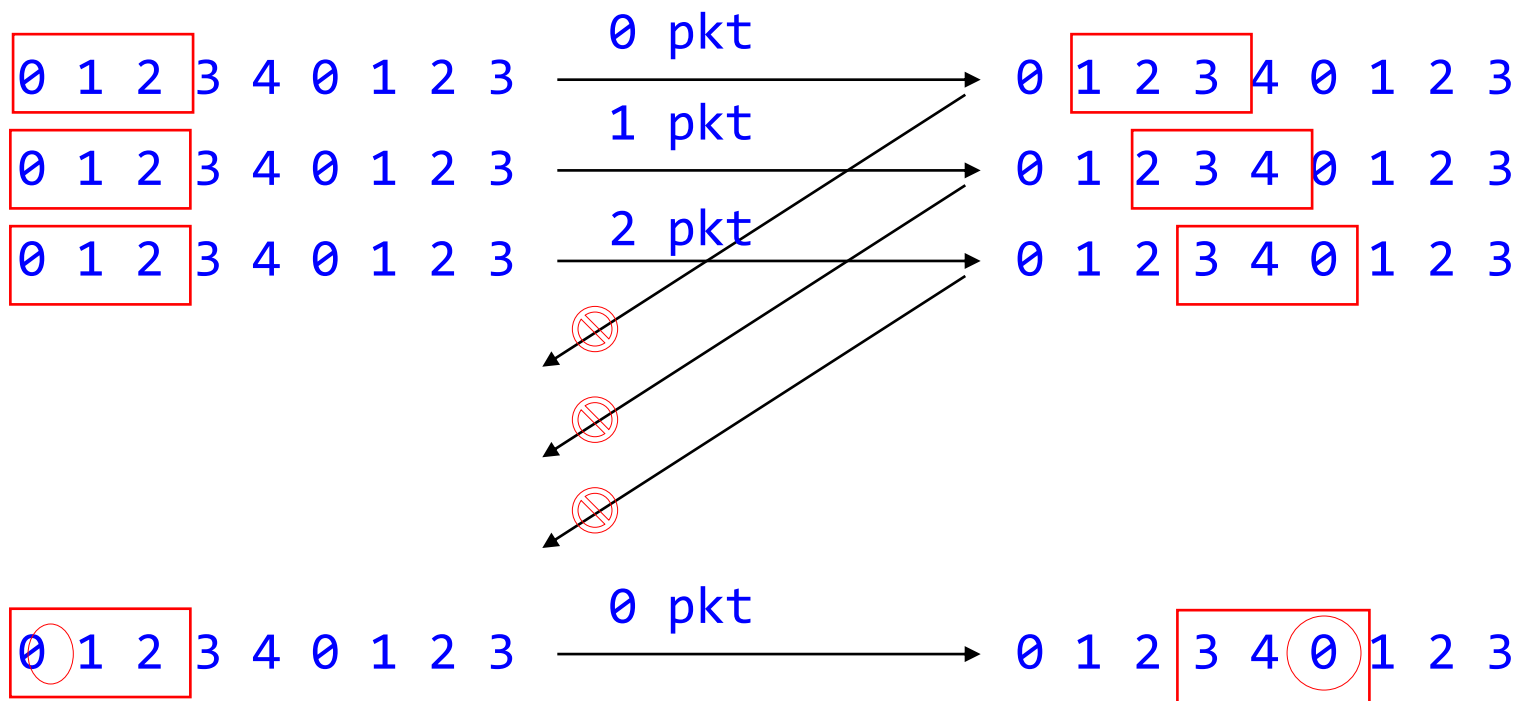
Q; windows size와 seq#의 관계를 어떻게
windows size를 seq#의 개수의 절반



Selective repeat 의 문제

Example

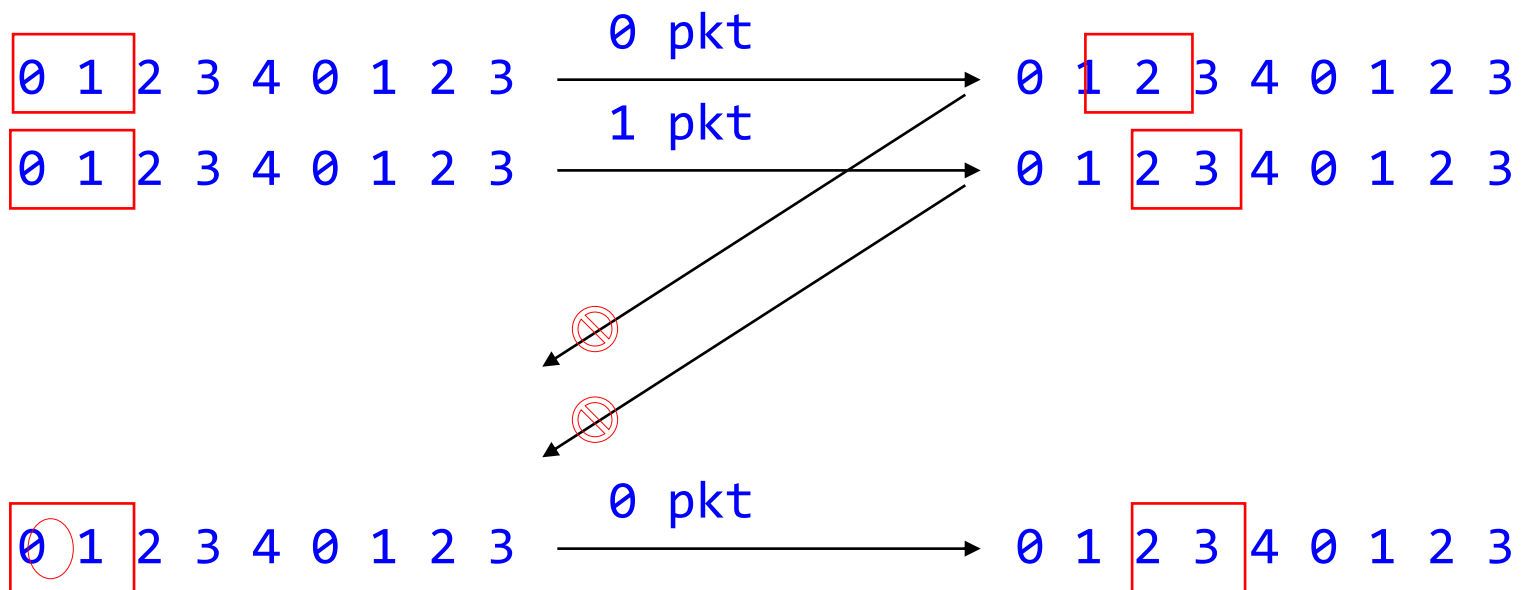
- Seq# : 0, 1, 2, 3, 4
- windows size = 3



Selective repeat 의 문제

Example

- Seq# : 0, 1, 2, 3, 4
- windows size = 2



4장 Transport Layer

4.1 Transport Layer 서비스 개요

4.2 다중화와 역다중화

4.3 Connectionless transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

- segment structure
- reliable data transfer
- flow control
- connection control

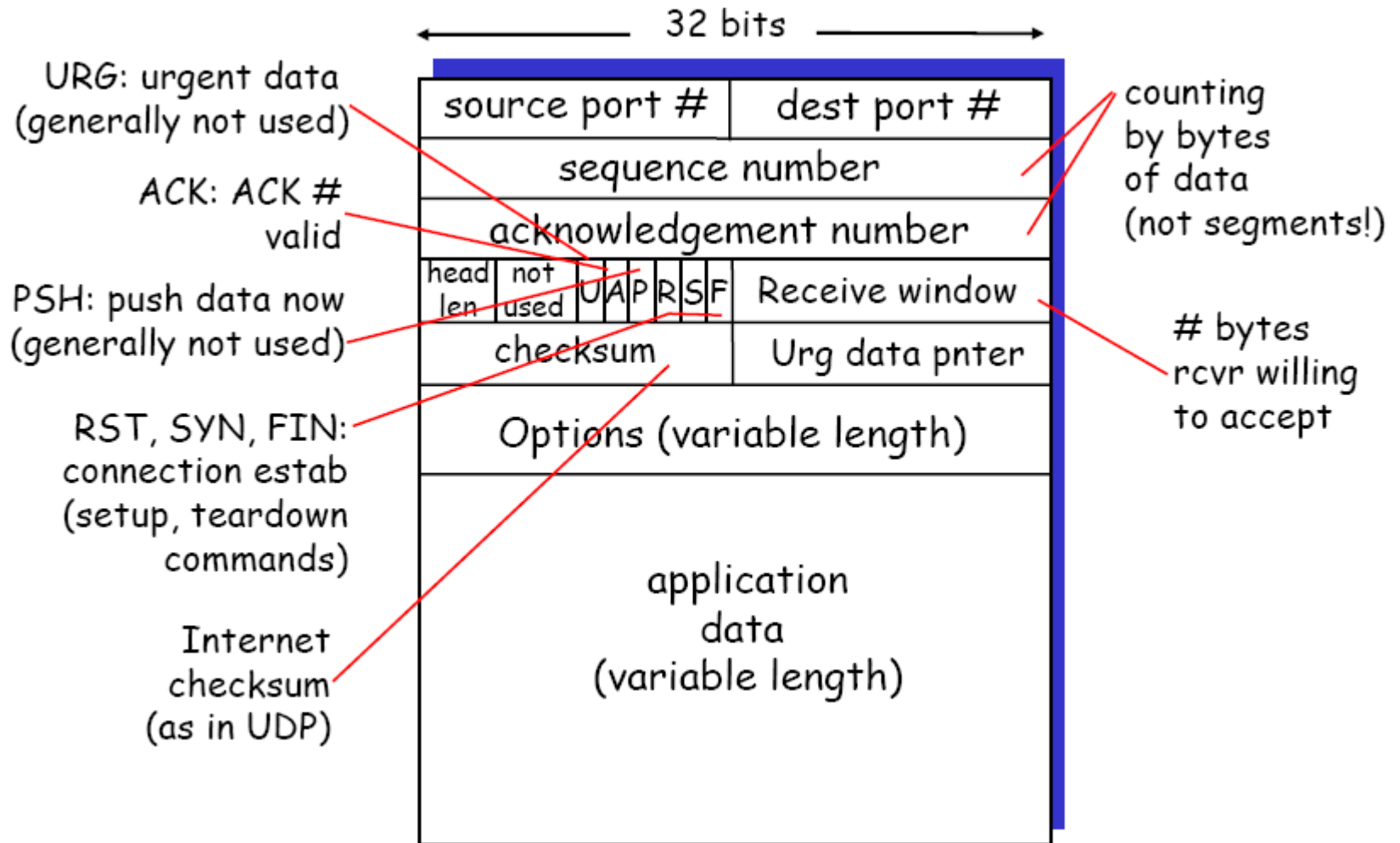
4.6 혼잡제어의 원리

4.7 TCP의 혼잡제어

- point-to-point
 - 단일 송.수신자 간에 통신
 - 신뢰적인 in-order byte stream
 - Message에 구분이 없다.
 - pipelined
 - 혼잡제어나 흐름제어를 통해 window size를 제어한다.
 - 송,수신측은 buffer를 갖는다.
- full duplex
 - 동일 connection에 양단이 동시에 data를 전송 할 수 있다.
 - MSS : maximum segment size (segment에서 app layer data의 최대 크기)
 - connection oriented
 - handshake를 먼저 수행한다.
 - Flow control
 - sender가 receiver를 압박하지 못하도록 receiver가 전송량을 통제한다.



TCP segment의 구조



TCP seq#와 ACK

➤ seq#

- Segment의 첫번째 byte의 stream에서의 byte 순서 번호

➤ ACKs:

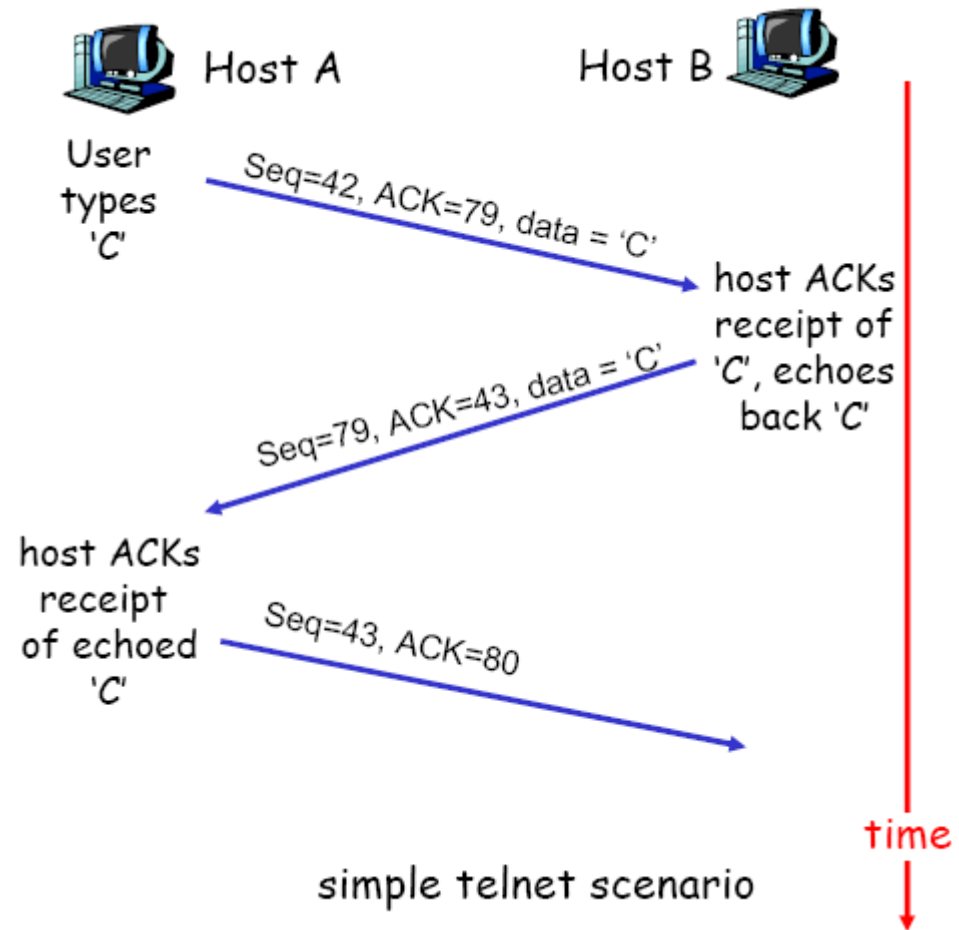
- 다음에 받을 첫번째 byte의 순서 번호
- Cumulative ACK가 가능하다.

Q : 순서가 틀린 segment를 수신한 경우 어떻게 할까??

A : RFC에서는 어떤 규칙도 없다.

기본적으로 두가지 방법이 있다.

- 즉시 버린다.
- buffering한다. (중간에 빠진 data를 받기 위해..)



TCP : RTT(Round Trip Time) & Timeout

Q : TCP의 timeout는 어떻게 설정 할 까???

- RTT보다 커야한다.
 - but RTT 값은??
- 너무 작은 경우
 - 불필요한 재전송이 일어난다.
- 너무 큰 경우
 - Segment 손실에 대한 대응이 늦어진다.

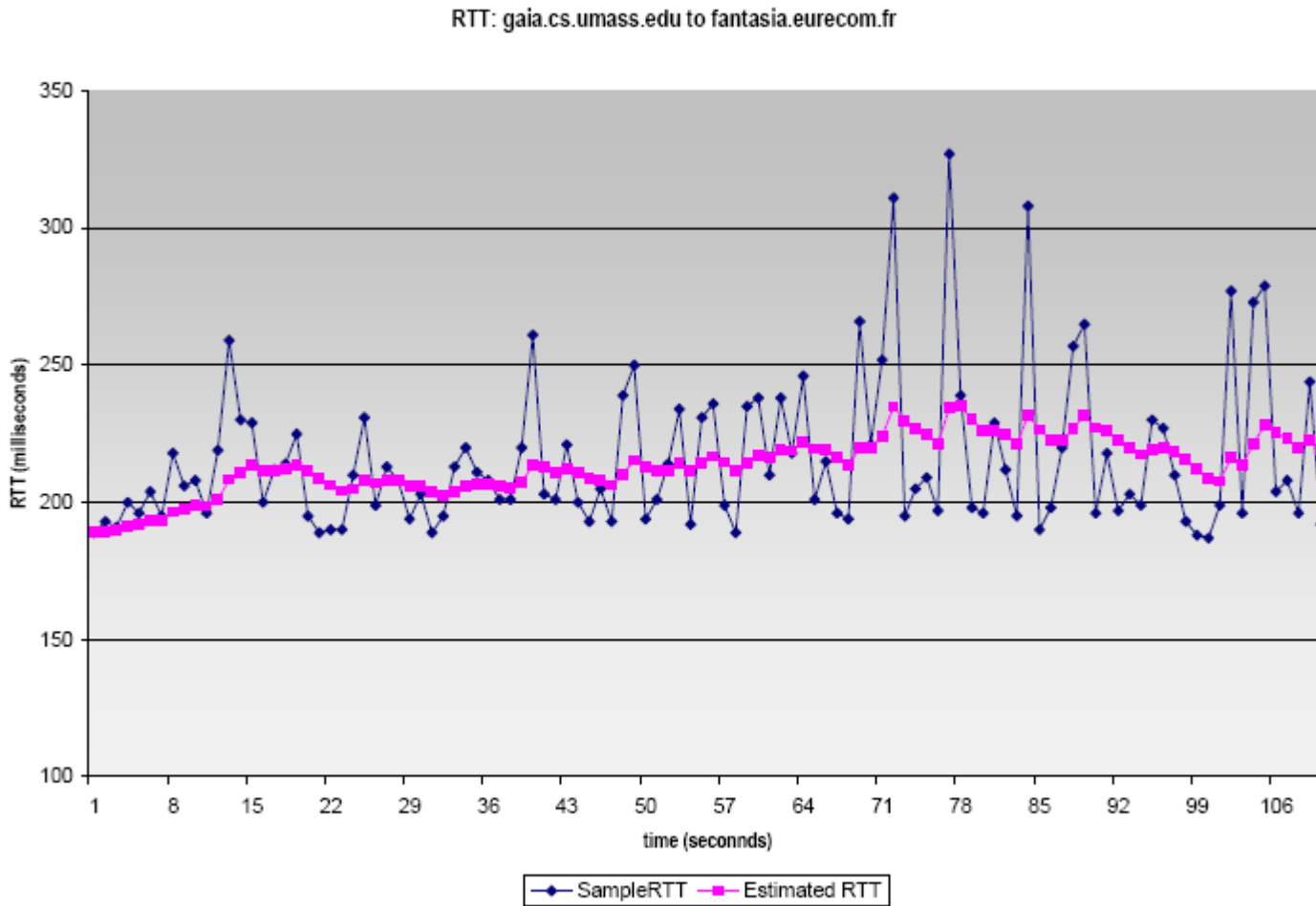
Q : EstimatedRTT는 어떻게 설정 할 까???

- SampleRTT
 - : segment가 송신된 시간으로 부터 ACK가 도착한 시간간격
 - 재전송 segment는 무시된다.
- EstiamteRTT는 매우 smooth하게 변경된다.
 - EstiamatedRTT는 SampleRTT의 가중 평균 값이다.

TCP : RTT(Round Trip Time) & Timeout

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- Exponential weighted moving average
- influence of past sample decreases exponentially fast
- typical value: $\alpha = 0.125$



TCP : RTT(Round Trip Time) & Timeout

timeout 설정

- EstimatedRTT보다 약간의 여유값을 더한 값으로 설정
 - SampleRTT의 변화량이 큰경우는 따라서 커져야한다. (반대의 경우도 동일)
- DevRTT
 - : SampleRTT가 EstimatedRTT로 부터 얼마나 벗어나는지에 대한 예측

$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(β 의 권장값 : 0.25)

실제 timeout 설정

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$

4장 Transport Layer

4.1 Transport Layer 서비스 개요

4.2 다중화와 역다중화

4.3 Connectionless transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

- segment structure
- reliable data transfer
- flow control
- connection control

4.6 혼잡제어의 원리

4.7 TCP의 혼잡제어

TCP reliable data transfer

- TCP는 비신뢰적인 IP 상위에서 신뢰적인 통신 서비스를 제공한다.
- Pipelined segment
- 누적 ACK
- TCP는 단일 재전송 timer를 사용한다.
 - 전송 segment 각각에 대한 timer는 개념적으로는 쉽지만 overhead가 크다.
- 재전송을 일으키는 이벤트
 - timeout
 - 중복 ACK
- 간소화된 TCP sender
 - 중복 ACK 무시
 - 혼잡제어, 흐름제어 무시

App로 부터 data 수신 :

- seq#를 가진 segment 생성
- seq#는 첫번째 바이트의 바이트스트림 번호이다.
- timer 시작
(다른 segment에서 의해서 timer가 실행중이 아닐때)
- timer의 만료 주기
TimeoutInterval

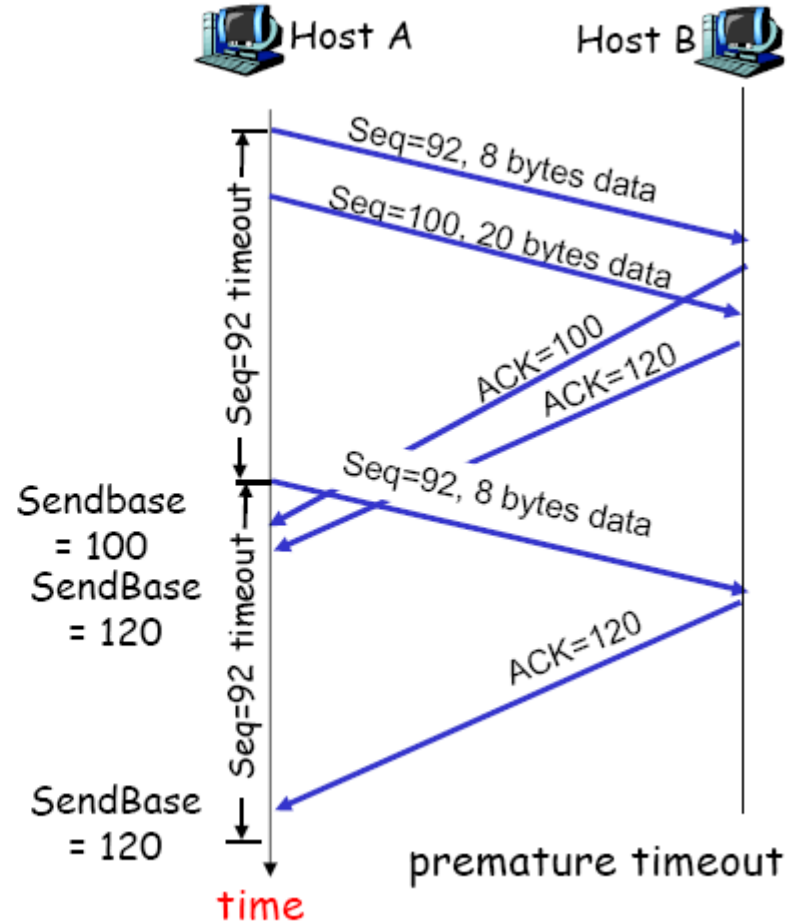
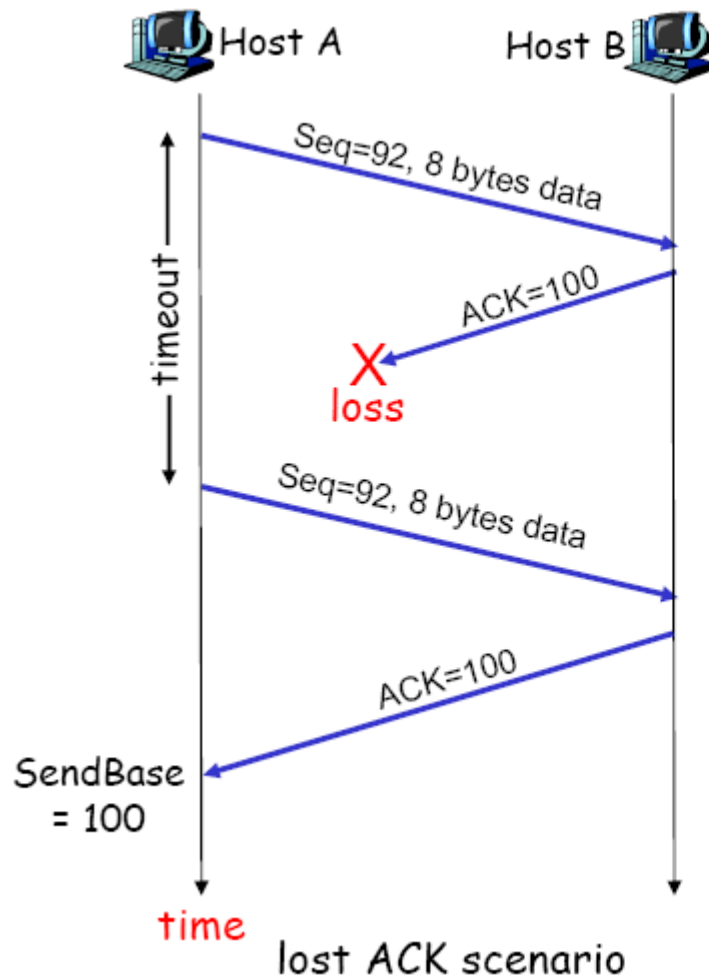
timeout :

- timeout을 일으킨 segment를 재전송
- timer 재시작

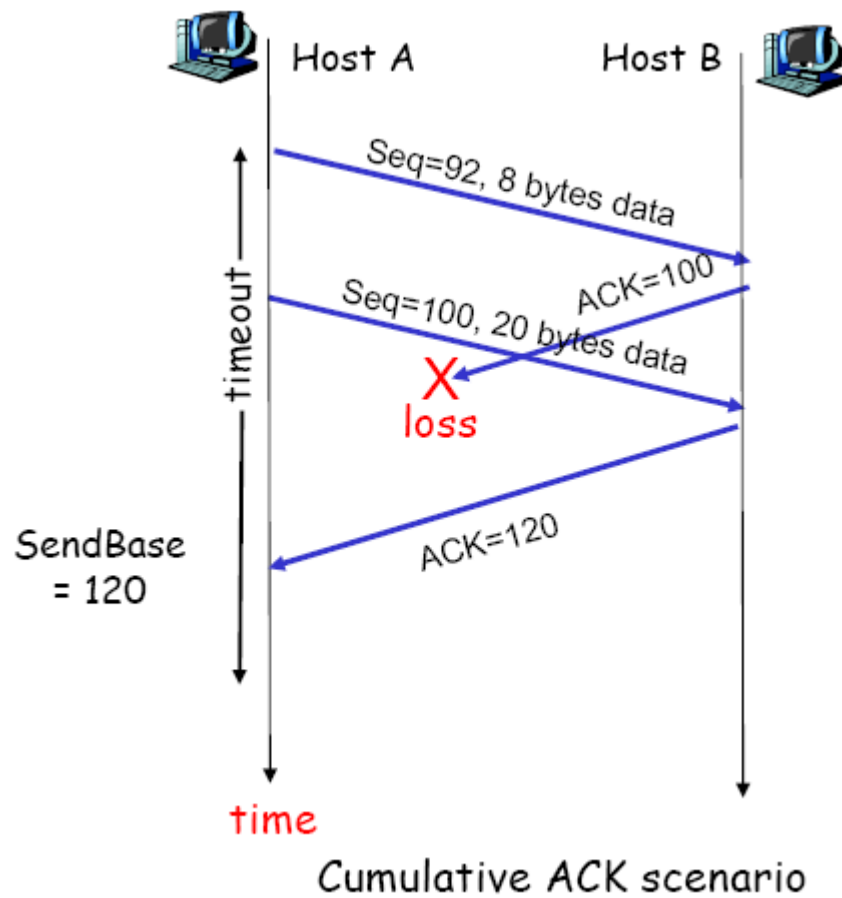
ACK :

- ACK가 확인 안된 segment의 ACK인 경우
 - ACK의 seq#가 sendbase보다 크면 이전에 확인 응답안된 segment에 대해 확인하고 sendbase를 갱신한다.
 - 아직 확인 응답 안된 segment가 존재하면 timer를 재시작한다.

TCP의 흥미로운 시나리오



TCP의 흥미로운 시나리오

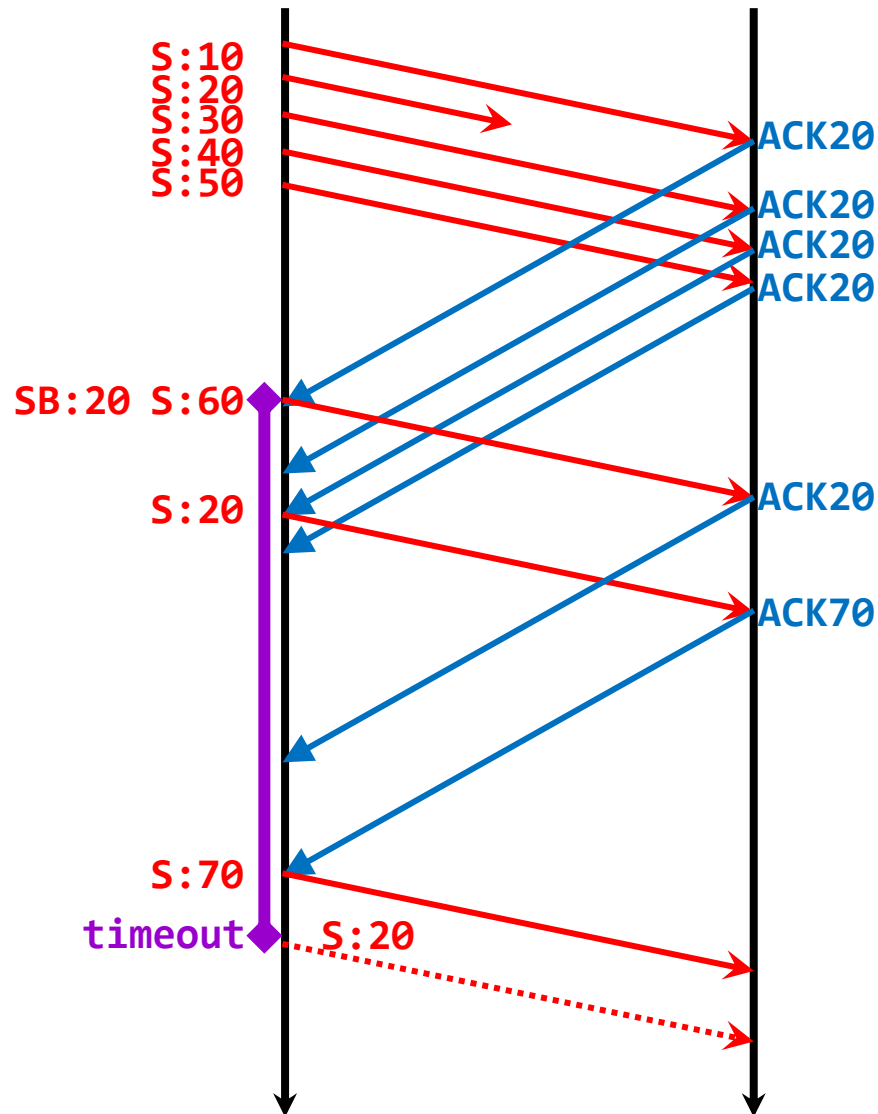


TCP의 ACK생성 권고

Receiver의 event	Receiver의 동작
기다리는 seq#를 가진 segment도착. 이미 기다리는 seq#까지의 모든 segment 수신	지연된 ACK. 다음 순서의 segment를 500ms 기다렸다가 도착하지 않으면 ACK를 전송한다.
기다리는 seq#를 가진 segment도착. 이전에 수신한 seq에대한 ACK는 아직 전송전	하나의 누적된 ACK를 전송
기다리는 것보다 높은 순서의 seq#를 가진 segment가 도착. Gap이 발견	즉시 중복된 ACK전송 (기다리는 다음 순서의 seq#)
수신 segment의 gap부분에 해당하는 segment 도착	즉시 ACK전송 (segment가 gap의 낮은 쪽에서 시작하도록)

- timeout이 유발하는 재전송의 문제
 - 주기가 비교적 길어서 segment를 잃어 버렸을 때 송신자를 기다리게 해서 종단간의 지연이 증가한다.
- 중복된 ACK와 segment loss
 - 수신자는 기다리는것보다 큰 seq#의 segment를 수신하면 이를 buffering하고 중복 ACK를 전송한다.
 - segment를 손실하면 많은 중복된 ACK가 전송된다.
- 만일 3번의 중복된 ACK가 sender에게 전송되면 해당 segment가 loss되었다고 판단한다.
 - 이경우 timeout이전이라도 해당 segment를 재전송한다.
 - > fast retransmit

Seq#가 10, 20, 30, 40, ...인 경우 20이 loss 되면 30, 40, 50, ... 을 보낼 때 마다 ACK(20)이 중복해서 sender에게 전달 된다.



Fast retransmit algorithm

```
event: ACK received, with ACK field value of y
  if (y > SendBase) {
    SendBase = y
    if (there are currently not-yet-acknowledged segments)
      start timer
  }
  else {
    increment count of dup ACKs received for y
    if (count of dup ACKs received for y = 3) {
      resend segment with sequence number y
    }
  }
```

a duplicate ACK for
already ACKed segment

fast retransmit

4장 Transport Layer

4.1 Transport Layer 서비스 개요

4.2 다중화와 역다중화

4.3 Connectionless transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

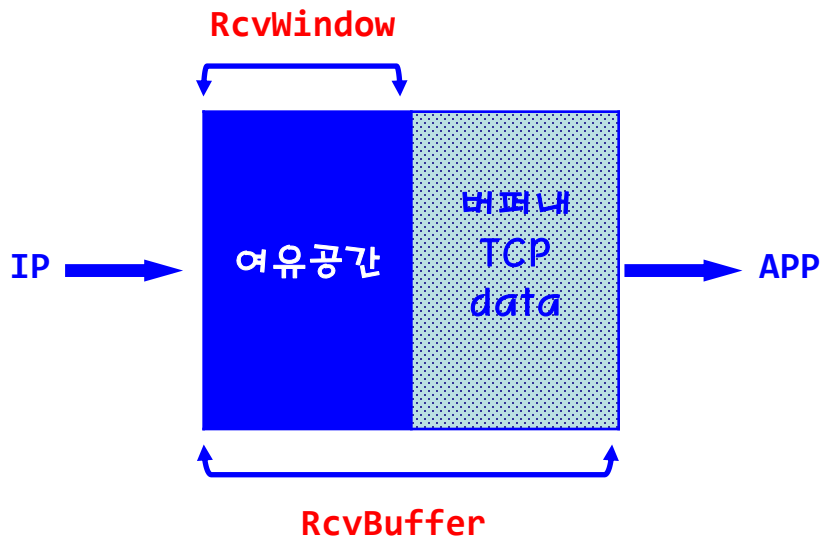
- segment structure
- reliable data transfer
- flow control
- connection control

4.6 혼잡제어의 원리

4.7 TCP의 혼잡제어

TCP Flow control

- TCP connection에서 수신측이 가용한 버퍼의 크기를 알려주어 흐름을 제어한다.



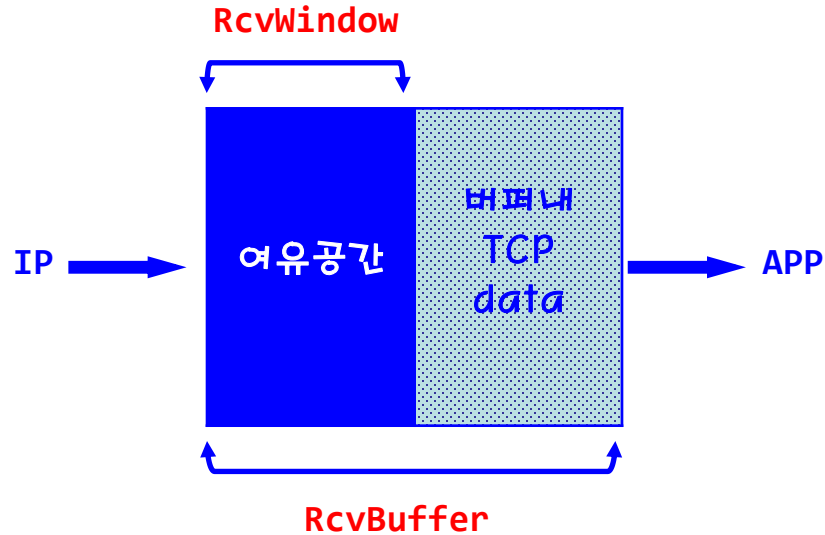
- App process가 수신 buffer로부터 data를 늦게 읽으면 RcvWindow가 줄어들 수 있다.

flow control

Sender가 receiver의 buffer가 overflow 되도록 너무 빠르게 data를 전송하지 못하도록 하는것

- 흐름제어는 속도를 일치시키는 서비스이다.
 - Receiver쪽 app이 data를 읽는 속도와 sender의 전송 속도를 일치시키는 것이다.

TCP Flow control



- segment내에 RcvWindow size를 저장 receiver가 sender에게 알려 줌
- Sender는 RcvWindow크기보다 적은 확인 응답된 data량을 유지함으로써 수신 buffer에 overflow가 발생하지 않았음을 확신한다.

(out-of-order segment는 버린다고 가정)

- 버퍼내에 여유 공간
= RcvWindow
= RcvBuffer - [LastByteRcvd - LastByteRead]

4장 Transport Layer

4.1 Transport Layer 서비스 개요

4.2 다중화와 역다중화

4.3 Connectionless transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

- segment structure
- reliable data transfer
- flow control
- connection control

4.6 혼잡제어의 원리

4.7 TCP의 혼잡제어

Recall: TCP sender와 receiver는 connection을 초기화한 다음 data segment을 전송한다.

- TCP 초기화 변수
 - Seq#, buffer, flow control 정보들
ex) RcvWindow
- client : connection initiator
- server : client의 접속을 승인

Three way handshake:

step 1: client가 server에게 TCP SYN segment를 전송

- 초기 seq# 설정
- data는 없다.

step 2: server는 SYN를 수신하고 SYNACK를 전송

- server : 변수및 buffer 할당
- server의 초기 seq# 설정

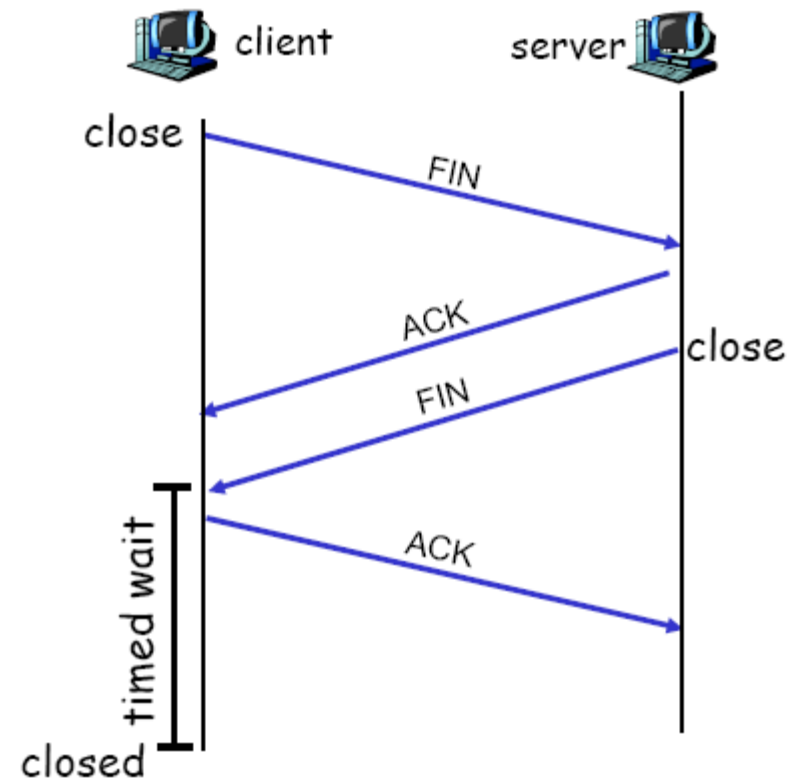
step 3: client는 SYNACK수신 ACK전송 (data를 추가해도 된다.)

- client : 변수및 buffer 할당

Closing a connection:

Step 1: client는 connection 을 종료하기위해 FIN bit가 1로 설정된 TCP segment를 server에게 전달

Step 2: server는 FIN을 수신하면 ACK를 응답하고 connection을 종료한다는 FIN를 client에게 전송한다.

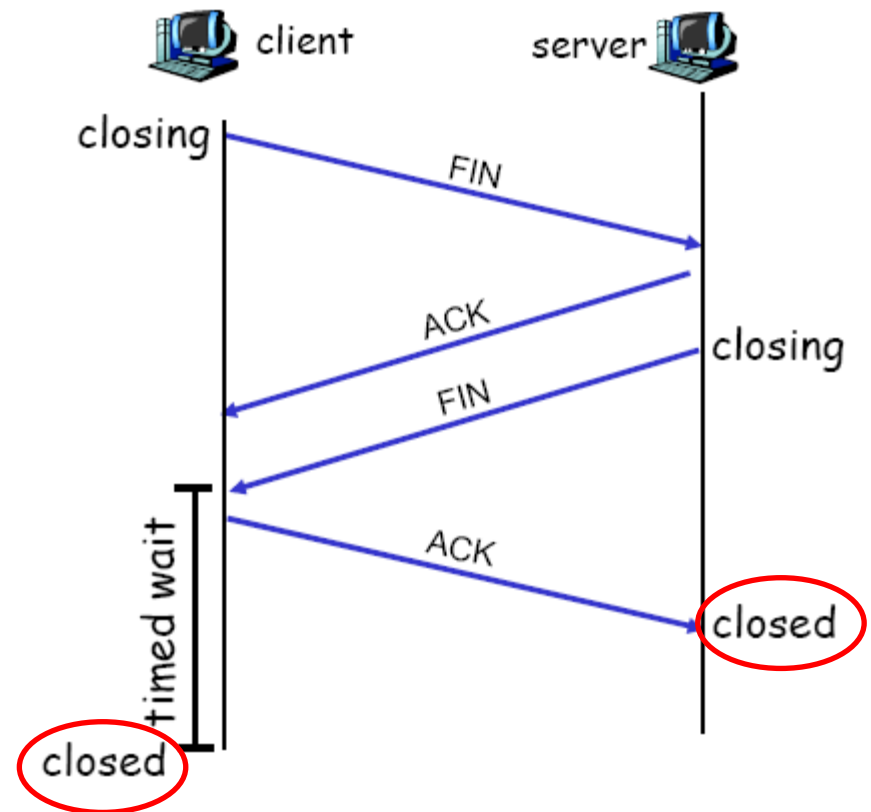


TCP의 connection 관리

Step 3: client는 FIN을 수신하면 ACK를 응답하고 일정시간을 기다린후 (time wait) connection을 종료한다.

Step 4: server는 ACK를 수신하면 connection을 종료한다.

Note : with small modification can handle simultaneous FINs.



4장 Transport Layer

4.1 Transport Layer 서비스 개요

4.2 다중화와 역다중화

4.3 Connectionless transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

- segment structure
- reliable data transfer
- flow control
- connection control

4.6 혼잡제어의 원리

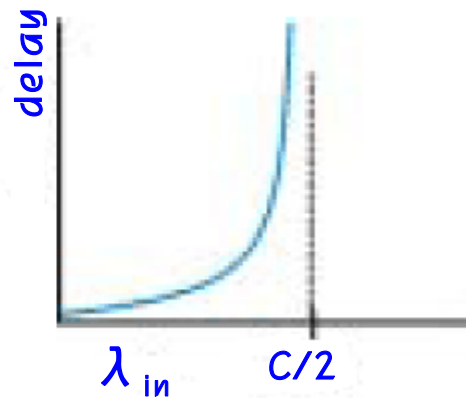
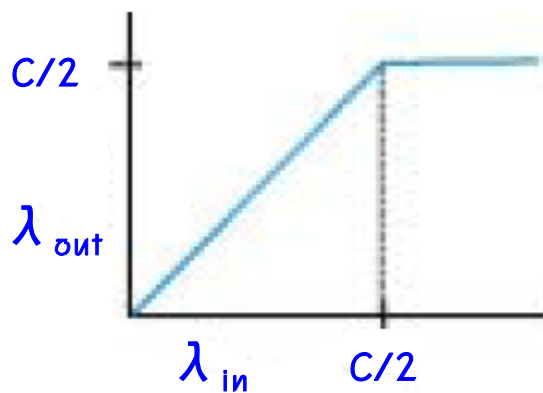
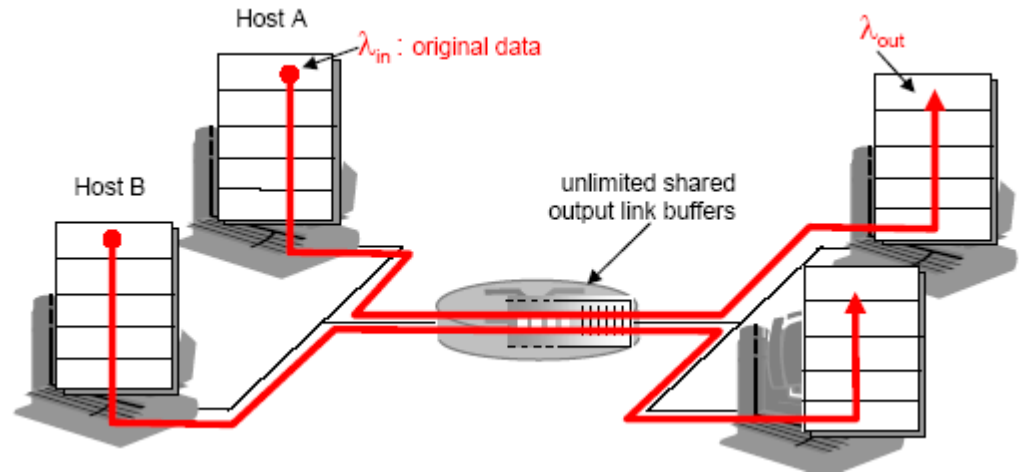
4.7 TCP의 혼잡제어

Congestion :

- 높은 전송률로 data를 전송하려는 많은 근원지로 인한 문제
- flow control과는 다르다.
- 현상
 - pkt의 손실 (router buffer의 overflow)
 - 지연의 증가 (router buffer에서 queueing delay 증가)
- Sender에 의해서 조정
 - 실제 혼잡도를 조사하는것은 아니다 (TCP)
 - ATM등과 같이 혼잡도를 조사하기 위해서는 네트워크 장치들의 협조가 필요함
- 다음과 같은 경우 Data 전송량을 줄인다.
 - timeout이 빈번히 일어날때
 - 재전송량이 많아 질때

congestion scenario 1

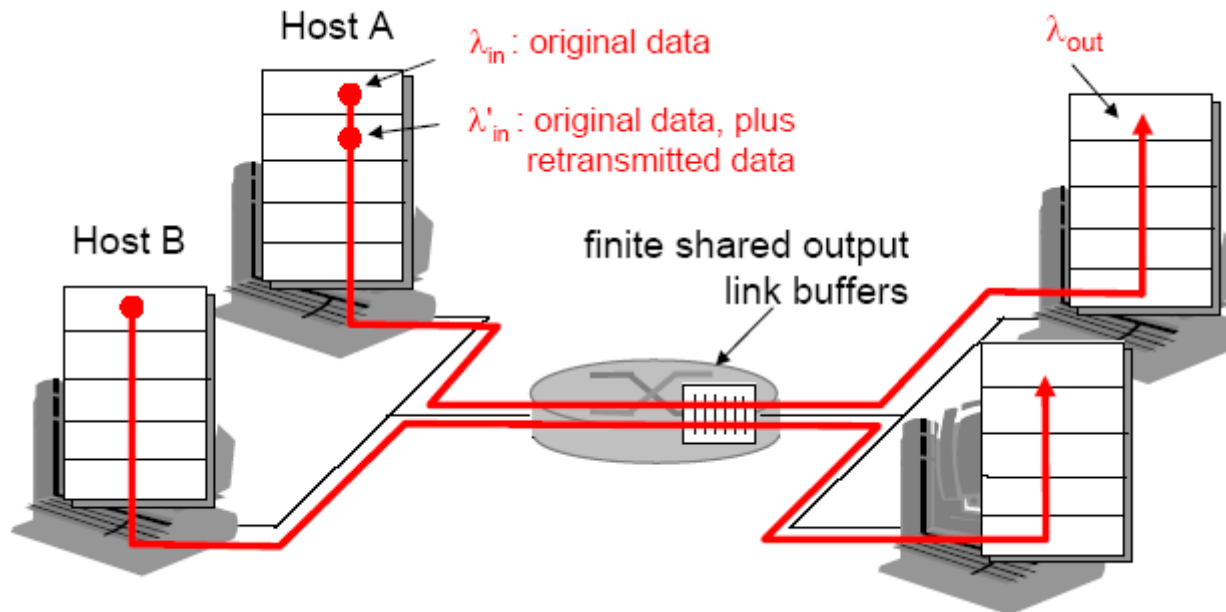
- 두개의 sender와 reciver
- 무한 버퍼를 가진 하나의 라우터
- 재전송은 없다.



- 혼잡시에 매우 큰 지연이 있다.
- 패킷의 처리량이 링크의 용량에 접근하게 되면 매우 큰 queueing delay가 발생한다.

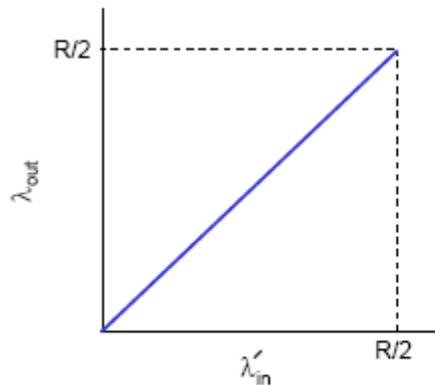
congestion scenario 2

- 유한 buffer를 가진 라우터
- 손실된 pkt에 대한 재전송이 이루어진다.

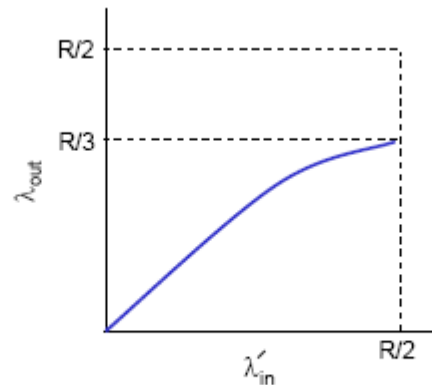


congestion scenario 2

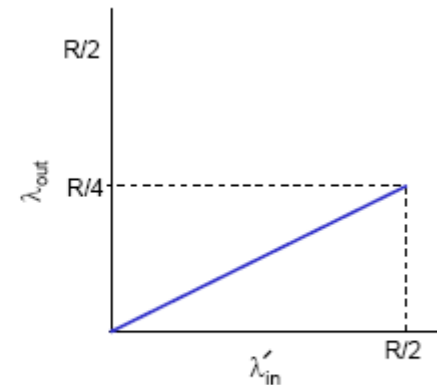
- 손실이 발생하지 않는 이상적인 경우 : $\lambda_{in} = \lambda_{out}$ (그림 a)
- 손실된 pkt만 재전송하는 perfect한 경우 : $\lambda'_{in} > \lambda_{out}$ (그림 b)
- pkt의 delay로 인한 불필요한 재전송으로 λ'_{in} 이 매우 커지고 λ_{out} 이 더욱 줄어든다. (그림 c)



a.



b.



c.

congestion의 cost

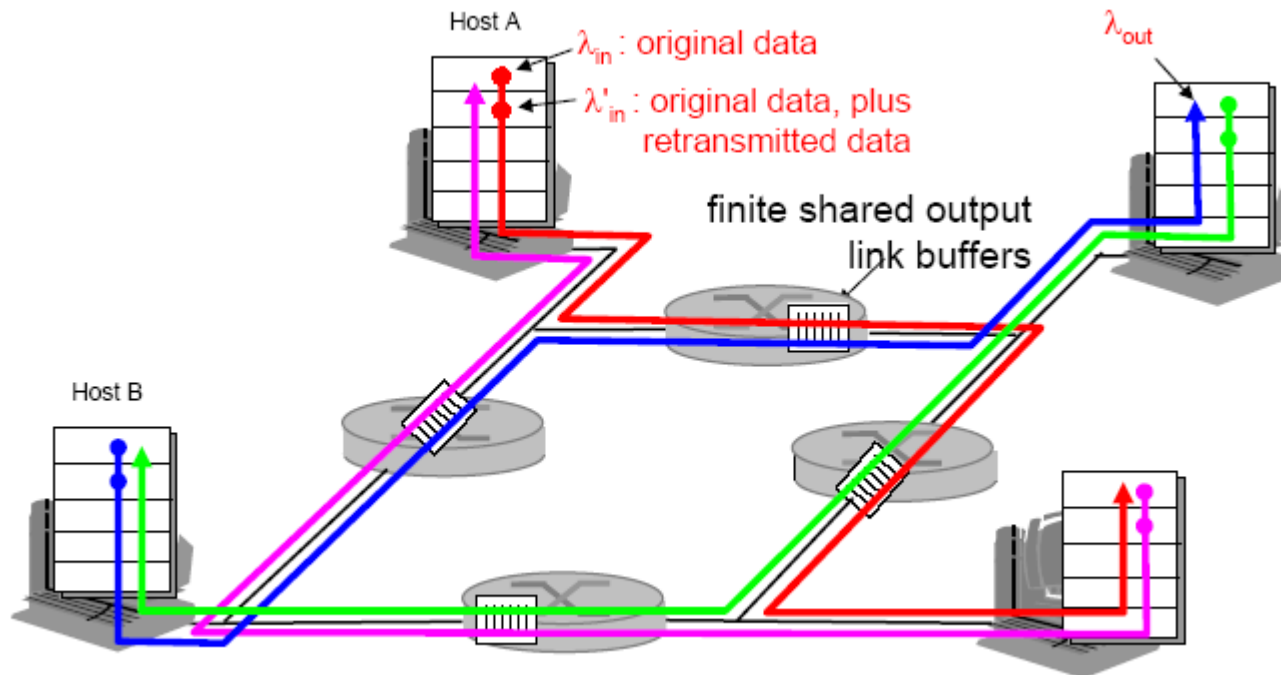
- Buffer overflow로 인해 버려진 pkt에 대한 재전송
- 큰 지연으로 인한 불필요한 재전송

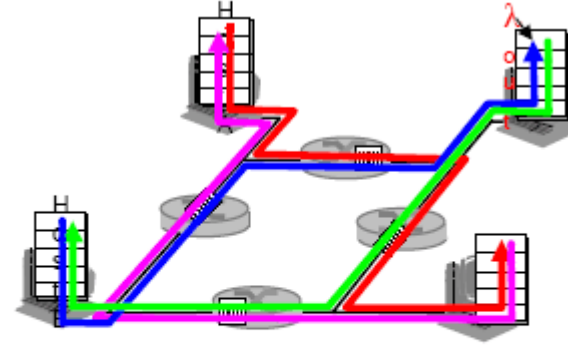
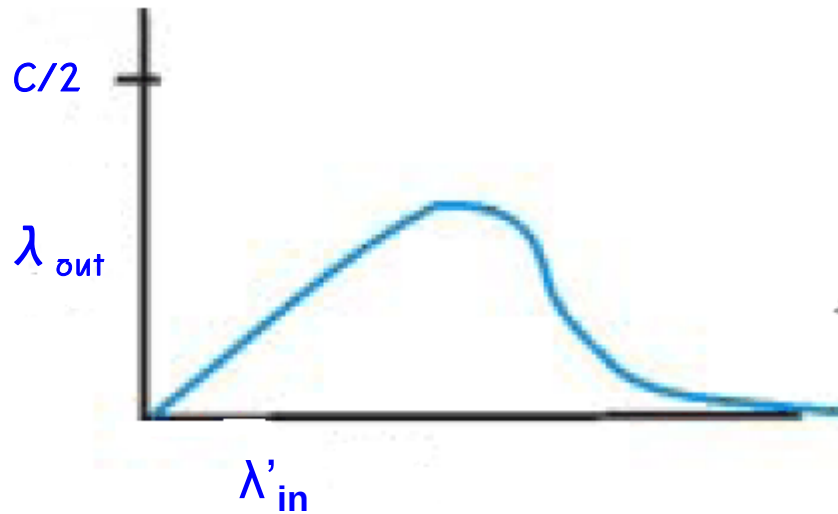
congestion scenario 3

- 4개의 sender
- 여러 개의 라우터와 다중 경로
- timeout/재전송

Q : λ_{in} 이나 λ'_{in} 가 커지면 어떤 현상이 일어나는가??

A : 다음 장의 도표를 참고





congestion의 또하나의 cost

- 혼잡으로 인해 pkt가 경로상에서 버려질때 버려지는 지점까지 pkt를 전송하는데 사용된 router의 전송 용량은 쓸모없는것이 되었다.

4장 Transport Layer

4.1 Transport Layer 서비스 개요

4.2 다중화와 역다중화

4.3 Connectionless transport : UDP

4.4 신뢰적 data 전송의 원리

4.5 Connection oriented transport : TCP

- segment structure
- reliable data transfer
- flow control
- connection control

4.6 혼잡제어의 원리

4.7 TCP의 혼잡제어

TCP congestion control

- end-end control (network 으로부터 지원은 없다)

- sender의 전송량을 제한

$\text{LastbyteSend} - \text{LastbyteAcked} \leq \text{CongWin}$ (TCP 수신buffer가 충분히 큰경우 : 혼잡제어만 필요)
[$\leq \min\{\text{congWin}, \text{RcvWindow}\}$]
(혼잡제어) (흐름제어)

- 송신률

$$\text{rate} = \text{CongWin} / \text{RTT}$$
(byte/sec)

- CongWin는 network 혼잡도에 따라 동적으로 변한다.

Sender가 congestion에 대한 반응

- 손실이 발생
 - timeout 또는 3번의 중복 ACK
 - 손실에 의한 재전송이 발생
- TCP sender는 손실이 발생하면 CongWin을 줄인다.

혼잡제어 알고리즘

- AIMD
- slow start
- timeout event에 대한 반응

multiplicative decrease

loss event가 발생하면 CongWin의 size를 반으로 줄인다.

- 1MSS 이하로 떨어지지 않는다는.

additive increase

loss event가 없으면 매번 ACK를 확인 할 때 각 RTT마다 congWin의 증가폭이 1MSS가 되도록 조금씩 congWin을 증가 시킨다.

-> 이러한 선형 증가를 혼잡 회피라고 한다.



Long-lived TCP connection

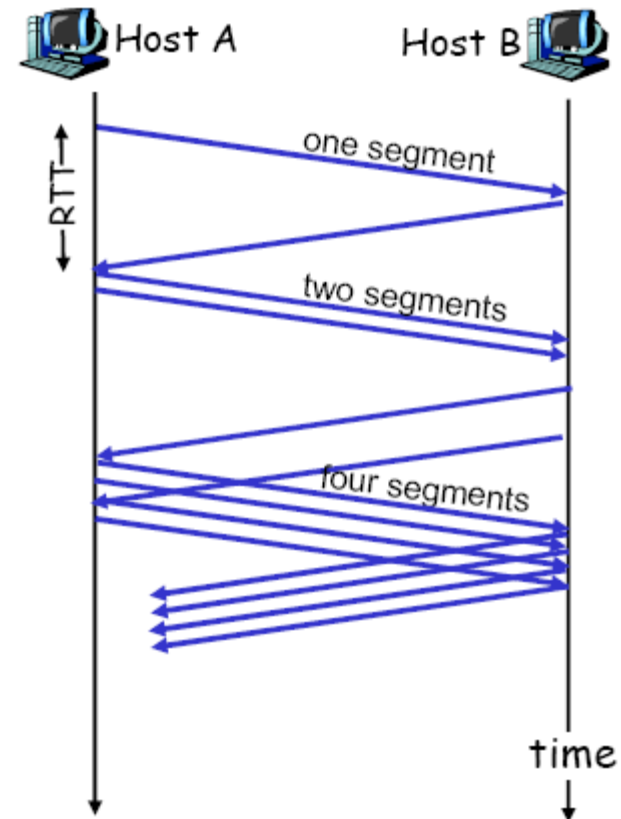
- TCP연결이 시작될때 CongWin의 크기는 1 MSS로 초기화 되고 초기 전송률은 MSS/RTT가 된다.
- 첫번째 loss가 발생할 때 까지 각 ack를 확인할 때 마다 congWin을 1MSS만큼 증가 시켜 각 RTT마다 congWin이 두배가 되도록 한다.
- 사용가능한 대역폭은 MSS/RTT보다 매우 크다
 - 선형비율로 증가시키면 전송률이 적당한 수준에 오를때 까지 긴 지연이 초래된다.

congWin의 증가치

AIMD : 각 RTT마다 1MSS
SS : 각 ACK마다 1MSS
각 RTT마다 두배

TCP Slow start

- 접속이 되면 loss event가 발생할때 까지 지수적으로 속도를 증가 시킨다.
 - 각 RTT마다 CongWin이 두배
 - 모든 ACK가 수신될때마다 CongWin을 증가 시킨다.
- 초기의 속도는 매우 느리지만 지수 적으로 빠르게 증가하는 모델이다.



- 세번의 중복 ACK수신
 - CongWin은 반으로 줄어든다.
 - Congwin은 선형적으로 증가한다.
 - AIMD
- 그러나 timeout이 발생하면
 - CongWin을 1MSS로 줄인다.
 - 지수적으로 CongWin을 증가시킨다.
 - threshold에 도달하면 다시 선형적으로 증가한다.
 - : 혼잡 회피에 들어간다.
- threshold : CongWin의 절반

개념

- 세번 중복된 ACKs는 네트워크가 거의 대역폭 한계에 도달했다고 판단
- timeout은 삼중 ACK보다 심각하게 처리한다.

Q :언제 지수적인 증가가 일어나는가?

A : timeout에 의해 CongWin이 1로
줄어든 이후에 threshold까지

구현

- threshold는 가변적이다.
- Loss event가 일어나면 threshold는 congWin의 1/2로 설정된다.



- CongWin은 threshold에 도달 할때 까지 slow-start방식으로 빠르게 증가한다.
- CongWin이 threshold에 도달하면 혼잡회피를 위해 선형적으로 천천히 증가한다.
- 3번 중복된 ACK가 감지되면 CongWin은 threshold 까지 줄어든고 선형으로 증가한다.
- Timeout이 발생하면 CongWin은 1 MSS로 줄었다가 다시 threshold에 도달 할 때 까지 빠르게 증가한다.